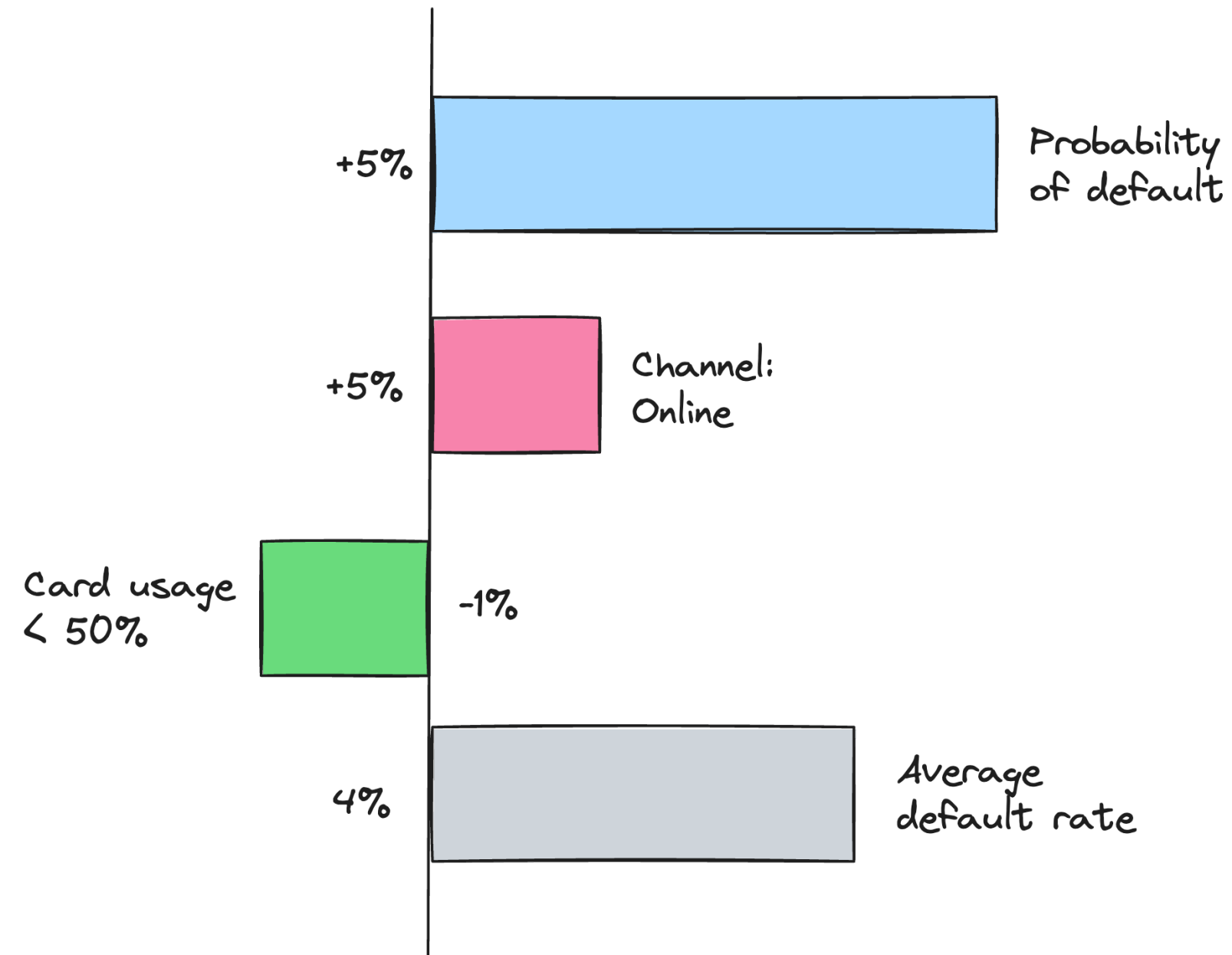


Explainable Credit Risk Models



weight of
evidence.



weight of evidence

Origins

Factor Principle

Credit Scoring

WOE Scorecards

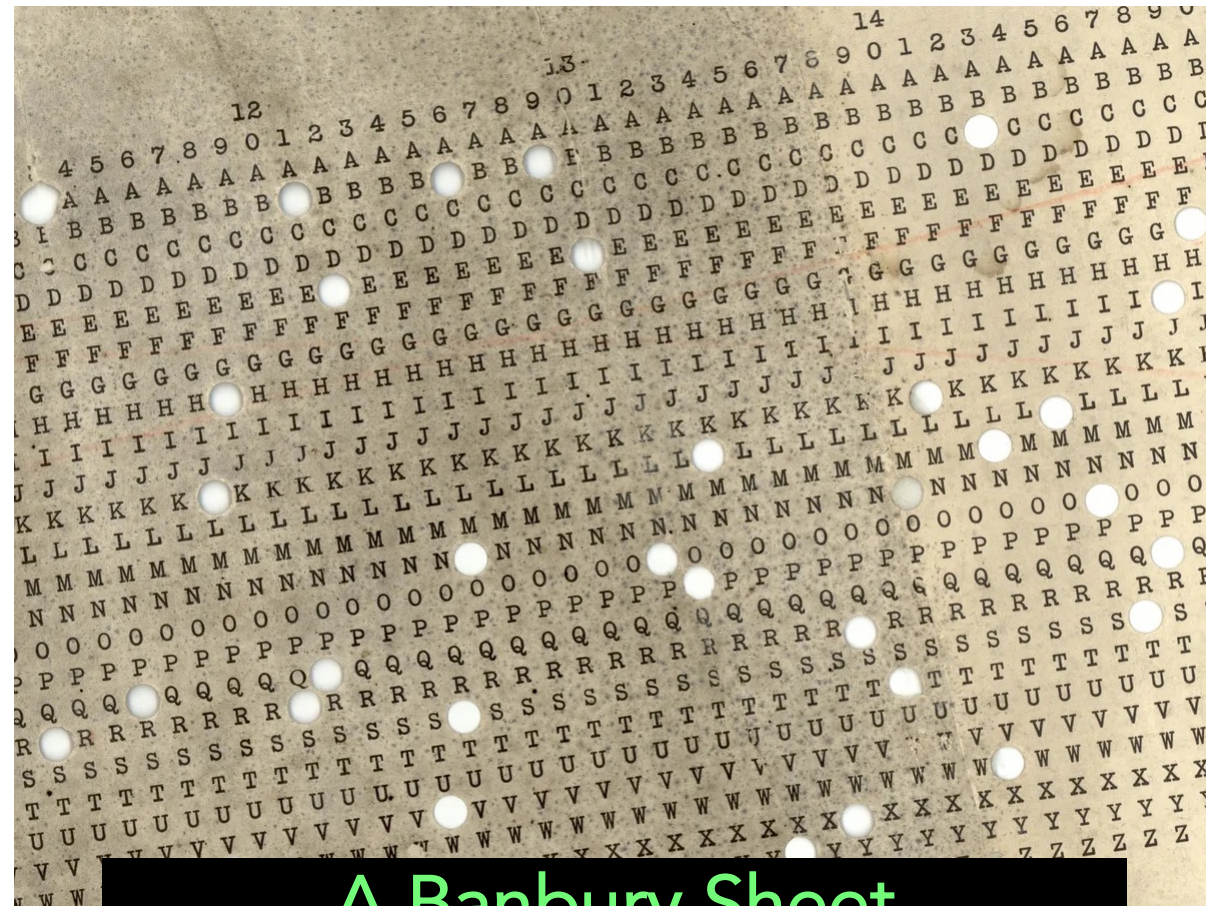


Alan Turing (B. Cumberbatch)
using weights of evidence to
crack the Enigma code

The Imitation Game
(2014, Graham Moore)

$$\log \frac{p(Y|X)}{p(\neg Y|X)} = \log \frac{p(Y)}{p(\neg Y)} + \sum w_i$$

The original scoring mechanism devised by Alan Turing was called **Banburismus** (named after an English town, Banbury), which relied on a computation of log factors (weights of evidence)



A Banbury Sheet

The evidence in favour of particular hypotheses was tallied using sheets of paper that were specially printed in Banbury, a town about 30 miles from Bletchley.

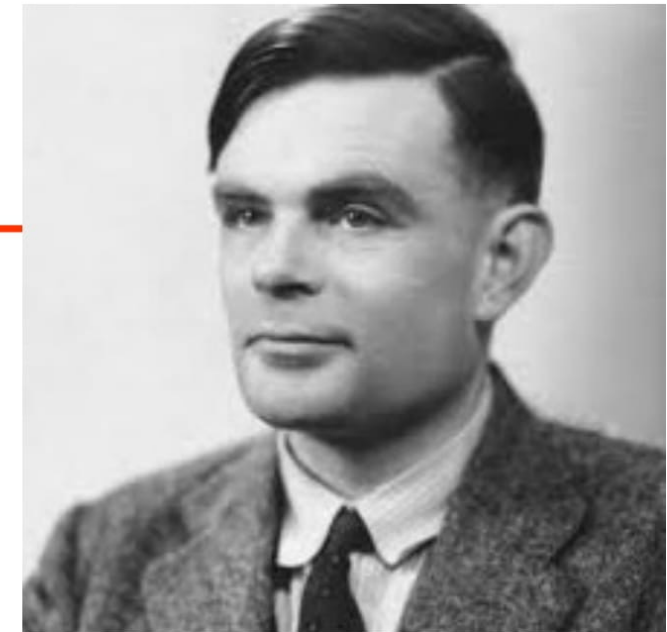
Each hole on the Banbury sheet contributed to a cumulative score, just like modern credit scorecards add up WOE contributions.



Bletchley Park, UK

See Alan Turing's lost notes, found in the walls of Bletchley Park 70 years later ([link](#)).

Alan Turing's approach to explanation



- 'Naive Bayes' classifier:

$$\frac{p(H_0|s_1, \dots, s_p)}{p(H_1|s_1, \dots, s_p)} = \prod_i \frac{p(s_i|H_0)}{p(s_i|H_1)} \times \frac{p(H_0)}{p(H_1)}.$$

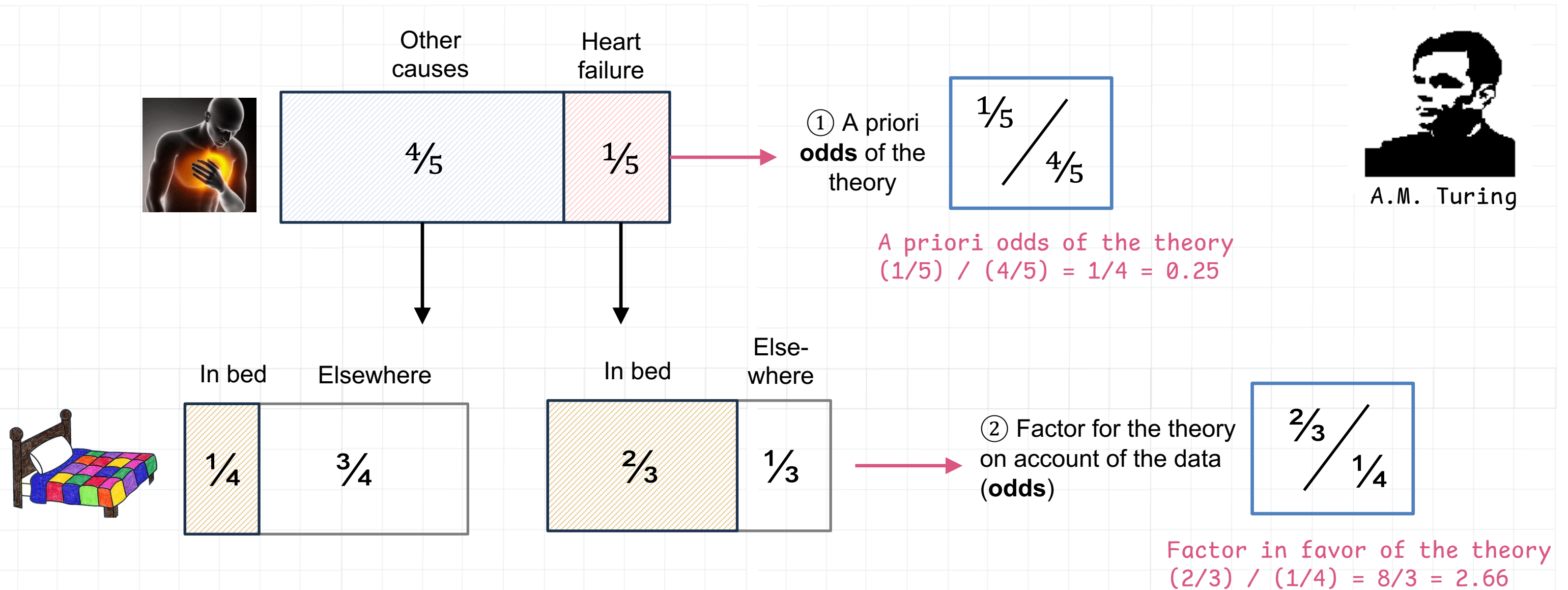
- So

$$\log \frac{p(H_0|s_1, \dots, s_p)}{p(H_1|s_1, \dots, s_p)} = \sum_i w_i + \log \frac{p(H_0)}{p(H_1)},$$

where w_i is the 'weight of evidence' $\log \frac{p(s_i|H_0)}{p(s_i|H_1)}$ (Turing and Good)

- Weights of evidence are positive/negative if evidence s_i is for/against H_0
- (Can use w_i 's as predictors in a logistic regression to improve their calibration)
- Multiplying by 10 and rounding helps explanation

"The theory is that the man died of heart failure, and the data is that he died in his bed"
Example from A.M. Turing The Applications of Probability to Cryptography, 1941



When we use Turing's decibans we get: $\log_{10}(2.66) \times 10 = 4.3$

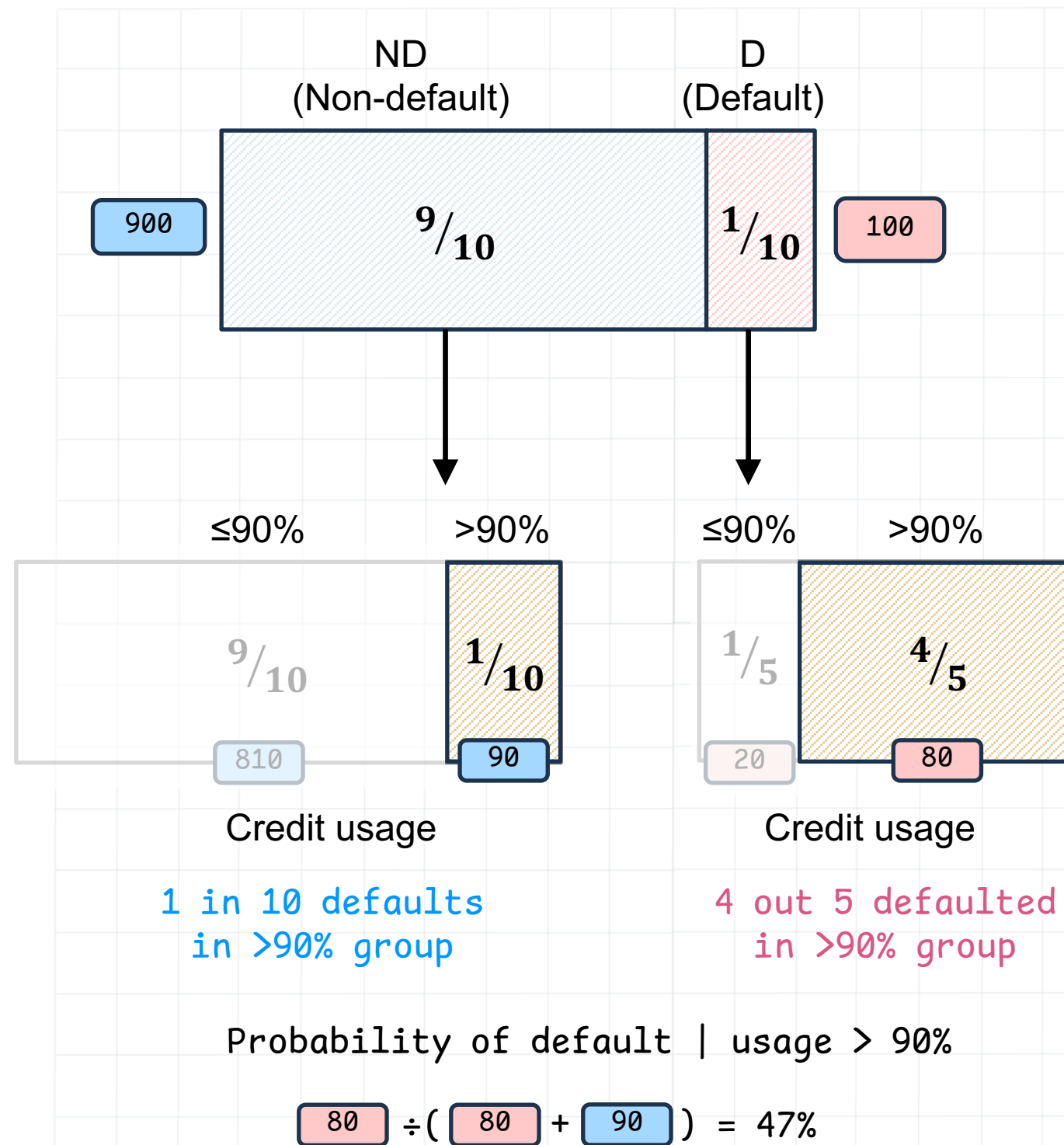
"We score 4.3 decibans in favour of the heart failure theory"

The log factor = Weight of Evidence (WOE)

③ A posteriori odds of the theory
 $0.25 \times 2.66 = 0.67$

Credit Scoring – Credit Card Usage

Estimating probability of default with Weight of Evidence (WOE)



We are interested in the case of credit usage > 90%.
How likely is credit default given this evidence?

A priori odds of default are $(1/10) / (9/10) = 0.11$.

Factor is $(4/5) / (1/10) = 8.0$.

This factor means we have 8 bads for 1 good credit when usage is > 90%.

A posteriori odds of default = $0.11 \times 8.0 = 0.88$.

A posteriori $p(\text{default+}) = 0.88 / (1 + 0.88) = 0.47$.

Weight of Evidence (WOE) = $\log(8.0) = 2.0794$.

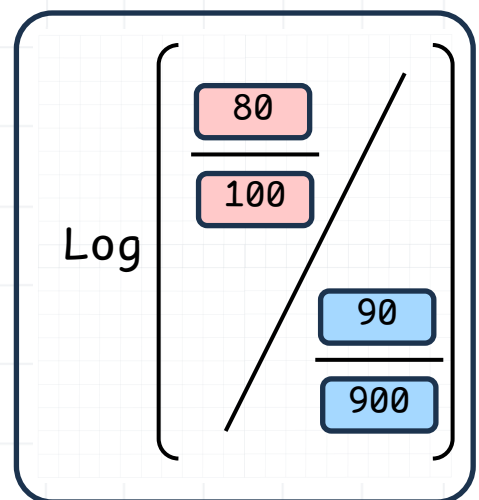
Log (a priori) odds = $\log(0.11) = -2.2073$.

Logit score = $-2.2073 + 2.0794 = -0.128$.

Probability of default = $\sigma(-0.128) = 0.4680$.

$$\sigma = \exp(-0.128) / (1 + \exp(-0.128))$$

WOE (natural log)



Credit Scoring – Credit Card Usage (continued)

Two ways to derive weights of evidence using the Bayes' theorem

2x2 Contingency Table

Credit Usage	Default = 0	Default = 1
≤90%	810	20
> 90%	90	100
Total	900	100

Variant 1 (classic)

$$810 + 20 = 830$$

$$90 + 80 = 170$$

$$900 + 100 = 1000$$

Probability (rate)

Credit Usage	Default rate
≤90%	2.4%
> 90%	47%
Total	10%

Variant 2 (shortcut)

$$20 / 830 = 0.024$$

$$80 / 170 = 0.471$$

$$100 / 1000 = 0.10$$

$$P(\text{default} \mid > 90\%) = 0.471$$

$$P(\text{no default} \mid > 90\%) = 1 - 0.471 = 0.529$$

$$P \text{ in favor of default} = 0.10$$

$$P \text{ against default} = 1 - 0.10 = 0.90$$

$$\frac{(0.47 / 0.529)}{(0.10 / 0.90)} = 8.0$$

Odds (usage >90%) Prior odds



I.J. Good

Conditional Probability Table

Credit Usage	Default = 0	Default = 1
≤90%	0.90	0.20
> 90%	0.10	0.80
Total	1.00	1.00

$$0.2 / 0.9 = 0.22$$

$$0.8 / 0.1 = 8.0$$

$$\text{WOE} = \log(8) = 2.07$$

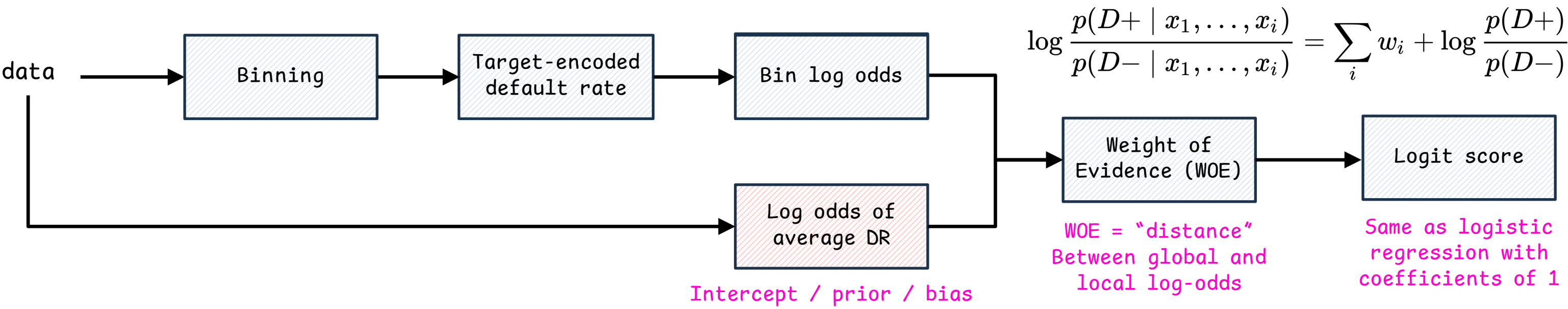
$$\text{WOE} = \log(8) = 2.07$$

credit usage >90% increases the odds of default by a factor of 8x

Real-world pipeline

Naïve Bayes Classifier with WOE

Using weights of evidence to build scorecards



Target encoding for one categorical variable

Acquisition channel	Default (D+)		Acquisition channel	Default (D+)		Acquisition channel
Branch	0	➡	Branch	0	➡	0
Online	1		Online	0		1
Partner	0		Partner	1/3		1/3
Partner	0					1/3
Partner	1					1/3
						1.3

Log-odds = logit

$$\text{logit}(p) = \log\left(\frac{p}{1 - p}\right)$$

Logit transformation transforms probabilities into odds and then takes the natural logarithm, which creates additive scores in the log-odds space.

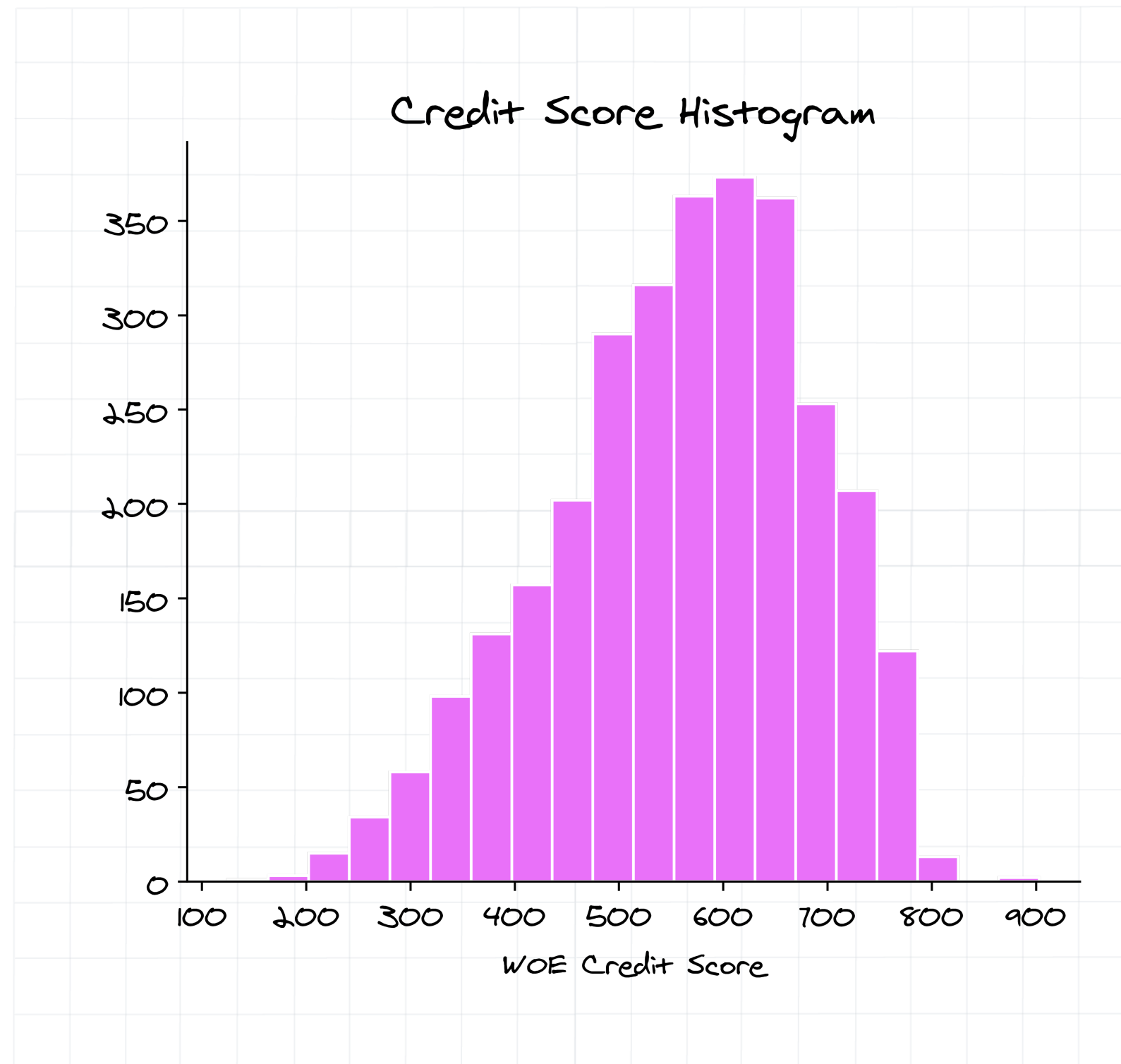
Weight of Evidence (WOE)

$$WOE = \log\left(\frac{DR_{\text{bin } i}}{1 - DR_{\text{bin } i}}\right) - \log\left(\frac{DR_{\text{sample}}}{1 - DR_{\text{sample}}}\right)$$

WOE can be seen as "distance" of group risk from baseline risk level.

Naïve Bayes Classifier with WOE (continued)

Using weights of evidence to build scorecards



The following parameters were used for the transformation of WOE scores into credit scores with points to double the odds (PDO) method:

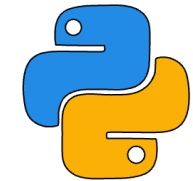
- PDO = 30 (each +30 points doubles the odds of good)
- Target Score = 500
- Odds = 20

Within WOE-based scorecard, we assume all n features to be equally weighted ($1/n$) unlike in logistic regression

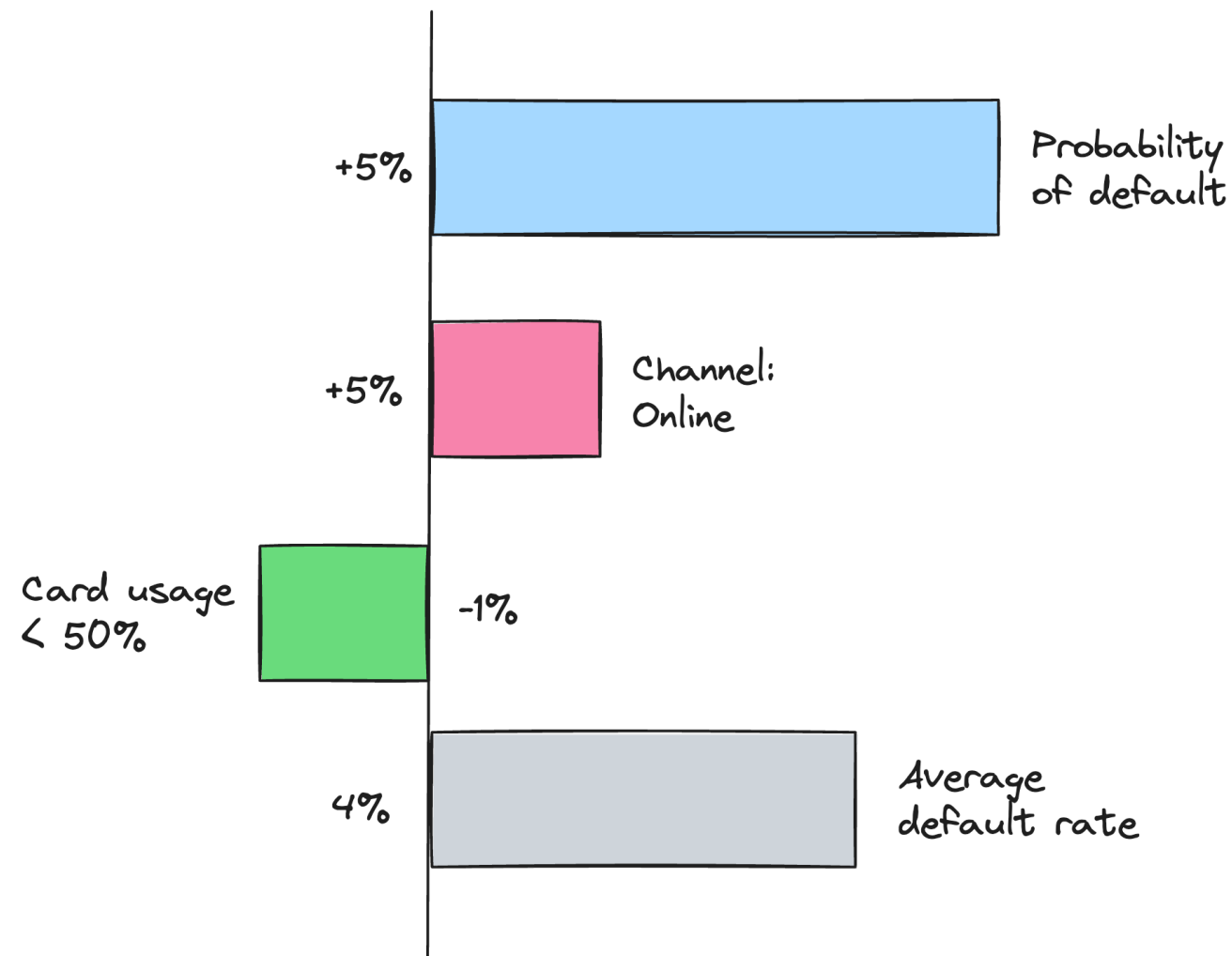
The resulting WOE-based scorecard has the same properties as if we used Logistic Regression (LR) or a combination of WOE and LR since it's the same underlying logit model

FastWoe

Fast Weight of Evidence (WOE) encoding and inference



```
!pip install fastwoe
from fastwoe import FastWoe
X_woe = FastWoe().fit_transform(X)
```



FastWoe is a Python library for efficient **Weight of Evidence (WOE)** encoding of categorical features and statistical inference based on scikit-learn's TargetEncoder.

FastWoe provides numerical binning with histograms, decision trees, and k-means from FAISS (including GPU speed-up ⚡)

FastWoe provides classical detailed summary tables for binned variables, including WOE standard errors and Wald Z-scores from WOE.

It also implements a Naïve Bayes Classifier with `predict_proba` and `predict_ci` methods for quick interpretable modeling.

weight of
evidence.

logistic
regression.



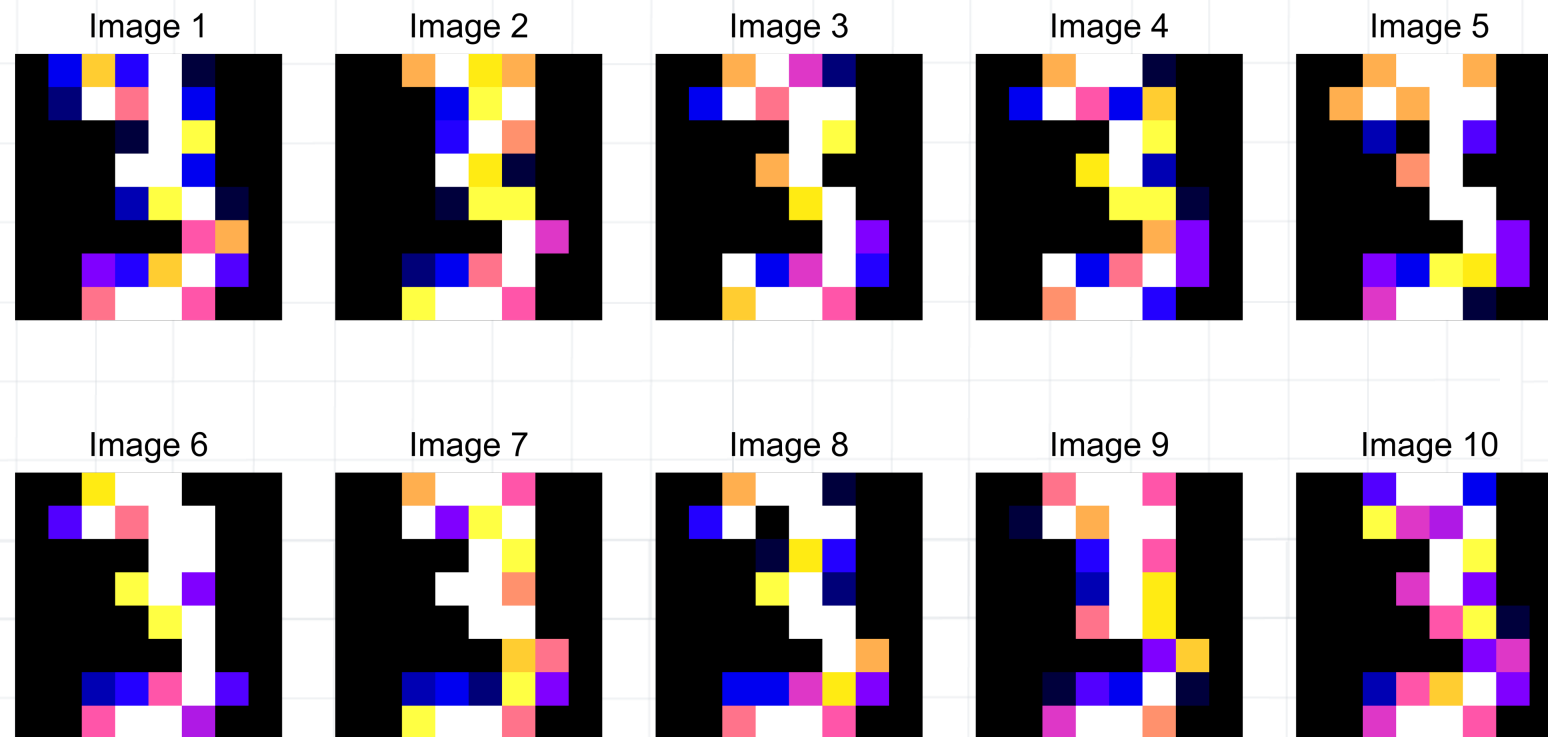
logistic regression

Generative vs Discriminative
WOE + Logistic Regression
Case of equivalence of WOE,
OHE, and Target Encoding

Generative and Discriminative Classifiers

WOE = Generative (uses Bayes' rule) | Logistic Regression = Discriminative (learns boundary)

WOE Generative Naive Bayes
Generated Images of Digit 3



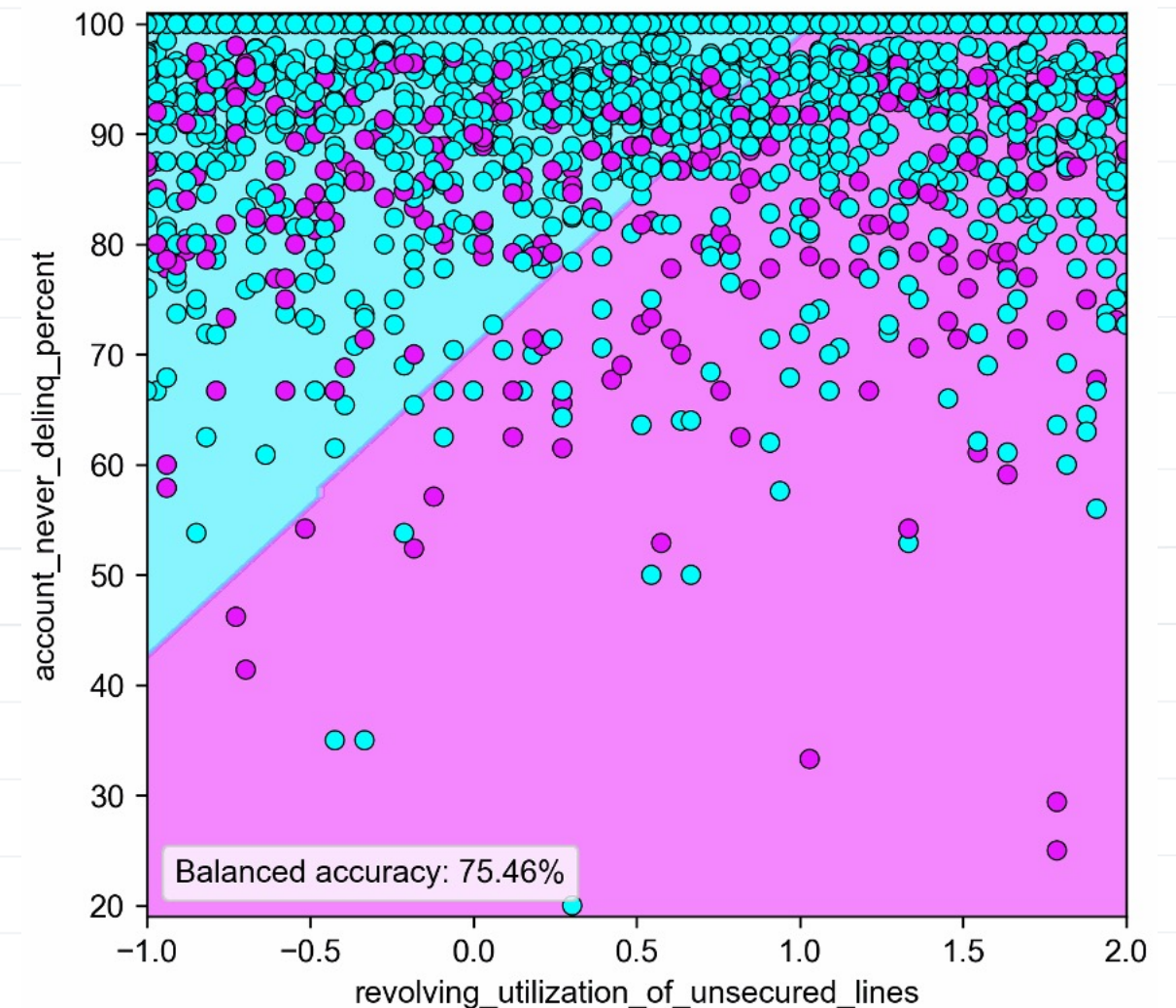
Weight of Evidence is a generative algorithm as a member of the Naïve Bayes' family

WOE learns $P(X | Y) \rightarrow$ can reconstruct $P(Y | X)$

Generative: model the joint distribution, can generate data.
Discriminative: separate classes given features.

L2 Logistic Regression

n_iter_no_change: 1



A classifier is a discriminative algorithm that separates points to predict class membership given features

Generative and Discriminative Classifiers

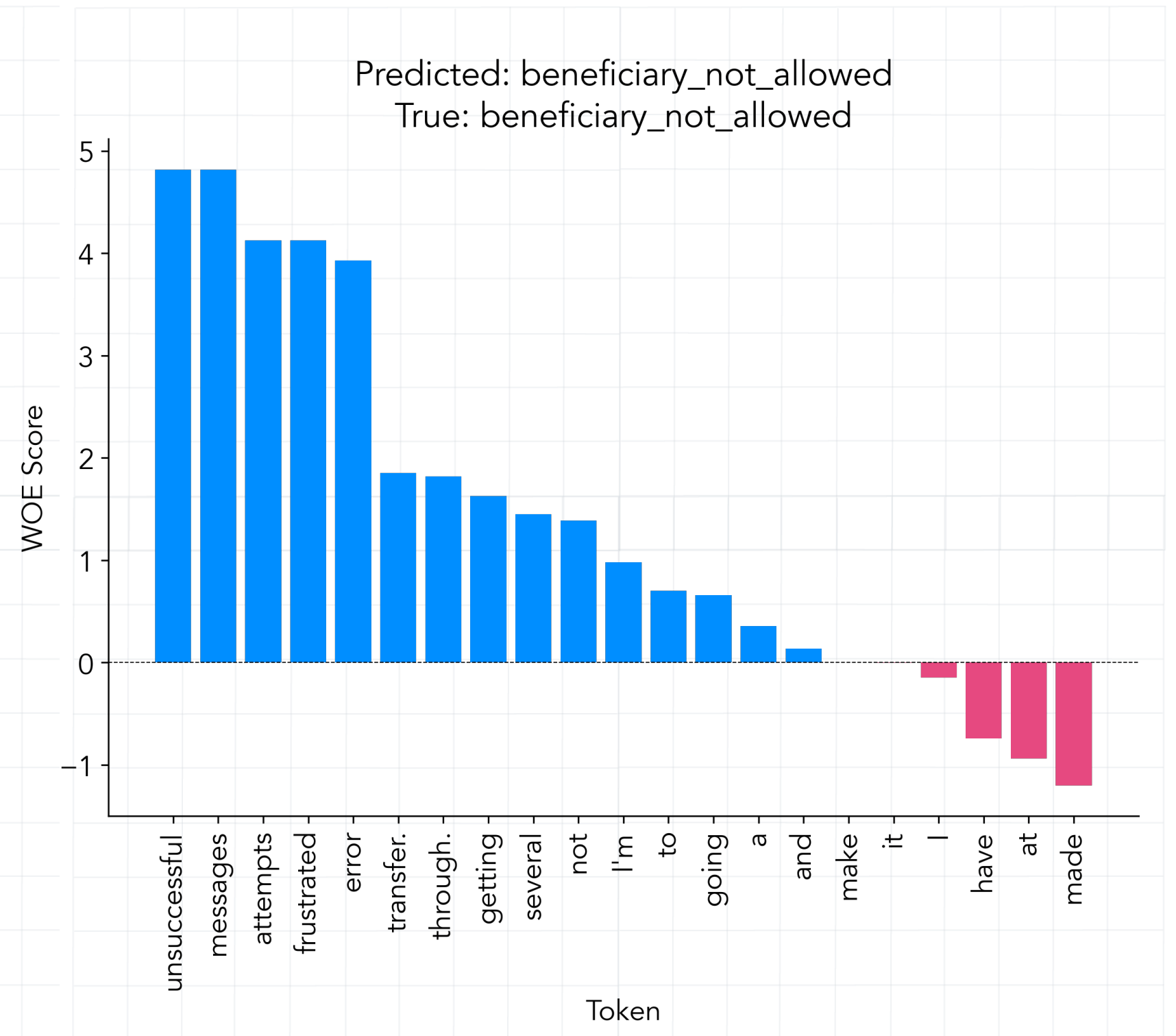
WOE + logistic regression for text classification

Weight of Evidence (WOE) can also be used in natural language processing tasks by tokenizing text and calculating WOE to encode the log likelihood ratios for each class (one-vs-rest)

- **Generative step:** WOE encodes the evidence of each token for a class, essentially summarizing log likelihood ratios.
- **Discriminative step:** WOE-encoded features are fed into a classifier (logistic regression or another discriminative model) to produce final predictions.

This is a classic example of generative – discriminative pair in machine learning models

Weight of Evidence is a powerful technique that can be used across a multitude of tasks, including AI explainability



Why combine WOE and Logistic Regression?

Lessons from Spiegelhalter-Knill-Jones method

Using Weight of Evidence (WOE) as inputs assumes that the variables are independent of one another (within each outcome class), which is unrealistic in many applied settings.

To allow for this the Spiegelhalter-Knill-Jones (1984) method calculated "adjusted weights of evidence," which are obtained by entering the raw weights of evidence as independent variables in a logistic regression equation.

Credit scorecards picked up this method around 1980-1990s by moving from dummy variables to WOE transformation for categorical data.

D. Hand (Hand & Henley, 1997) notes that weight of evidence transformation was becoming increasingly common in the 1990s.

48

SPIEGELHALTER AND KNILL-JONES

[Part 1,

$$100 \ln \frac{p(D | s)}{p(\bar{D} | s)} = w_0 + \sum_{j=1}^p w_j,$$

where any variable that is missing scores zero weight of evidence. However, in view of our previous argument for parsimonious models with good calibration, we require a means of selecting variables and estimating "partial" weights of evidence which take account of the fact that many pieces of information are obtained simultaneously. We have adopted a logistic model

$$100 \ln \frac{p(D | s)}{p(\bar{D} | s)} = a_0 + \sum_{j \in A} a_j w_j,$$

where A indexes the set of variables used to discriminate D . Since $p(\bar{D} | s) = 1 - p(D | s)$, this expression may be rearranged onto a probability scale

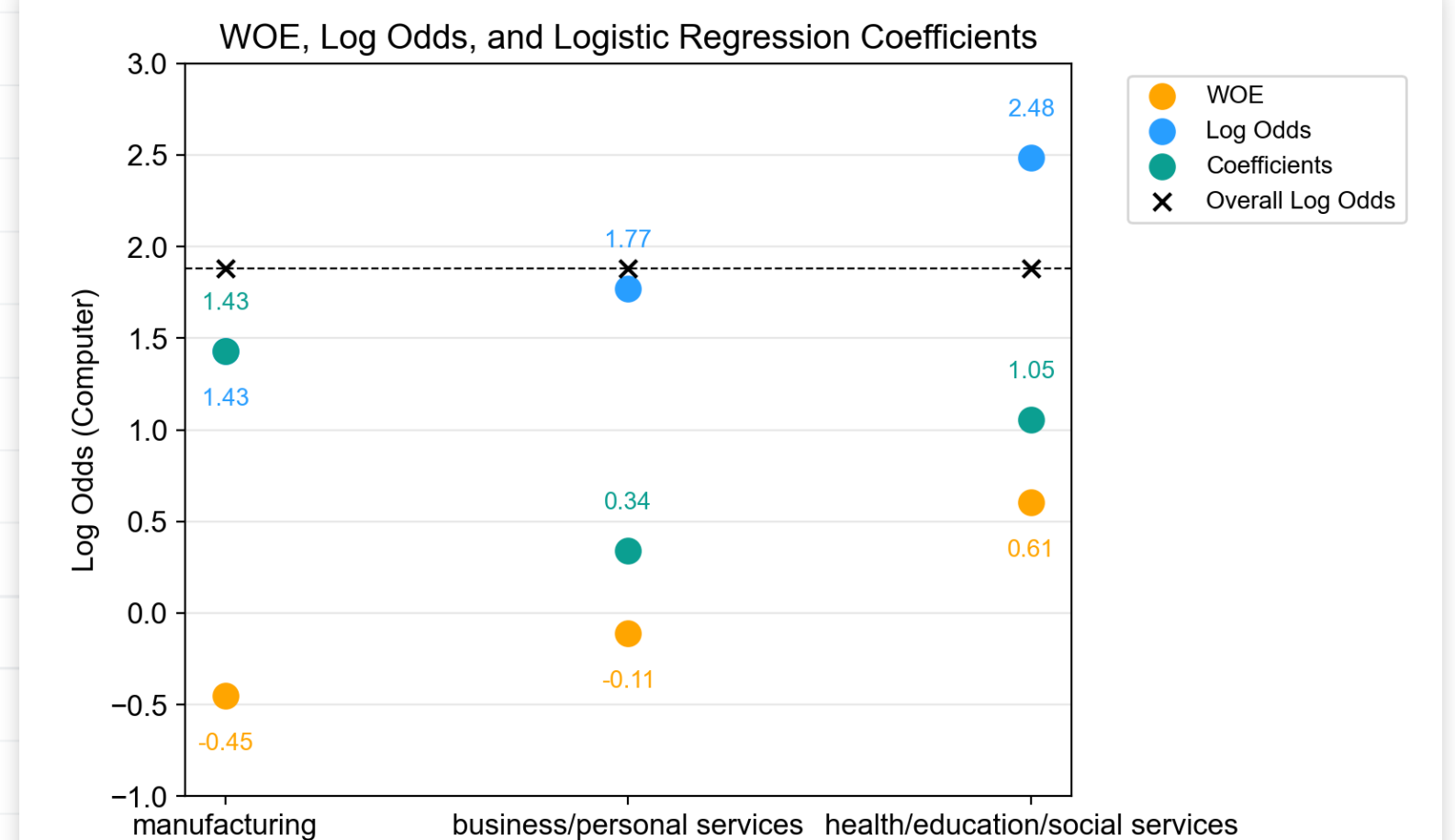
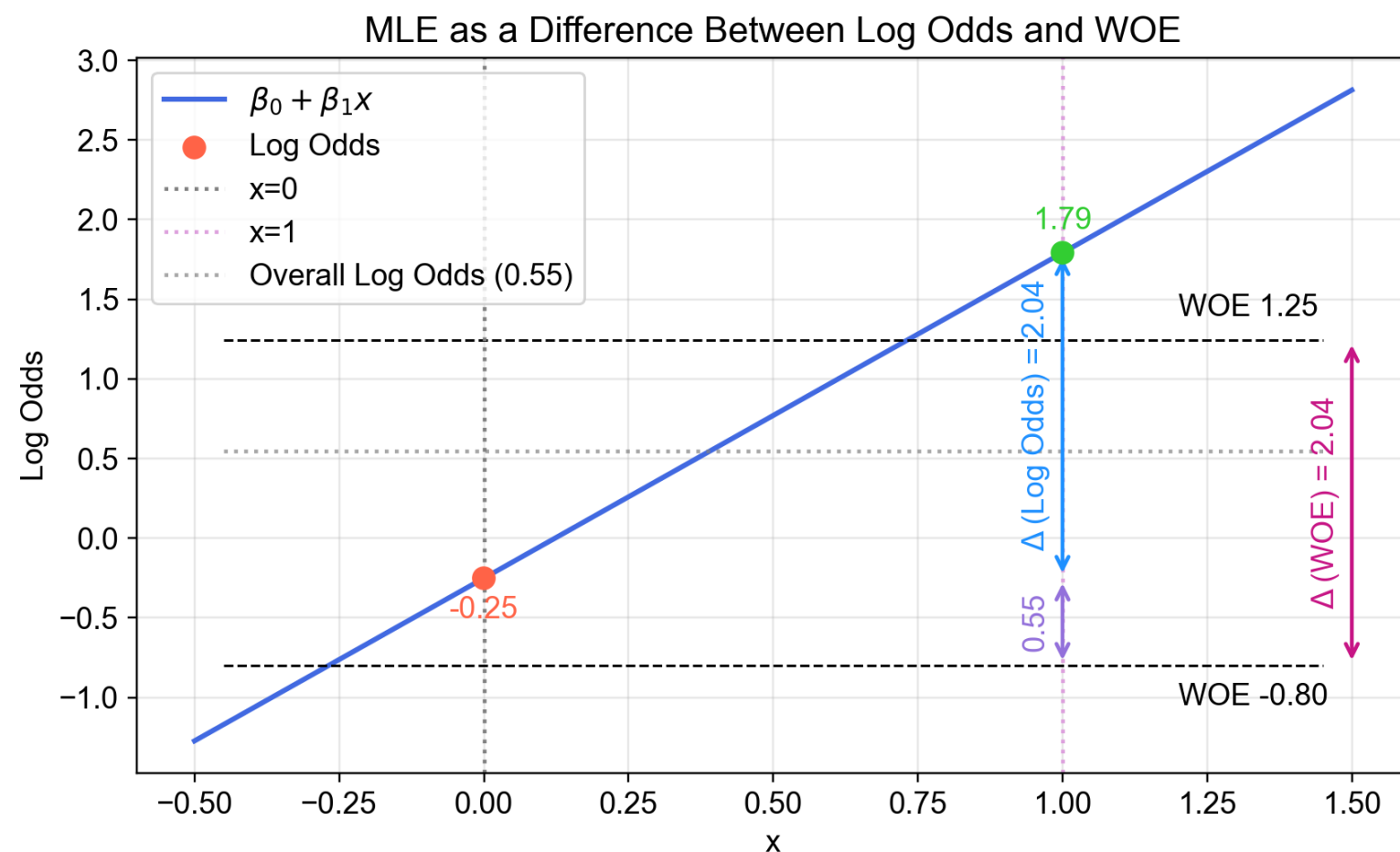
$$p(D | s) = [1 + \exp \{-T/100\}]^{-1},$$

Logistic regression fitted on WOE inputs can still be seen as Naive Bayes classifier, where each WOE term is re-weighted to account for feature dependencies and maximize likelihood.

According to D. Spiegelhalter, one of the authors of this system, mentions that this approach also **increases the calibration accuracy** of scoring.

When WOE = Logistic Regression = Target Encoding

Examples from univariate categorical analysis



Binary predictor and target (e.g., yes / no)

When using a single categorical predictor, WOE, log odds, and logistic regression coefficients align perfectly. Logistic regression simply shifts the log-odds by the global intercept, and WOE captures the difference from the overall average.

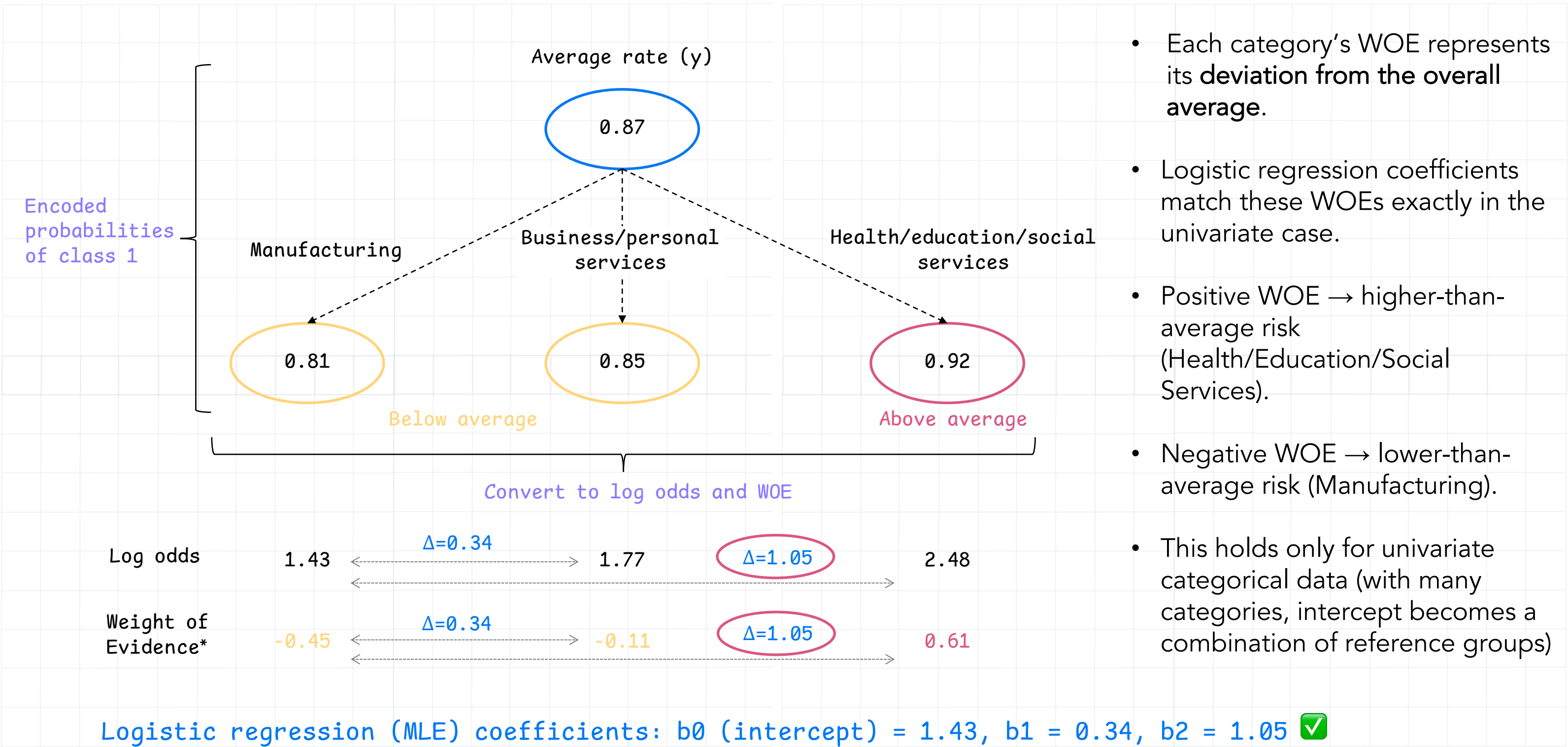
More categories in predictor

For each category, WOE = category-specific log odds minus the overall log odds. So, in the univariate case:
Difference in WOEs = Log Odds Difference = Logistic Regression Coefficient for each group

$$\beta_i = [\text{logit}(p_i) - \text{logit}(p_{\text{ref}})] = \text{WOE}_i - \text{WOE}_{\text{ref}}$$

When WOE = Logistic Regression = Target Encoding

Examples from univariate categorical analysis



- Each category's WOE represents its **deviation from the overall average**.
- Logistic regression coefficients match these WOE exactly in the univariate case.
- Positive WOE → higher-than-average risk (Health/Education/Social Services).
- Negative WOE → lower-than-average risk (Manufacturing).
- This holds only for univariate categorical data (with many categories, intercept becomes a combination of reference groups)

Pitfalls of using WOE with Logistic Regression

Tips to avoid producing wrong models

Coefficient sign flips

Sometimes we face a situation when one of the variables in WOE form flips its sign in the logistic regression fit. Why does this happen?

Usually, it is caused by one of the predictors carrying the same information as other variables, thus “reversing” the correct relationship with the target variable.

Some ways to mitigate this is:

- perform more granular feature selection based on Gini / IV → isolate and drop the problematic feature and choose a better one
- Perform more robust feature engineering to identify the strongest signal in the data

```
# Create a variable that correlates with the rest of the set
X["x_6"] = X["x_0"] + X["x_1"] + X["x_2"] + X["x_3"] + X["x_4"]
```

	x_0	x_1	x_2	x_3	x_4	x_5
x_0	100.0%	-0.0%	-2.3%	-1.2%	0.5%	48.1%
x_1	-0.0%	100.0%	2.0%	17.4%	-2.4%	64.5%
x_2	-2.3%	2.0%	100.0%	-60.8%	3.5%	21.9%
x_3	-1.2%	17.4%	-60.8%	100.0%	-8.3%	16.9%
x_4	0.5%	-2.4%	3.5%	-8.3%	100.0%	46.4%
x_5	48.1%	64.5%	21.9%	16.9%	46.4%	100.0%

Coefficients without the extra feature

feature	x_0	x_1	x_2	x_3	x_4
coef	-0.277767	-0.895691	-0.758532	-1.488302	-0.102627

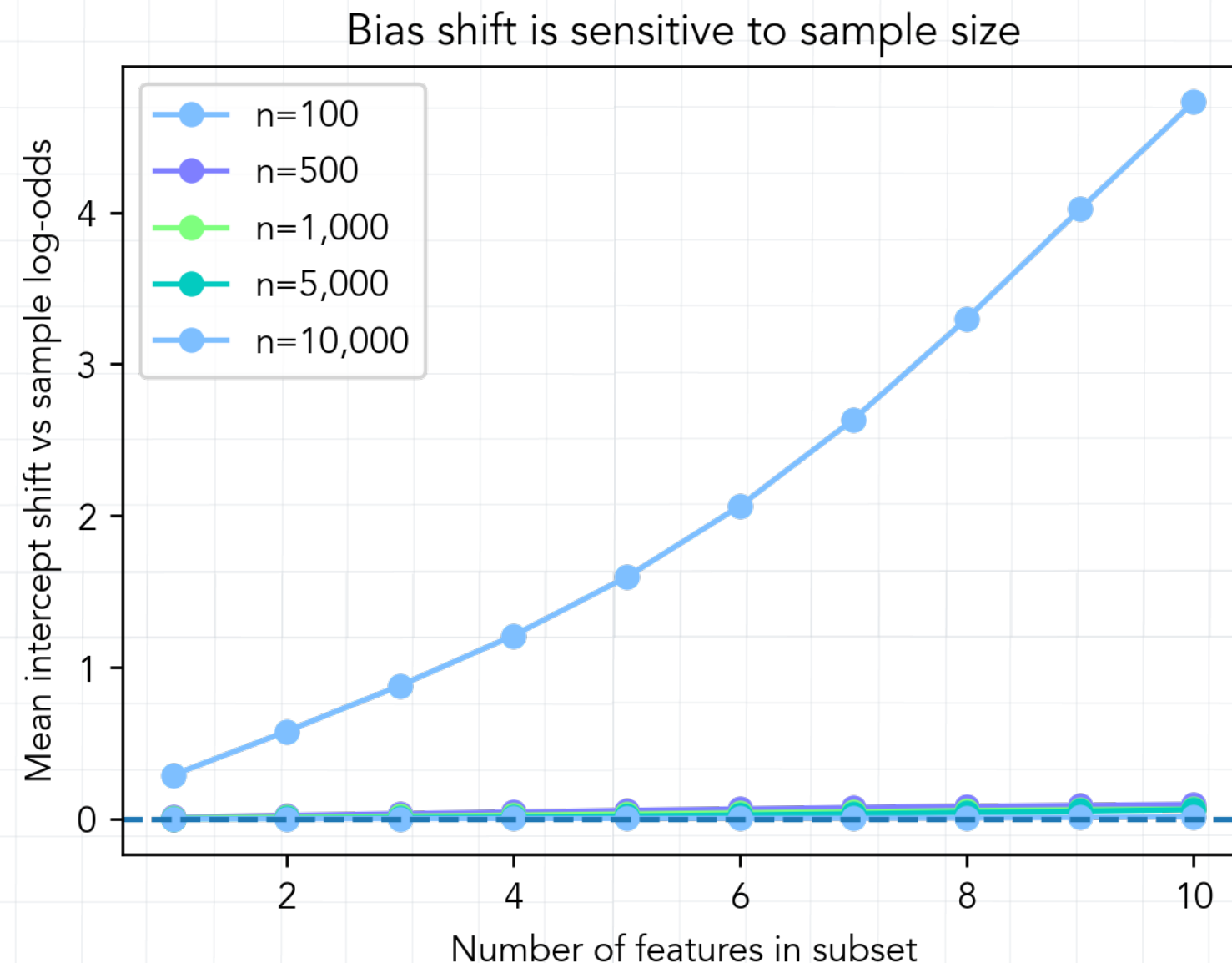
Coefficients with the extra feature

feature	x_0	x_1	x_2	x_3	x_4	x_5
coef	-0.299784	-0.949827	-0.758175	-1.452882	-0.038540	0.267718



Pitfalls of using WOE with Logistic Regression (continued)

Tips to avoid producing wrong models



For very small samples (n=100) ,we can see how adding more predictors shifts the intercept from the true value (bias)

Intercept shift bias

When using WOE predictors, depending on the dataset size, we can encounter a problem with a shift in the intercept value in logistic model*

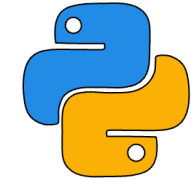
As we add more predictors, due to a joint optimization of intercept and feature weights with maximum likelihood, we can see that the resulting model's intercept will **veer away from the true value** that we observe in univariate WOE binning summaries (correct by default)

A practitioner needs to carefully examine the intercept value and perform corrections in case the baseline is incorrect → use prior prevalence correction or enforce the fixed value of intercept programmatically in case of deviations

*I owe this finding to Andrija Djurovic and Alan Forrest who identified this model risk and quantified it ([link](#)).

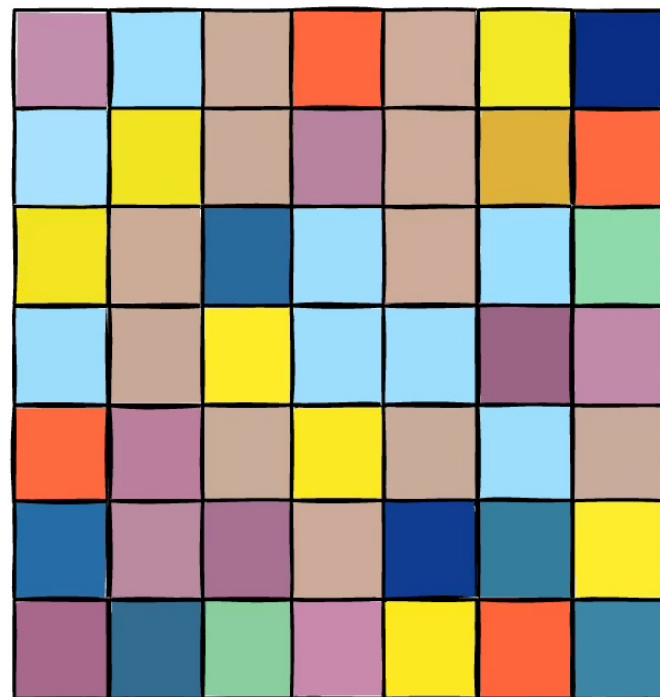
Fisher Scoring

Fisher Scoring with Python



```
!pip install fisher-scoring
from fisher_scoring import LogisticRegression
LogisticRegression().fit(X, y)
```

FISHER SCORING WITH PYTHON



The **fisher-scoring** package provides fast maximum likelihood Python  implementations with standard errors, confidence intervals, and rich summaries for:

Proportion modeling

- Logistic Regression
- Multinomial Logistic Regression

Imbalance and outliers

- Focal Loss Logistic Regression
- Robust Logistic Regression

Count data

- Poisson Regression
- Negative Binomial Regression

Integration with scikit-learn pipelines and works seamlessly with FastWoe for WOE + Logistic Regression pipelines with rich outputs .

weight of
evidence.

logistic
regression.

ensemble
learning.



ensemble learning

Link between WOE and ML
Bagging and WOE
estimators
WoeBoost

WOE Link to Machine Learning

WOE as tree weighting (P. Edwards)



Assume that we are using a decision tree to create bins based on the Gini impurity criterion. Each predictor is a decision tree, whose bins are used to create weights of evidence.

- $\omega_j(x_j)$ is a WOE value and is a predictive model of $P(Y=1)$ based only on WOE, i.e. a weak learner
- Here each β_j represents a partial adjustment made to the WOE scores
- By summing up the outputs of weak learners we're building a refined predictions

y	x_1	$\omega_1(x_1)$	β_1	x_2	$\omega_2(x_2)$	β_2	x_3	$\omega_3(x_3)$	β_3
0.99	-0.69	-0.03	-0.30	-1.30	4.18	-0.95	1.67	-1.44	-0.75
0.05	-0.90	0.03	-0.30	0.46	-2.45	-0.95	1.52	0.76	-0.75
0.01	-0.34	-0.03	-0.30	1.53	-4.19	-0.95	0.41	0.72	-0.75

1st weak learner

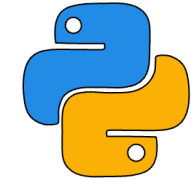
2nd weak learner

3rd weak learner

Cumulative evidence from all univariate models as
a sum product + intercept

WoeBoost

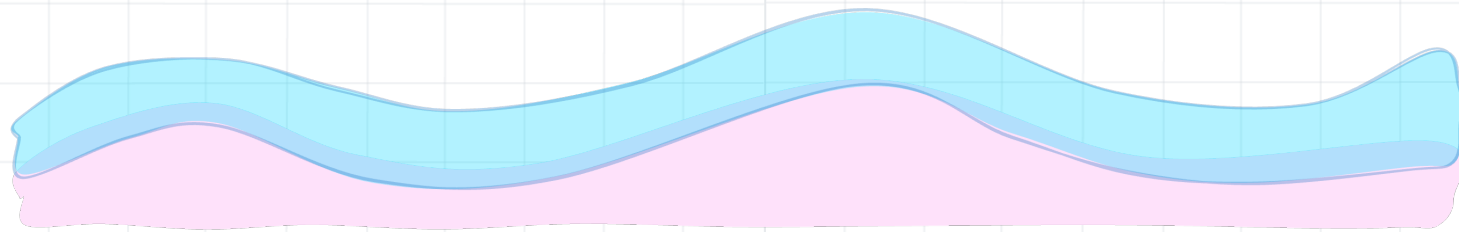
A boosted scoring system



```
!pip install woebost
from woebost import WoeBoostClassifier()
WoeBoostClassifier().fit(X, y)
```

woebost

A Boosted Scoring System



Natively handles categorical data, allows for monotonic constraints, converges without a learning rate, fully interpretable scorecards

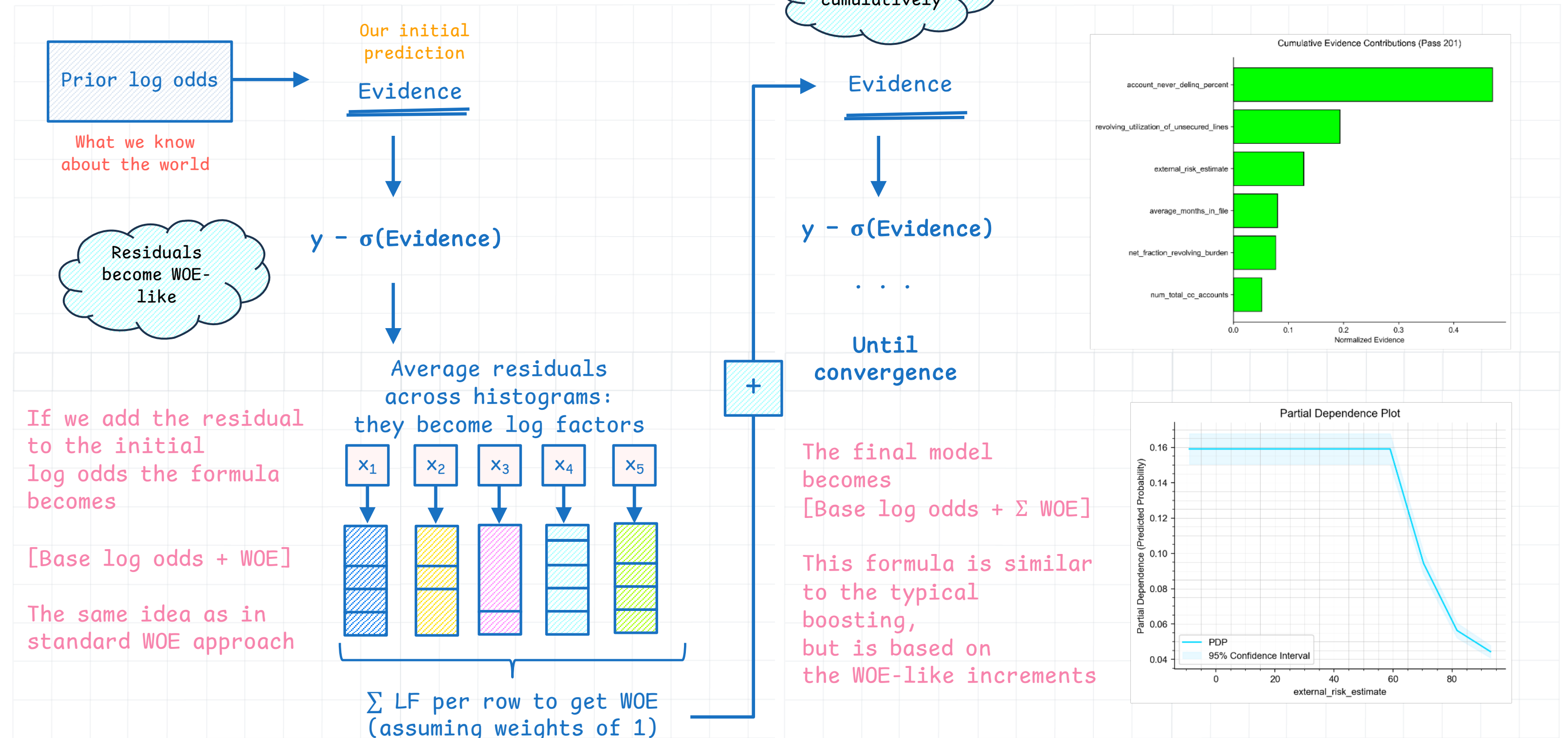
WoeBoost is a Python 🐍 package designed to bridge the gap between the predictive power of gradient boosting and the interpretability required in high-stakes domains such as finance, healthcare, and law.

It introduces an interpretable, evidence-driven framework for scoring tasks, inspired by the principles of Weight of Evidence (WOE) and the ideas of Alan M. Turing.

- 🔍 **Initialization:** Starts with prior log odds, representing baseline probabilities.
- 📈 **Iterative Updates:** Each boosting iteration calculates residual per each binned feature and sums residuals into total evidence (WOE), updating predictions.
- 🔗 **Evidence Accumulation:** Combines evidence from all iterations, producing a cumulative and interpretable scoring model.

WoeBoost

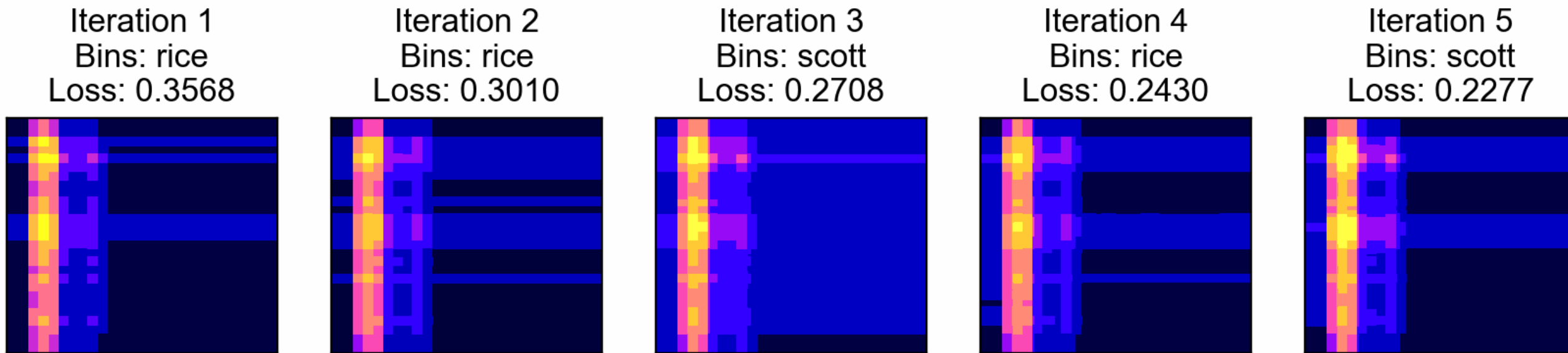
Exploring the WoeBoost algorithm



WoeBoost

Visualizing iterative updates of WoeBoost

WoeBoost Decision Boundary



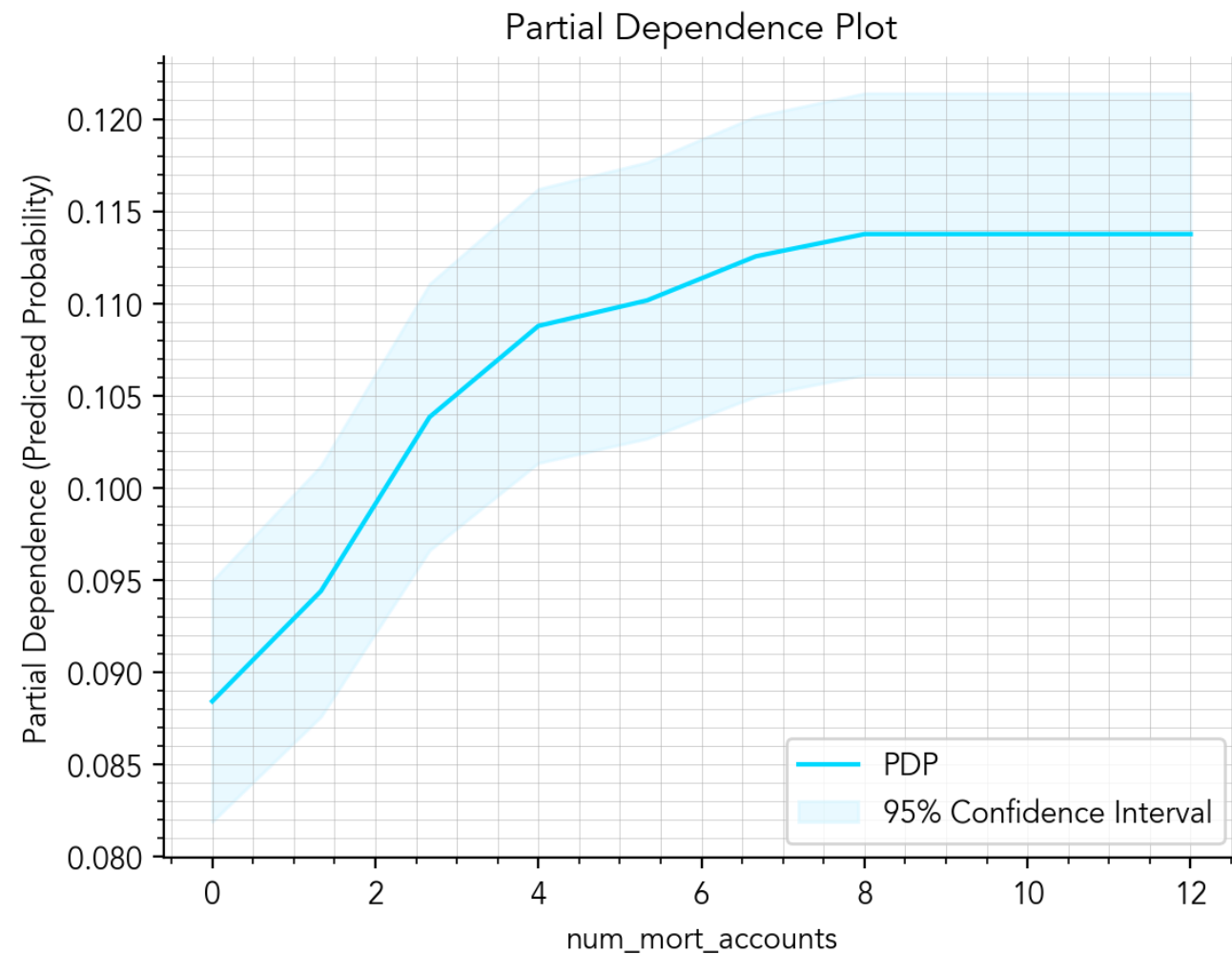
For numerical features, we can randomize binning strategy from numpy histograms at each boosting iteration (e.g., rice, scott etc).

For this F-shaped synthetic dataset, we can see that eventually the algorithm is able to model the data-generating process and produce the boundaries of the original data shape.

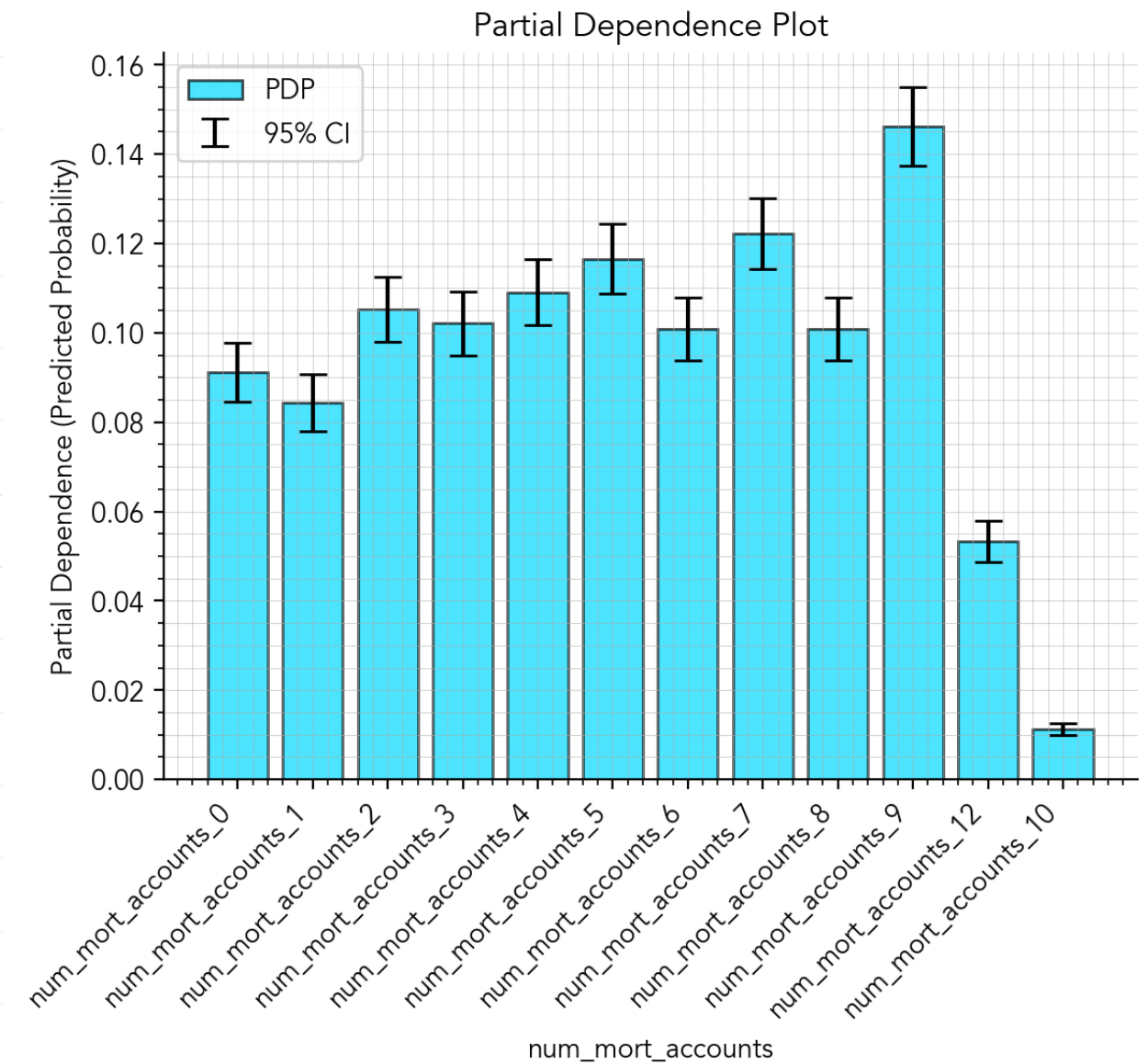
WoeBoost

Partial dependence analysis in WoeBoost

Treatment of number of mortgages as a **numerical** feature



Treatment of number of mortgages as a **categorical** feature



WoeBoost's performance can match CatBoost on credit datasets, yet, due to Python-only implementation it can be slow on big data. With free-threaded Python it will be 3-5x faster.

boosting.



boosting

AdaBoost

Bagging and RF

Additive Logistic

Regression

Regression vs. Classification

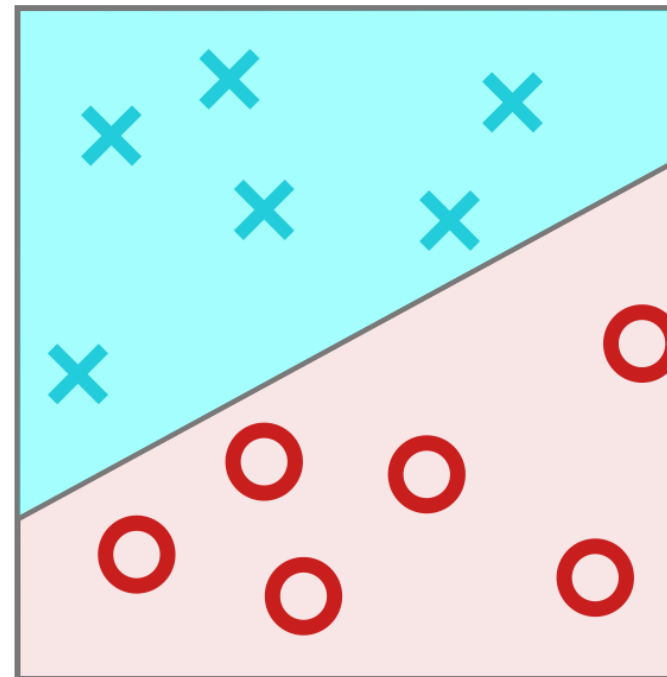
The misunderstood link between logistic regression and classification

In machine learning, regression usually refers to the task of predicting a numeric (continuous) target variable. However, the term can also describe a specific family of parametric models used for this purpose (e.g., linear regression, generalized linear models).

Logistic regression is one such regression model, a Generalized Linear Model (GLM), that models the log-odds of the target class. Although it is a regression algorithm, it is widely used for classification tasks because its outputs can be interpreted as probabilities of belonging to a class, just like other classification algorithms.

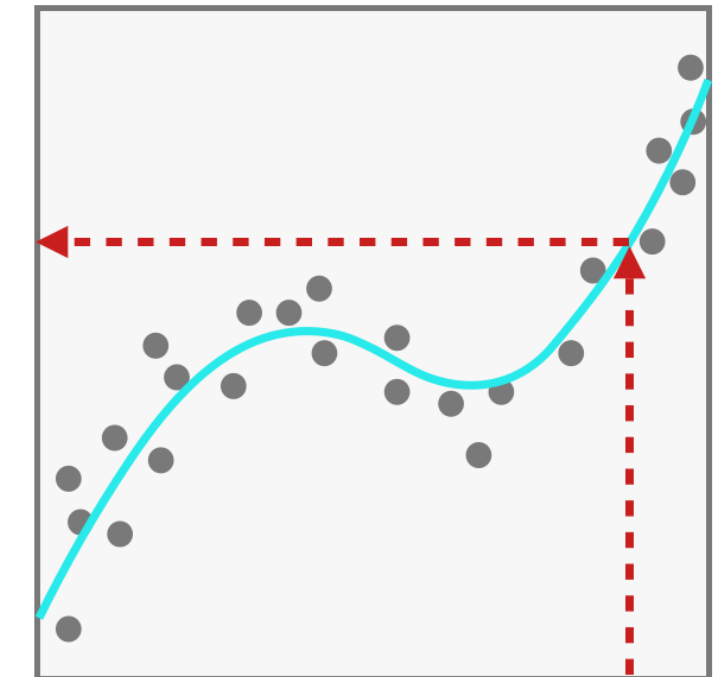
In other words, logistic regression outputs a continuous score between 0 and 1, which we can interpret as a probability and then convert into some hard class decision (e.g., approve vs. decline).

Classification Groups observations into "classes"



Here, the line classifies the observations into X's and O's

Regression predicts a numeric value

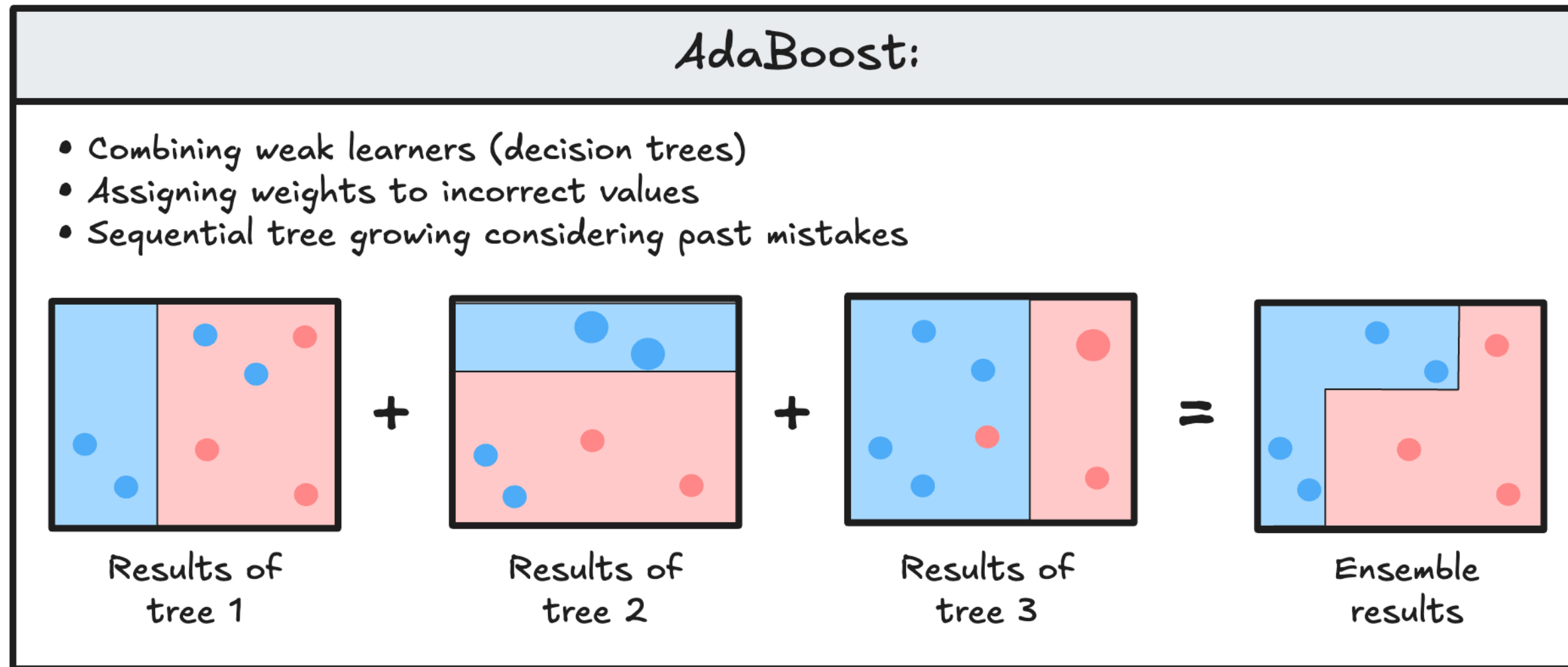


Here, the fitted line provides a predicted output, if we give it an input

Many classification algorithms produce soft scores, continuous values that can be interpreted as probabilities (e.g., logistic regression, gradient boosting). These scores are then converted into hard class labels by applying a decision threshold. Some algorithms, however, only produce hard labels. This is why the general term is classification.

AdaBoost

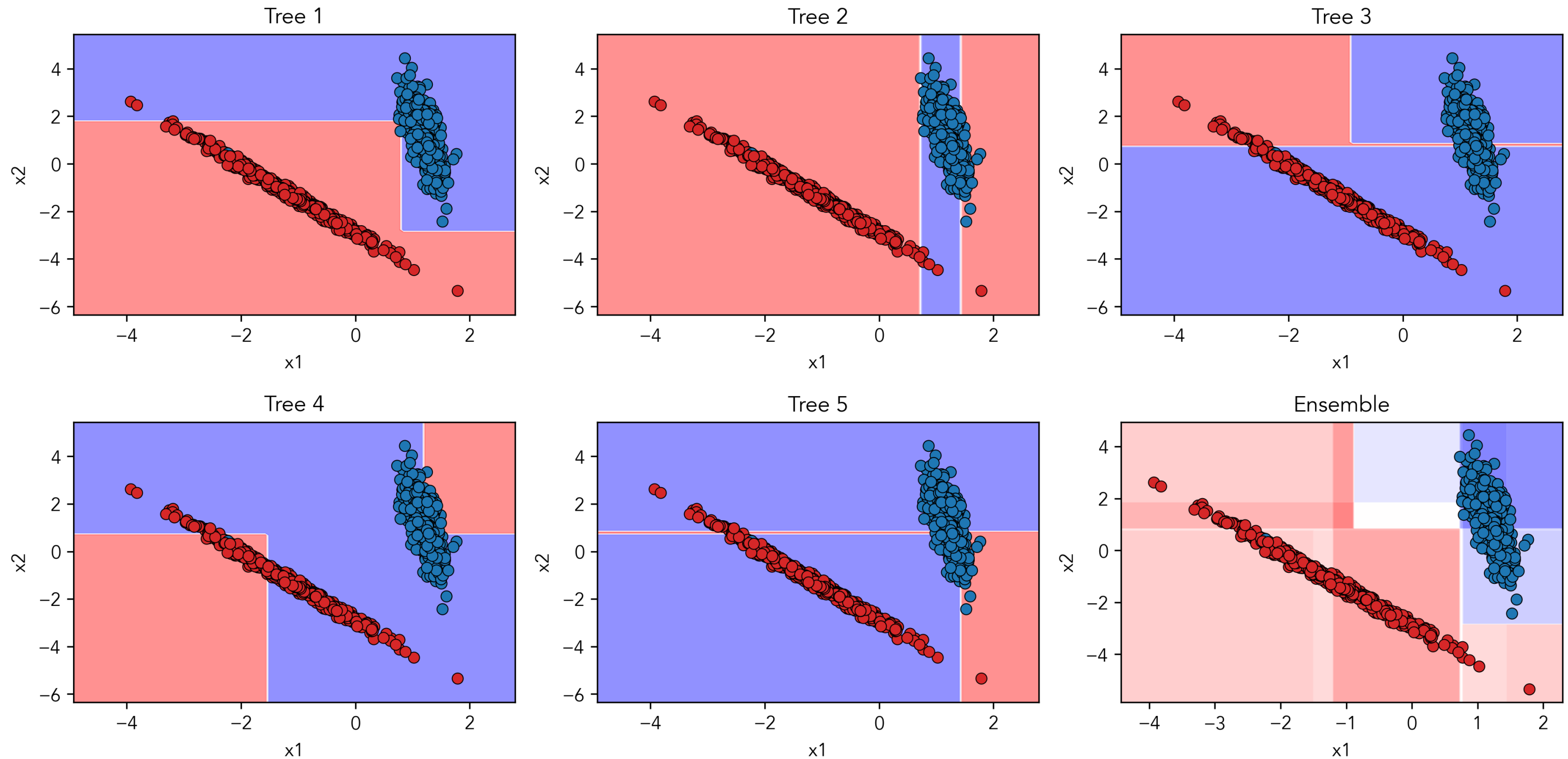
Turning weak learners into a strong ensemble by reweighting mistakes



Developed by Yoav Freund and Robert Schapire (1996), AdaBoost introduced the idea of boosting, turning weak learners into a strong ensemble by focusing on the hardest-to-classify points and minimizing the **exponential loss function**.

Real AdaBoost

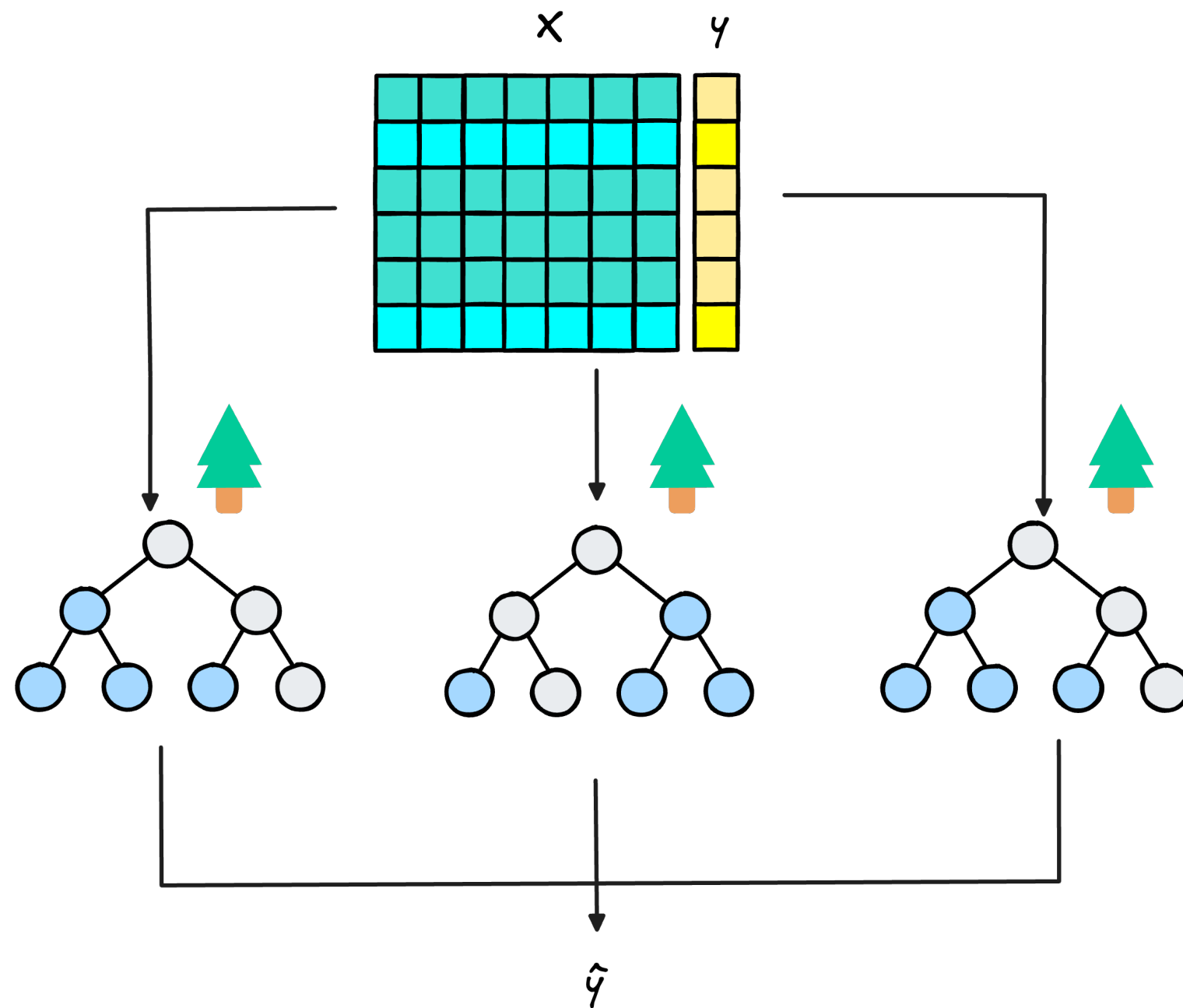
Real AdaBoost can reach a strong classifier with fewer trees than AdaBoost



Code example adapted based on Paul Edwards' SAS talk on Real AdaBoost ([link](#)).

Decision Trees, Bagging, and Random Forests

“Random selection of features to split each node that compares favorably to Adaboost”

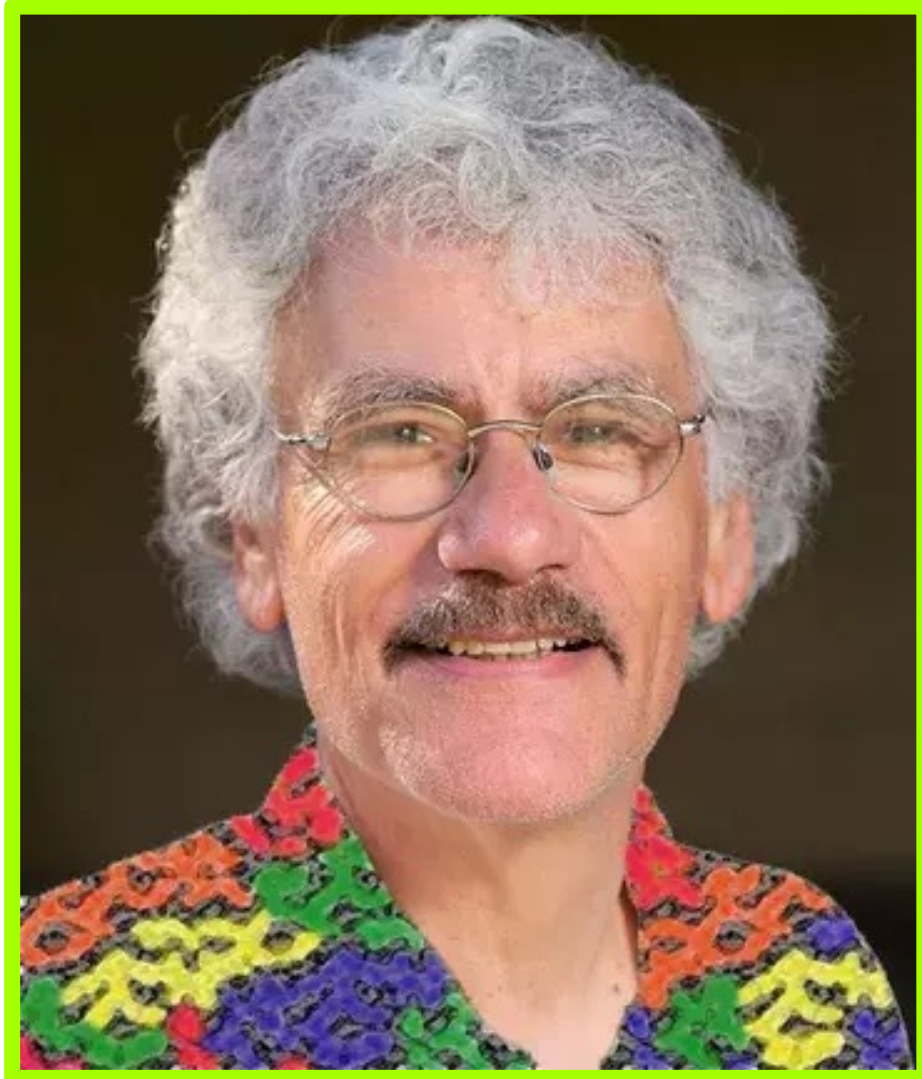


Random Forests, introduced by Leo Breiman in 2001, are a powerful ensemble method that extends the idea of bagging (1996). They build many decision trees on bootstrapped samples of the training data and then combine their predictions:

- **Randomization:** At each split, only a random subset of features is considered.
- **Aggregation:** Predictions from all trees are averaged (for regression) or combined by majority vote (for classification).
- **Performance:** Random Forests require minimal tuning, handle high-dimensional data well, and provide robust generalization.

Breiman described Random Forests as “the combination of many unpruned decision trees, each constructed on a random subset of the data and features, whose results are averaged to form the final prediction.”

Jerry Friedman is the statistician who changed machine learning with boosting paving the way to **the most powerful algorithms on tabular data**



Jerry Friedman (1939-)

One of the pioneers of modern machine learning and the mind behind **Gradient Boosting Machines (GBM)**.

His work introduced:

- **Gradient Boosting** – a framework to build strong predictors by sequentially fitting weak learners to the negative gradient of a loss function.
- **LogitBoost** – a special case of gradient boosting tailored to classification, where the model is fit to minimize the binomial log-likelihood.

“Boosting can be viewed as gradient descent in function space.” – Friedman, Hastie & Tibshirani (2000)

Additive Logistic Regression (aka LogitBoost)

A statistical view of boosting

When asked about a statistical formulation of why boosting works so well, Jerry Friedman responds:

"... when boosting came out much later, Rob and Trevor and I tried to show what it was doing, again from a statistical perspective... He [L. Breiman] thought our formal development was correct, but he didn't think that was the reason that led to boosting's apparent spectacular performance. But I was convinced that we had explained it. I think the machine learners, the PAC learning people never thought so, I don't think they completely understood the way we were looking at it."

The paper, often cited as FHT 2000, introduced boosting with logistic loss (binomial likelihood), which later became known as LogitBoost. LogitBoost became a reference implementation for most modern boosted tree classifiers.

Additive Logistic Regression: a Statistical View of Boosting ([link](#)).

The Annals of Statistics
2000, Vol. 28, No. 2, 337–407

SPECIAL INVITED PAPER

ADDITIVE LOGISTIC REGRESSION: A STATISTICAL VIEW OF BOOSTING

BY JEROME FRIEDMAN,¹ TREVOR HASTIE^{2, 3} AND
ROBERT TIBSHIRANI^{2, 4}

Stanford University

Boosting is one of the most important recent developments in classification methodology. Boosting works by sequentially applying a classification algorithm to reweighted versions of the training data and then taking a weighted majority vote of the sequence of classifiers thus produced. For many classification algorithms, this simple strategy results in dramatic improvements in performance. We show that this seemingly mysterious phenomenon can be understood in terms of well-known statistical principles, namely additive modeling and maximum likelihood. For the two-class problem, boosting can be viewed as an approximation to additive modeling on the logistic scale using maximum Bernoulli likelihood as a criterion. We develop more direct approximations and show that they exhibit nearly identical results to boosting. Direct multiclass generalizations based on multinomial likelihood are derived that exhibit performance comparable to other recently proposed multiclass generalizations of boosting in most situations, and far superior in some. We suggest a minor modification to boosting that can reduce computation, often by factors of 10 to 50. Finally, we apply these insights to produce an alternative formulation of boosting decision trees. This approach, based on best-first truncated tree induction, often leads to better performance, and can provide interpretable descriptions of the aggregate decision rule. It is also much faster computationally, making it more suitable to large-scale data mining applications.

Additive Logistic Regression (aka LogitBoost)

A statistical view of boosting

Applied Logistic Regression (2nd edition, 4.4)

$$\begin{aligned} z_i &= (1, x_i') \hat{\beta} + \frac{(y_i - \hat{\pi}_i)}{\hat{\pi}_i(1 - \hat{\pi}_i)} \\ &= \hat{\beta}_0 + \sum_{j=1}^p \hat{\beta}_j x_{ij} + \frac{(y_i - \hat{\pi}_i)}{\hat{\pi}_i(1 - \hat{\pi}_i)} \\ &= \ln\left(\frac{\hat{\pi}_i}{1 - \hat{\pi}_i}\right) + \frac{(y_i - \hat{\pi}_i)}{\hat{\pi}_i(1 - \hat{\pi}_i)} \end{aligned} \quad (4.1)$$

and a case weight

$$v_i = \hat{\pi}_i(1 - \hat{\pi}_i). \quad (4.2)$$

Note that all we need is access to the fitted values, $\hat{\pi}_i$, to compute the values of z_i and v_i . Next, we run a linear regression program using the values of z_i as the dependent variable, the values of x_i for our vector of independent variables, and the values of v_i for our case weights.

Proceeding further with the linear regression, it can be shown that the residuals from the fit are

$$(z_i - \hat{z}_i) = \frac{(y_i - \hat{\pi}_i)}{\hat{\pi}_i(1 - \hat{\pi}_i)}$$

FHT 2000: Algorithm 3 (LogitBoost)

LogitBoost (two classes)

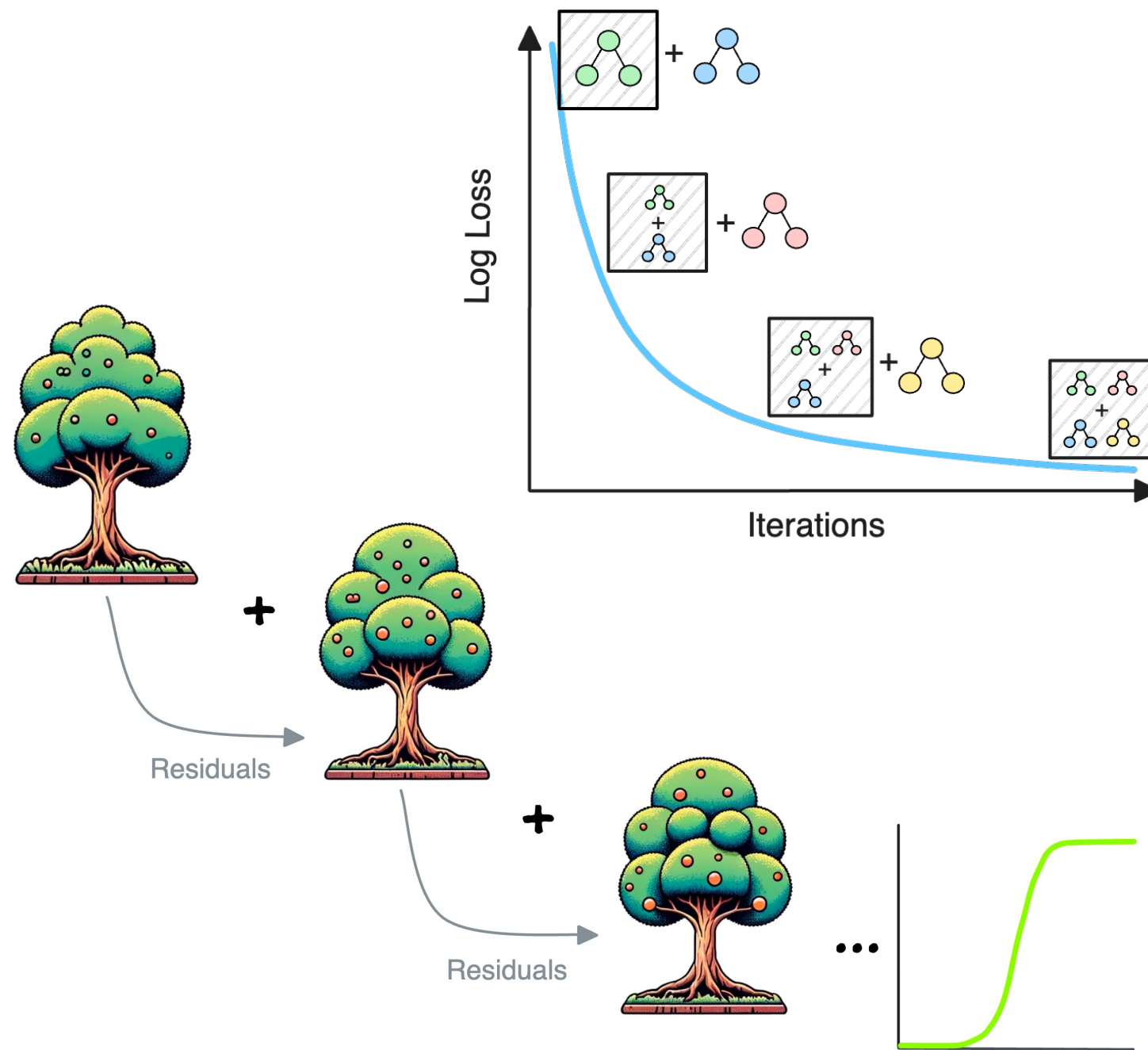
1. Start with weights $w_i = 1/N$ $i = 1, 2, \dots, N$, $F(x) = 0$ and probability estimates $p(x_i) = \frac{1}{2}$.
2. Repeat for $m = 1, 2, \dots, M$:
 - (a) Compute the working response and weights
$$z_i = \frac{y_i^* - p(x_i)}{p(x_i)(1 - p(x_i))},$$
$$w_i = p(x_i)(1 - p(x_i)).$$
 - (b) Fit the function $f_m(x)$ by a weighted least-squares regression of z_i to x_i using weights w_i .
 - (c) Update $F(x) \leftarrow F(x) + \frac{1}{2}f_m(x)$ and $p(x) \leftarrow (e^{F(x)})/(e^{F(x)} + e^{-F(x)})$.
3. Output the classifier $\text{sign}[F(x)] = \text{sign}[\sum_{m=1}^M f_m(x)]$.

ALGORITHM 3. *An adaptive Newton algorithm for fitting an additive logistic regression model.*

Same formulation, same loss function, same iterative procedure, the only difference being the non-parametric estimator (decision tree stumps)

LogitBoost Implementation in Python

From theory to code



```
import numpy as np
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import log_loss
from scipy.special import expit as sigmoid

class LogitBoost(BaseEstimator, ClassifierMixin):
    def __init__(self, n_iter=100, learning_rate=1.0, max_depth=1):
        self.n_iter = n_iter
        self.learning_rate = learning_rate
        self.max_depth = max_depth
        self.predictors = []
        self.loss_dict = []

    def fit(self, X, y):
        # Initialize weights
        w = np.ones(X.shape[0]) / X.shape[0]
        # Initialize predictions
        F = np.zeros(X.shape[0])

        for _ in range(self.n_iter):
            # Working response and weights
            p = sigmoid(F) # probability
            z = (y - p) / (p * (1 - p)) # working response
            w = p * (1 - p) # case weights

            # Fit a regressor (stump) to the working response
            tree = DecisionTreeRegressor(max_depth=self.max_depth)
            tree.fit(X, z, sample_weight=w)
            self.predictors.append(tree)

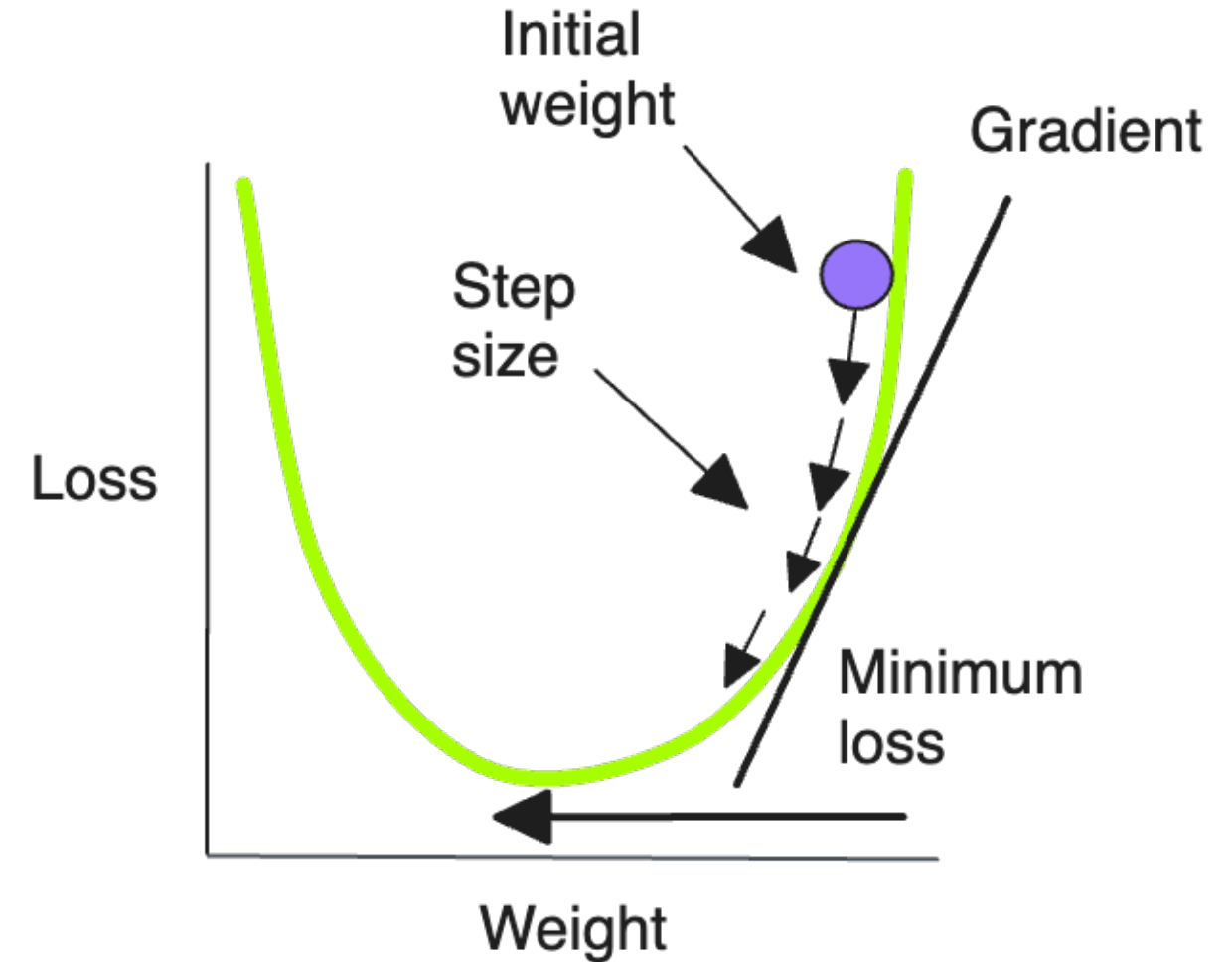
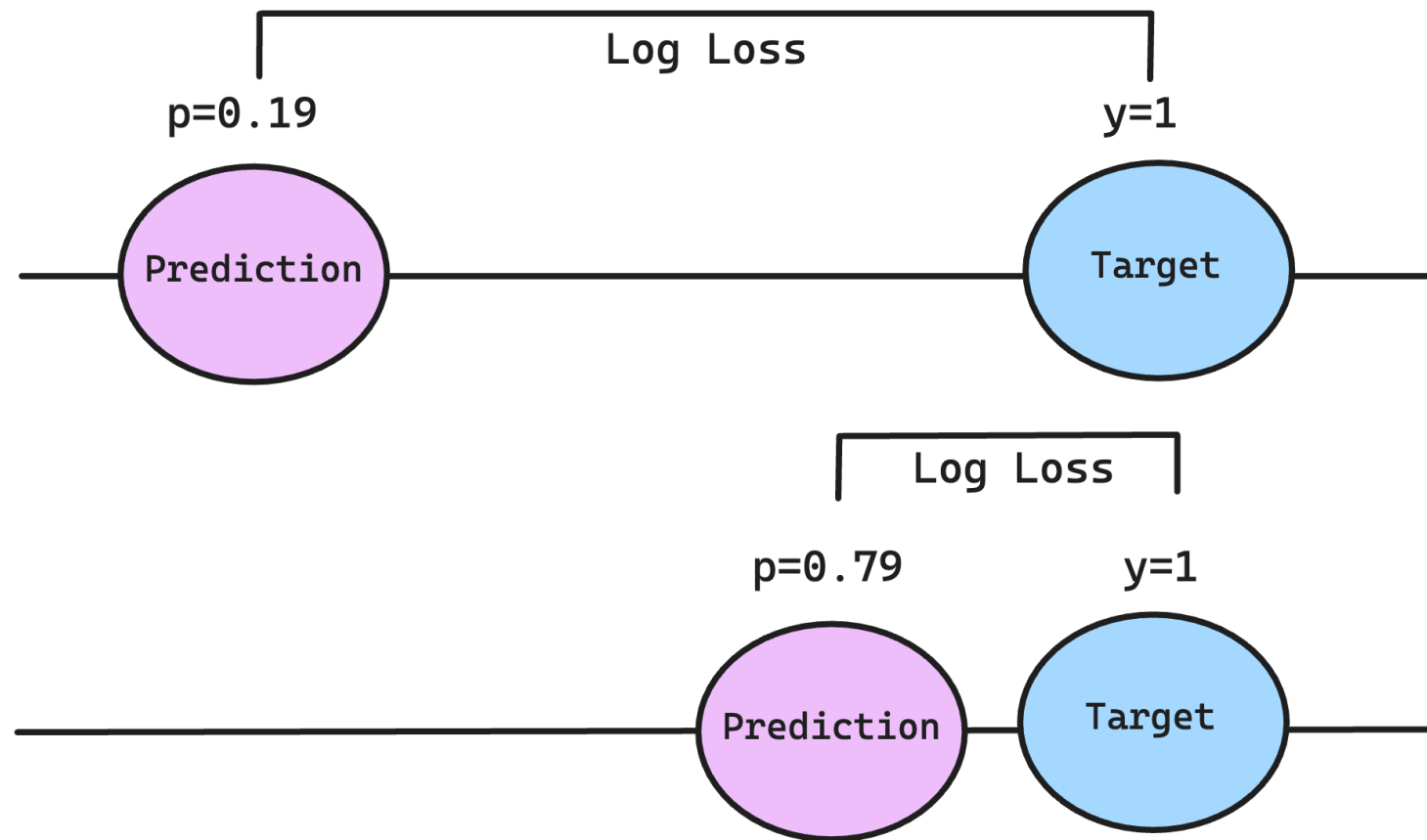
            # Update F
            F += self.learning_rate * tree.predict(X)
        return self
```

logitboost.

xgboost.

Loss functions

For binary classification, we are optimizing log loss



$$LL(p) = \sum_{i=1}^n \left[y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i) \right] \quad \text{LogLoss}(p) = - \sum_{i=1}^n \left[y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i) \right]$$

Log loss is negative Bernoulli log likelihood.
In ML models, we minimize loss.

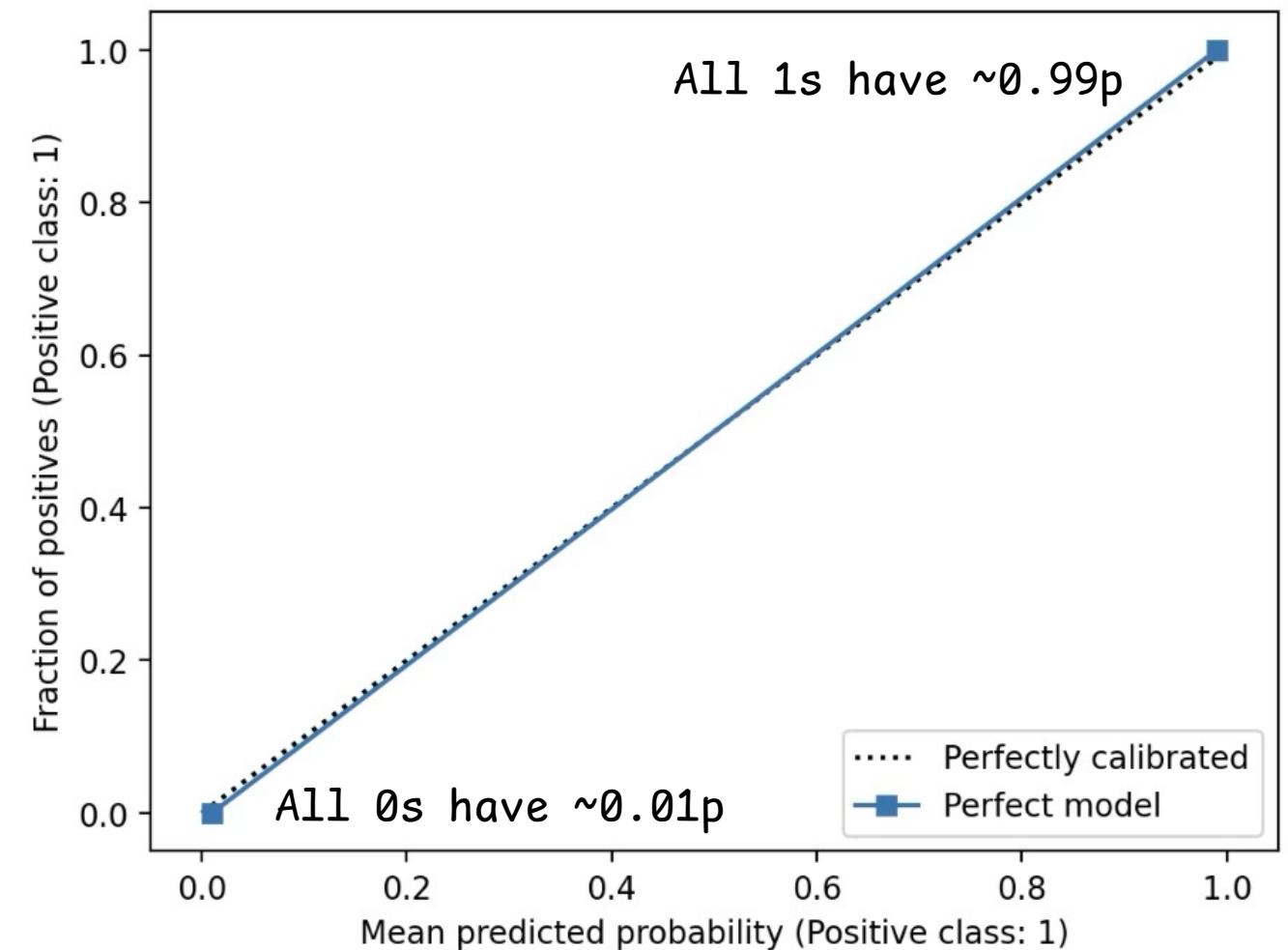
Loss functions

Log loss penalizes false predictions for being too “far off” from ground truth

Log loss calculation

Actual class	Prediction probability	$y * \ln(p)$	$(1-y) * \ln(1-p)$	Log loss
1	0.99	-0.01	0.00	0.01
0	0.01	0.00	-0.01	0.01
1	0.99	-0.01	0.00	0.01
...	...	...	...	...

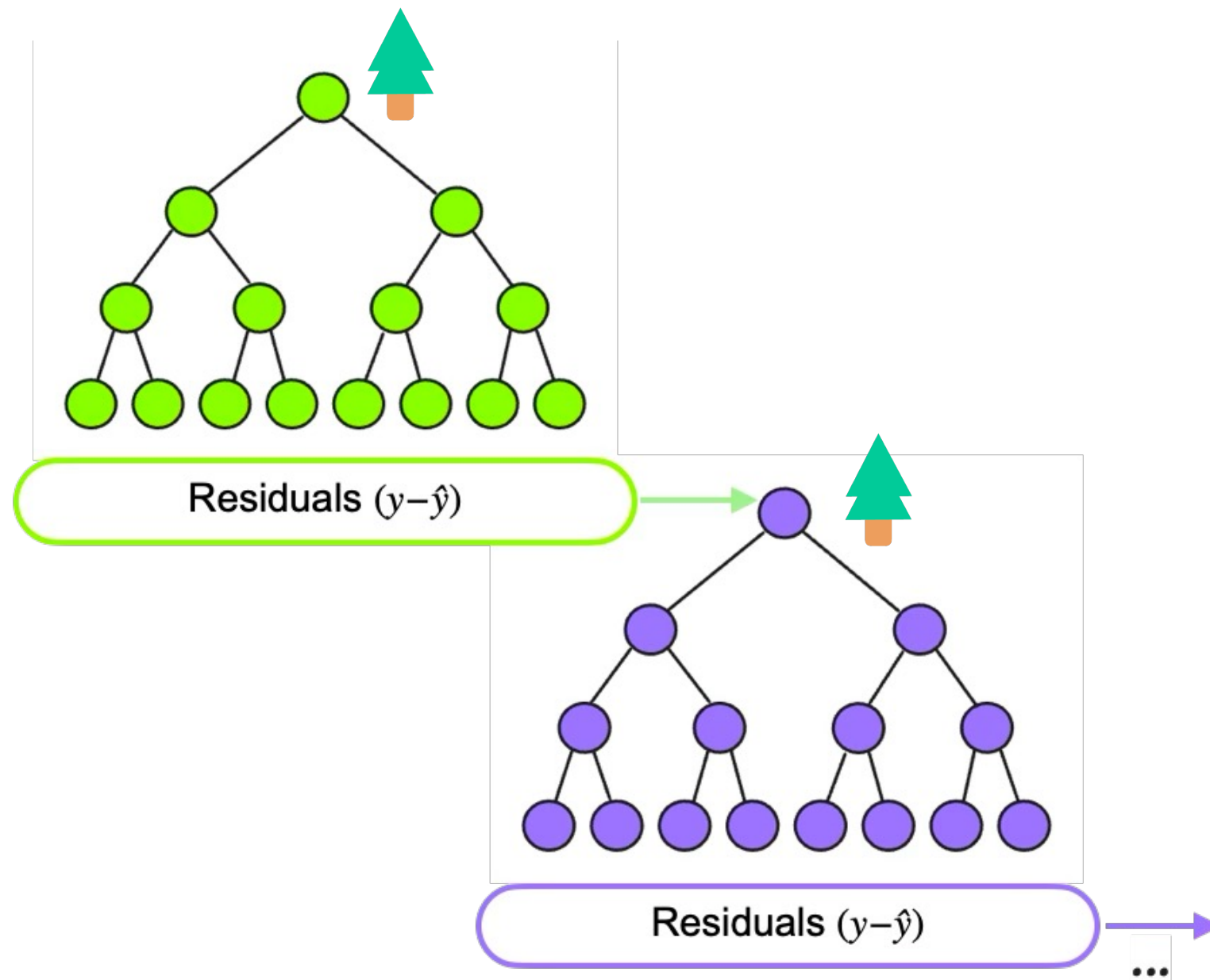
Ideally calibrated model



Log loss penalizes confident wrong predictions much more heavily than less confident ones.
An ideally calibrated model assigns probabilities ~ 1 for positives and ~ 0 for negatives.

Gradient Boosting – Gradient Boosted Decision Trees

Fitting residuals iteratively with weak learners



GBDT are a gradient boosting implementation, where a decision tree is used as a “weak learner”. This means that although each individual tree may be only slightly better than a random guess, a joint ensemble of such trees will generate an accurate prediction.

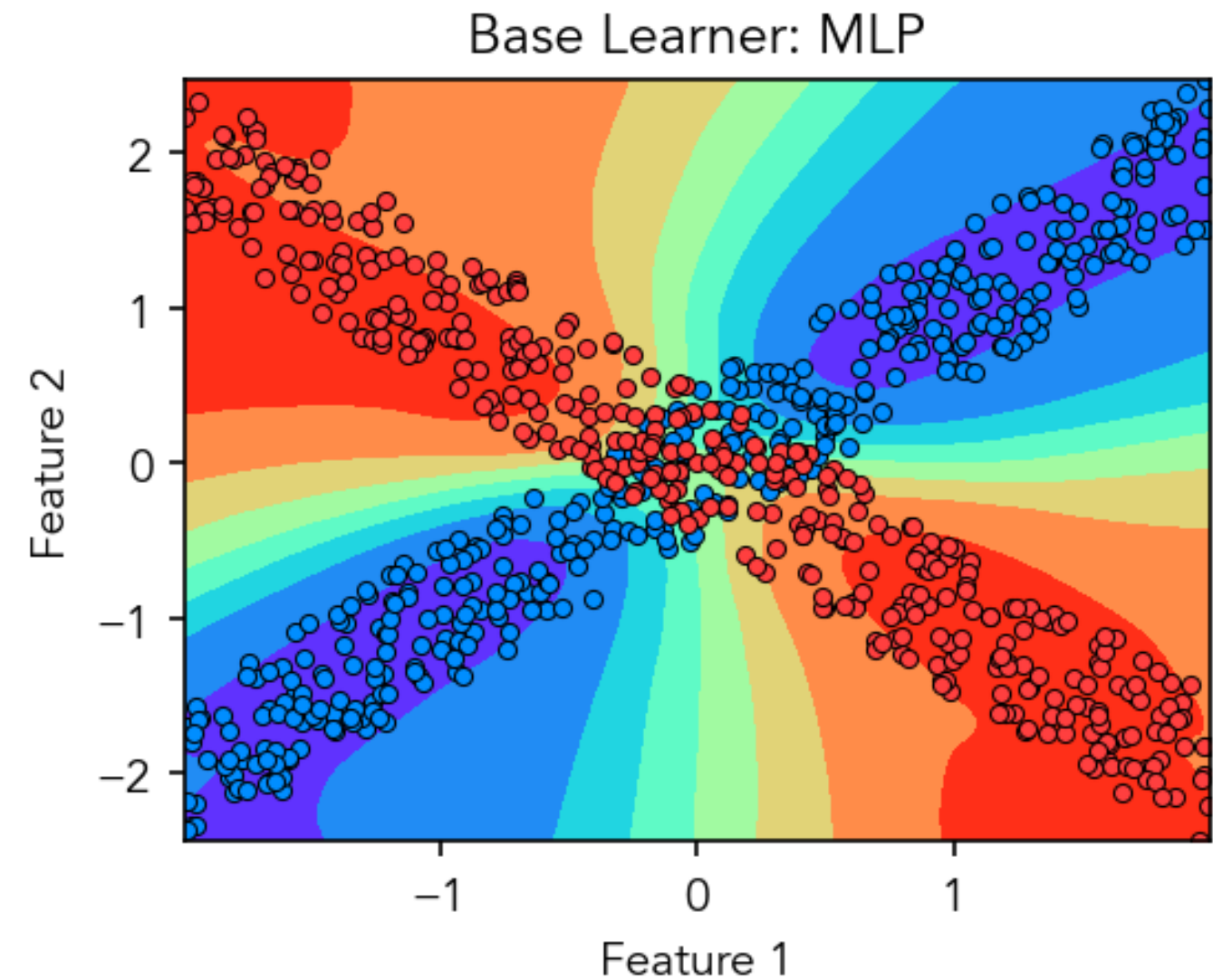
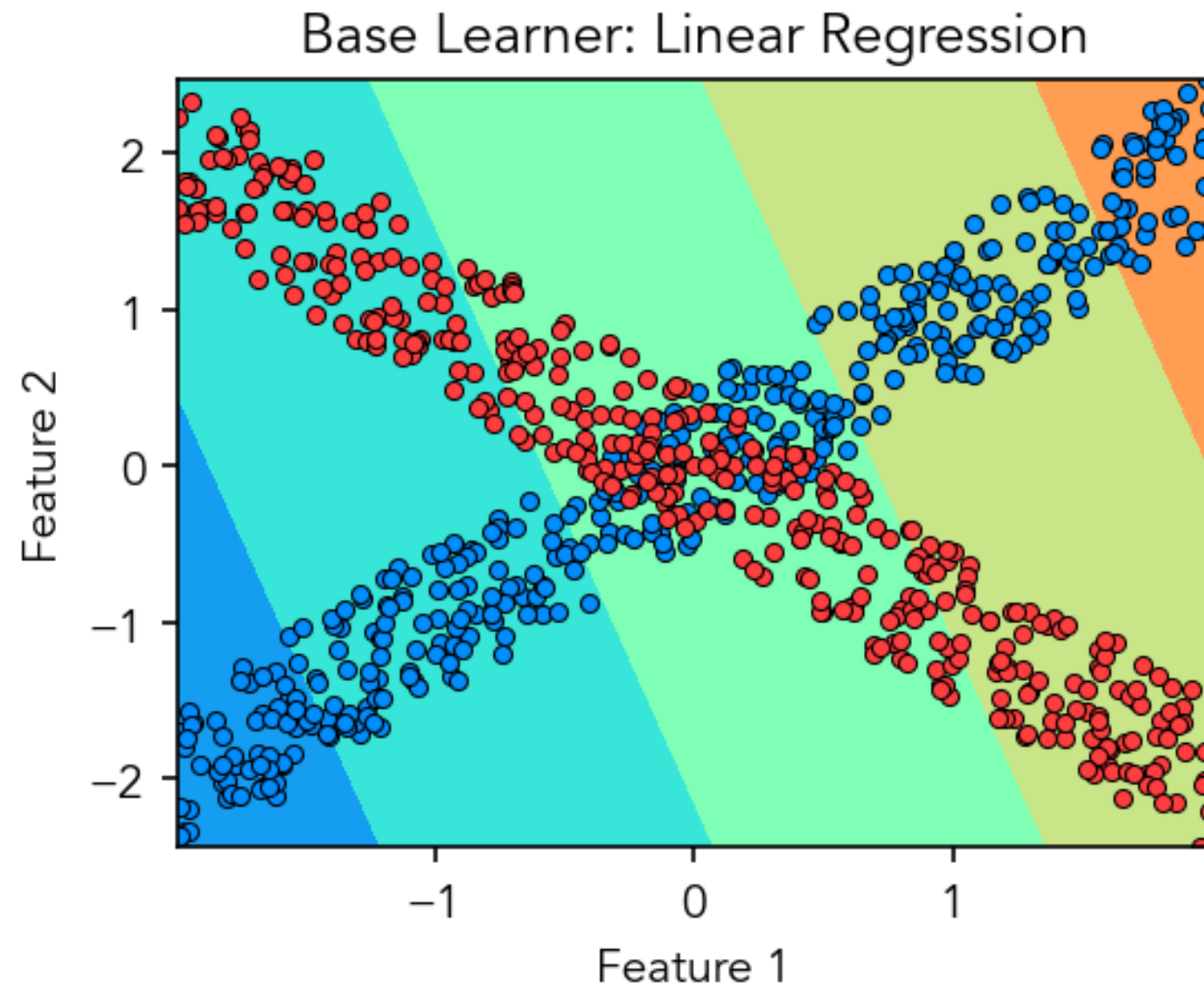
GBDT models work sequentially: we start with an initial prediction (e.g., log-odds of event rate for classification) and work with residuals iteratively using **regression** trees.

The idea is that each new tree is trying to predict the residuals of a predecessor and, as a result, the sum of predictions will approximate the observed outcome.

Instead of using gradient descent to estimate parameters (e.g., linear weights in logistic regression), we use gradient descent to estimate log-likelihood in a functional space.

We Can Use Gradient Boosting with Any Regressor*

Decision trees are not the only estimator in gradient boosting

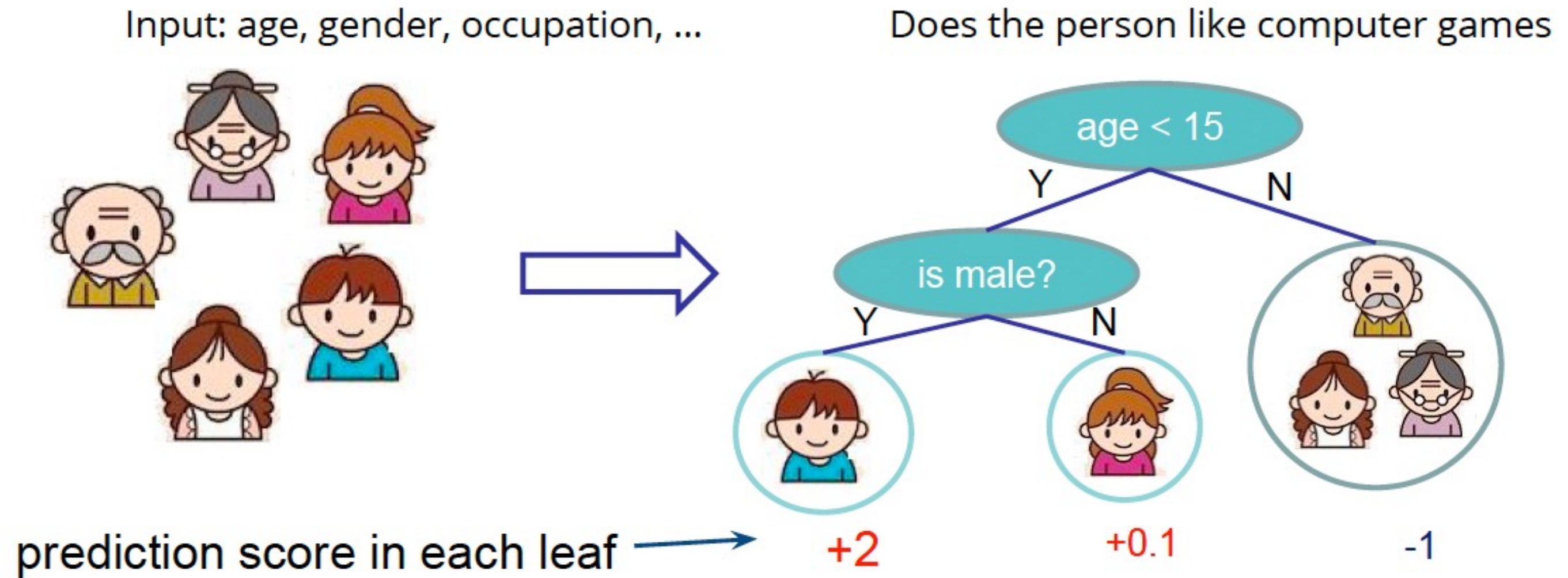


The power of gradient boosting lies in the technology. The choice of base learner controls the model's flexibility.

* But trees are still most efficient.

XGBoost – eXtreme Gradient Boosting

The “secret sauce” of tabular ML



Trees for tabular data: the *secret sauce* in machine learning

How Does XGBoost Work?

Diving into the details of the classification algorithm

🌿 Leaf weights = log odds

$$w = -\frac{G}{H + \lambda}$$

$$G = \sum g_i, H = \sum h_i$$

=== XGBoost Tree Dump ===

0:[f0<2] yes=1,no=2,missing=2

1:leaf=-2

2:leaf=1

Parsed split: feature f0 < 2.0

```
import numpy as np
import xgboost as xgb
import re

X = np.array([[0],[1],[2],[3],[4],[5]])
y = np.array([0, 0, 1, 1, 0, 1]) # binary labels
dtrain = xgb.DMatrix(X, label=y)

params = {
    "objective": "binary:logistic",
    "eval_metric": "logloss",
    "max_depth": 1,
    "eta": 1.0,
    "lambda": 0.0,
    "min_child_weight": 0,
    "gamma": 0.0
}

bst = xgb.train(params, dtrain, num_boost_round=1)

dump = bst.get_dump(dump_format="text")
print("=== XGBoost Tree Dump ===")
print(dump[0])

# Extract split rule using regex (e.g. "0:[f0<1.5]")
m = re.search(r"\[f(\d+)<([\d\.]+)\]", dump[0])
if m:
    feature_idx = int(m.group(1))
    threshold = float(m.group(2))
    print(f"\nParsed split: feature f{feature_idx} < {threshold}")
else:
    raise ValueError("No split found in tree!")
```

How Does XGBoost Work?

Diving into the details of the classification algorithm

Leaf weights = log odds

$$w = -\frac{G}{H + \lambda}$$

$$G = \sum g_i, H = \sum h_i$$

split	y=0	y=1	total	p(y=1)
x <= 2	2	1	3	33%
x > 2	1	2	3	67%

Note that for x>2 w is positive, while for x≤2 w is negative

=== XGBoost Tree Dump ===

```
0:[f0<2] yes=1,no=2,missing=2
  1:leaf=-2
  2:leaf=1
```

Parsed split: feature f0 < 2.0

Manual node stats:

```
Left  -> G=1.000, H=0.500
Right -> G=-1.000, H=1.000
```

Manual leaf weights:

```
Left leaf weight: -2.000000
Right leaf weight: 1.000000
```

```
# -----
# Manual gradient & hessian
# -----
y_pred = np.zeros_like(y, dtype=float) # first iteration raw score = 0
p = 1.0 / (1.0 + np.exp(-y_pred))      # sigmoid
g = p - y                               # gradient
h = p * (1.0 - p)                      # hessian

# Partition dataset using the actual split
left_idx = X[:, feature_idx] < threshold
right_idx = ~left_idx

def node_stats(idx):
    return g[idx].sum(), h[idx].sum()

G_L, H_L = node_stats(left_idx)
G_R, H_R = node_stats(right_idx)

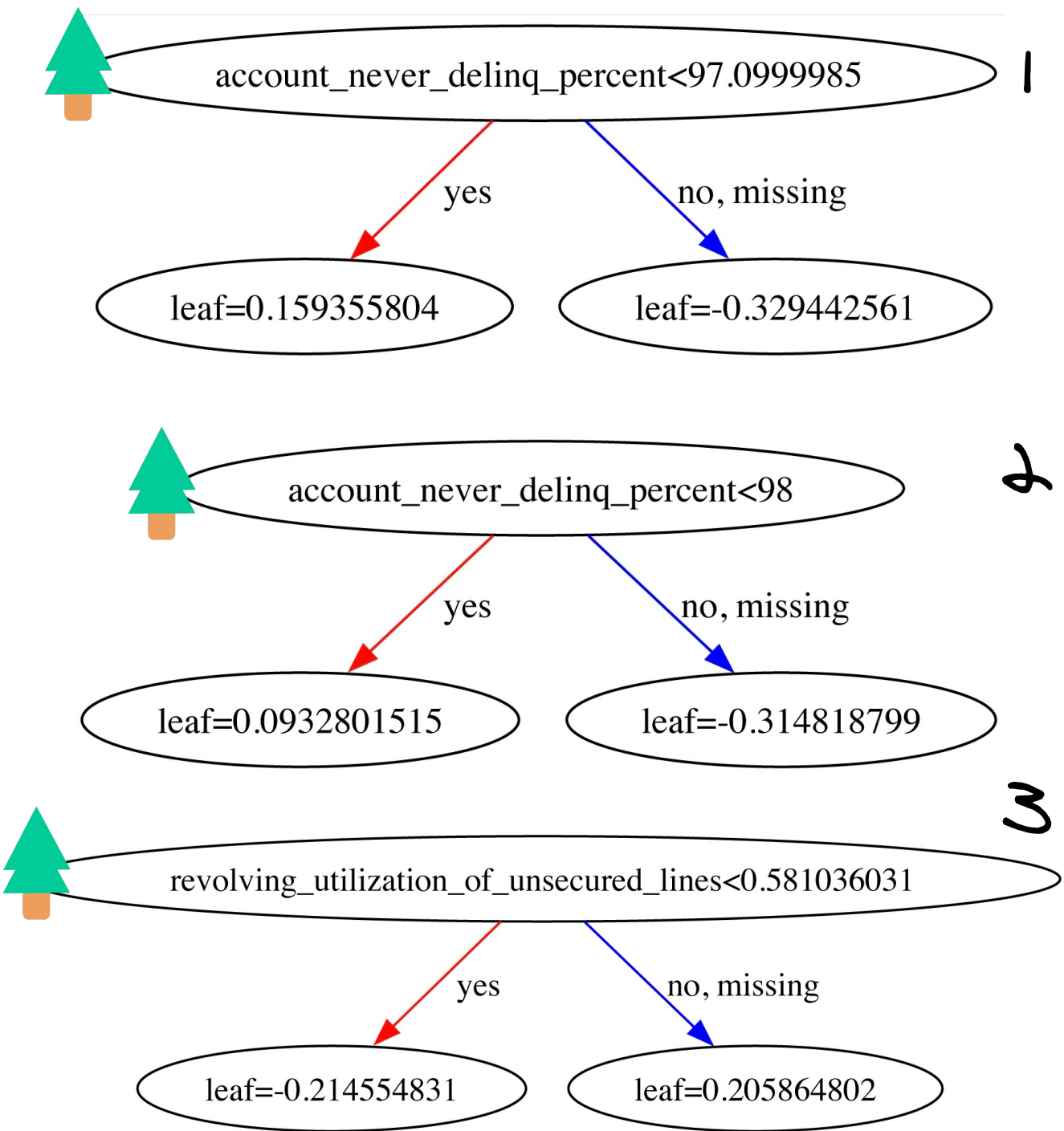
print("\nManual node stats:")
print(f"Left  -> G={G_L:.3f}, H={H_L:.3f}")
print(f"Right -> G={G_R:.3f}, H={H_R:.3f}")

# -----
# Manual leaf weights
# -----
lam = params["lambda"]
w_L = -G_L / (H_L + lam)
w_R = -G_R / (H_R + lam)

print("\nManual leaf weights:")
print(f"Left leaf weight: {w_L:.6f}")
print(f"Right leaf weight: {w_R:.6f}")
```

Example of calculating XGBoost predictions

How XGBoost combines tree contributions



In the Table below, we show a data sample for four customers:

revolving_utilization_of_unsecured_lines	account_never_delinq_percent	Prediction
0.181722	100.0	0.078548
0.344975	94.1	0.172881
0.197036	100.0	0.078548
0.383357	86.7	0.172881

We can manually reproduce the output by summing the leaf weights and a base score. If not specified, XGBoost calculates the base score automatically from data

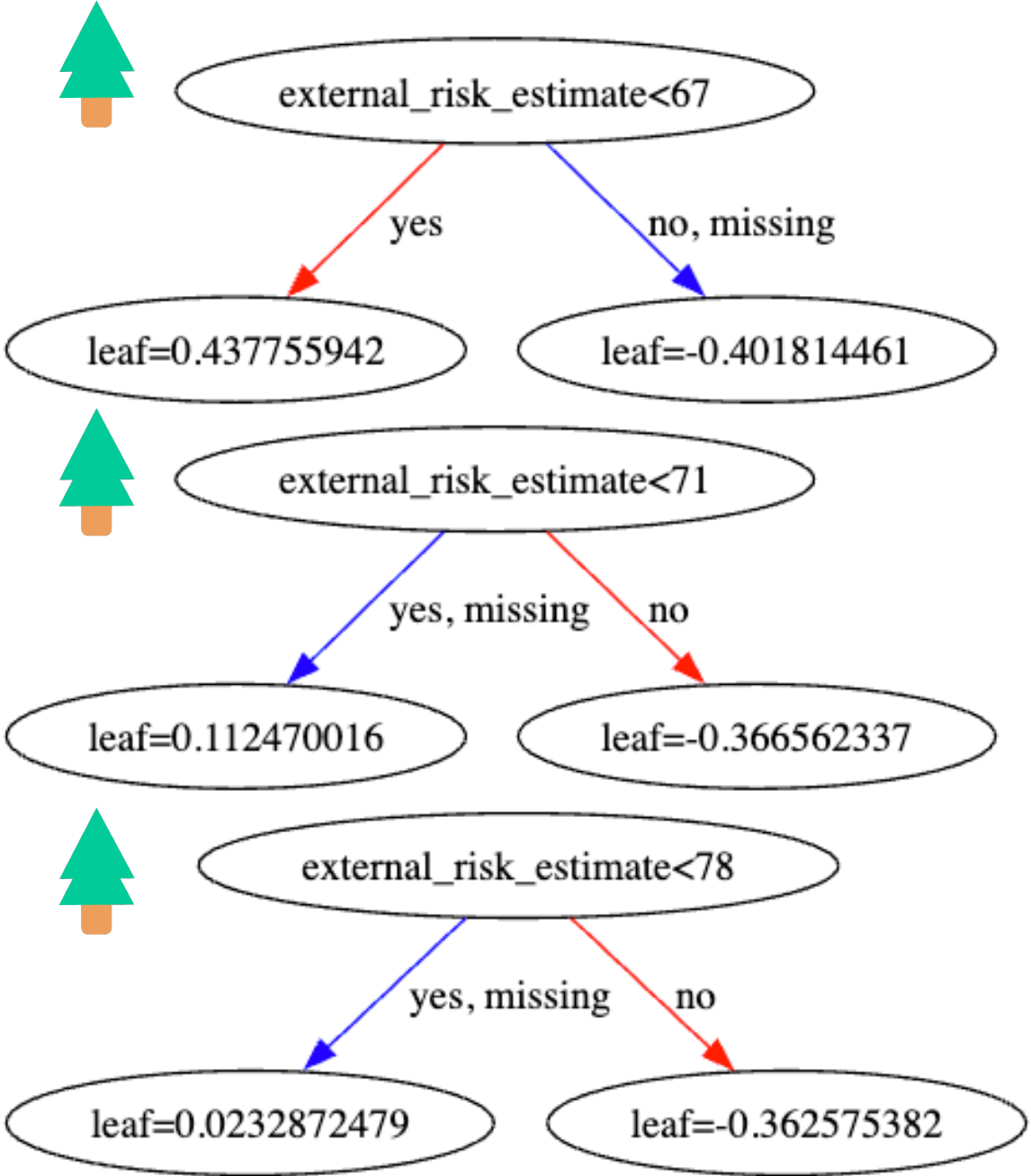
```
base_score = -1.603428

print(1 / (1 + np.exp(-(base_score + -0.329443 + -0.314819 + -0.214555)))) # first entry
# 0.07854769484530663
print(1 / (1 + np.exp(-(base_score + 0.159356 + 0.093280 + -0.214555)))) # second entry
# 0.17288072569958252
print(1 / (1 + np.exp(-(base_score + -0.329443 + -0.314819 + -0.214555)))) # third entry
# 0.07854769484530663
print(1 / (1 + np.exp(-(base_score + 0.159356 + 0.093280 + -0.214555)))) # fourth entry
# 0.17288072569958252
```

What is an intuitive interpretation of the leaf values in XGBoost base learners? ([link](#)).

Parsing XGBoost tree stumps

Reading splits and leaf weights

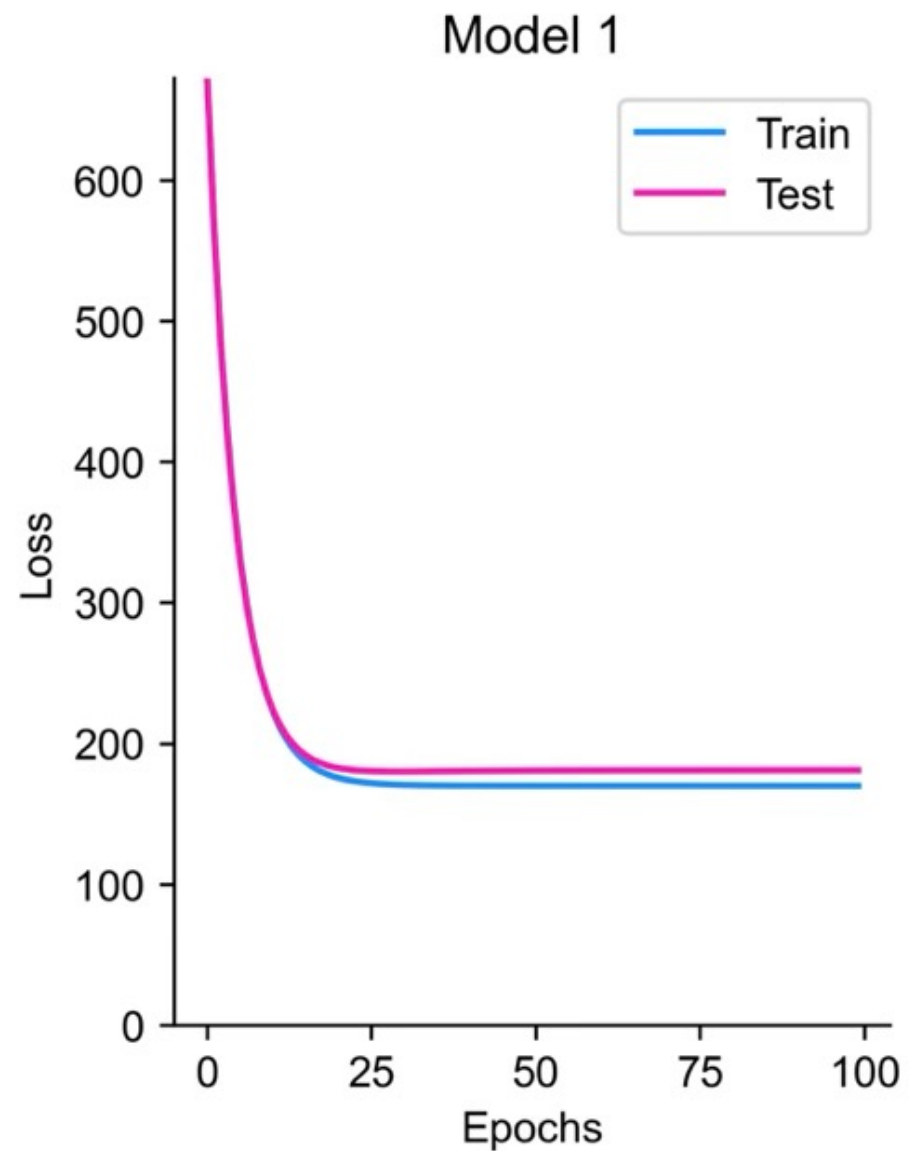


Tree	Feature	Sign	Split	XAddEvidence	Log odds
0	external_risk_estimate	<	67.000000	0.437756	
0	external_risk_estimate	>=	67.000000	-0.401814	
1	external_risk_estimate	<	71.000000	0.112470	
1	external_risk_estimate	>=	71.000000	-0.366562	
2	external_risk_estimate	<	78.000000	0.023287	
2	external_risk_estimate	>=	78.000000	-0.362575	
3	account_never_delinq_percent	<	97.699997	0.651601	
3	account_never_delinq_percent	>=	97.699997	-0.600417	
4	revolving_utilization_of_unsecured_lines	<	0.531367	-0.398247	
4	revolving_utilization_of_unsecured_lines	>=	0.531367	0.515060	
5	account_never_delinq_percent	<	97.000000	0.182951	
5	account_never_delinq_percent	>=	97.000000	-0.501965	
6	revolving_utilization_of_unsecured_lines	<	0.287150	-0.353462	
6	revolving_utilization_of_unsecured_lines	>=	0.287150	0.165219	
7	revolving_utilization_of_unsecured_lines	<	0.906686	-0.157069	
7	revolving_utilization_of_unsecured_lines	>=	0.906686	0.267321	
8	account_never_delinq_percent	<	98.400002	0.115218	
8	account_never_delinq_percent	>=	98.400002	-0.493801	

Bias Variance Trade-off

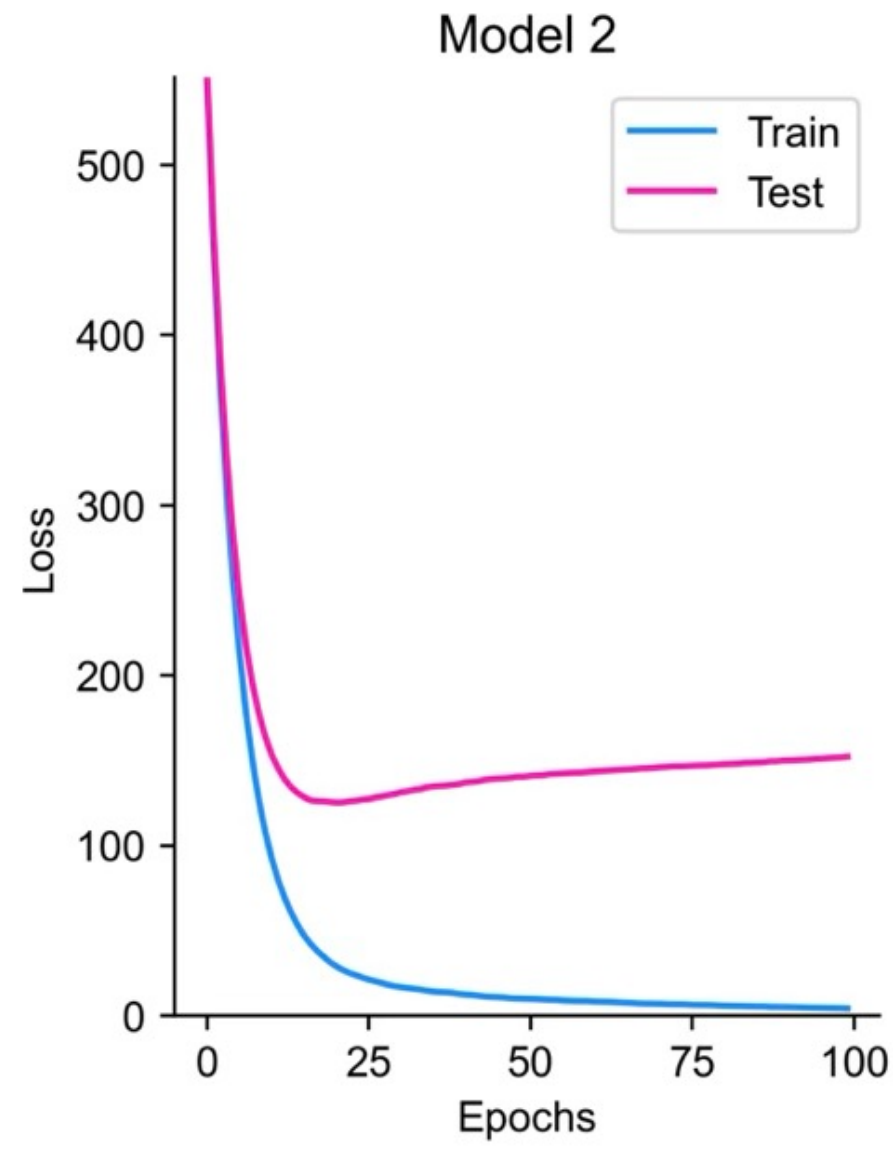
The holy quest of machine learning

Underfitting



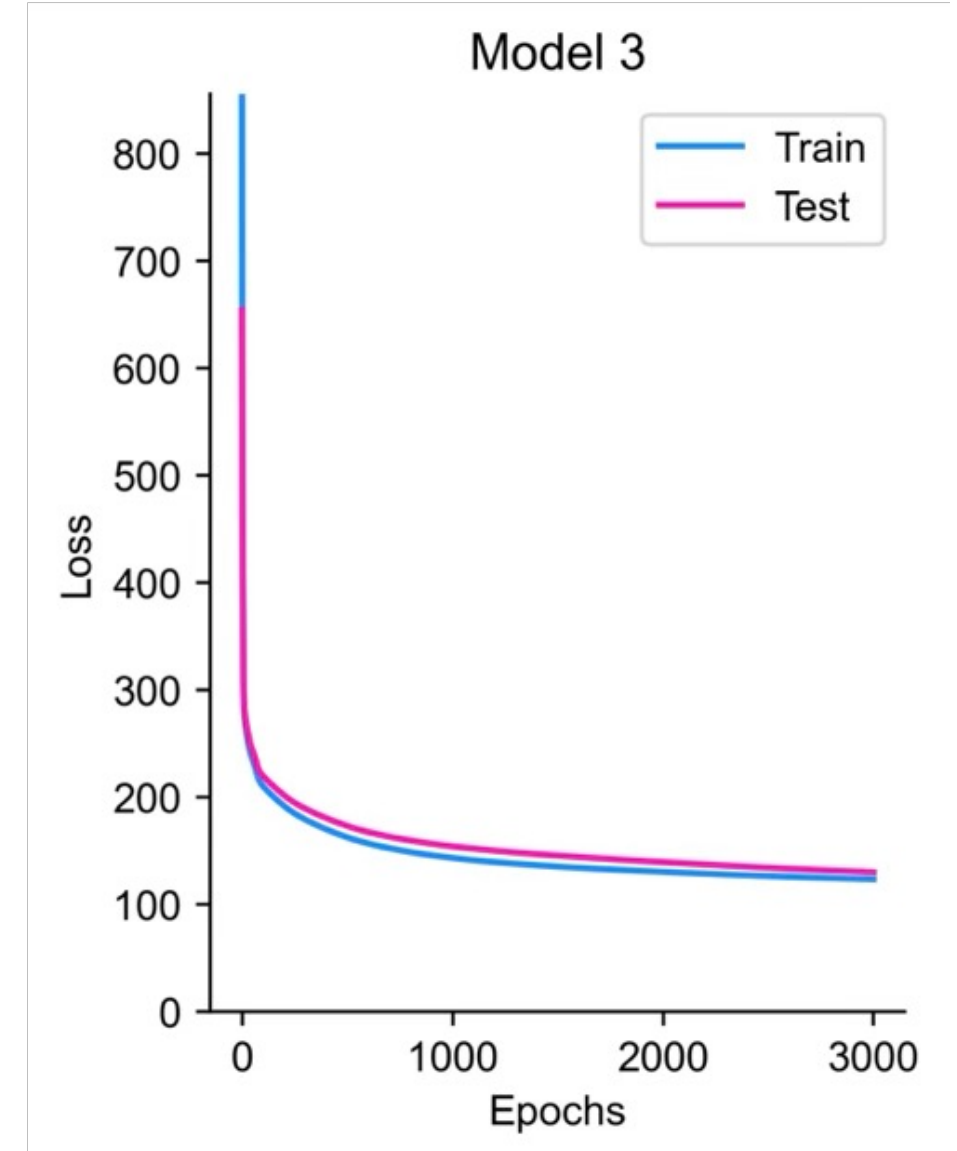
Both train and test errors are high

Overfitting



Train loss is low while the test loss is high

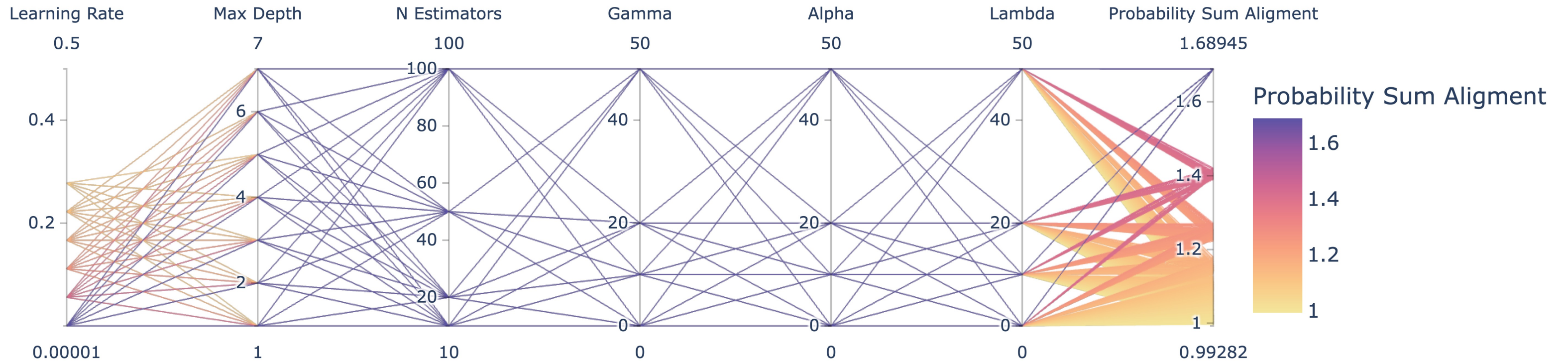
Just right



Balance between underfitting and overfitting

XGBoost – Hyperparameter Effects

Regularization and probability calibration



XGBoost offers several parameters that allow to impose regularization constraints during training. They are documented on a relatively high level, and their behavior is not so widely explained. Among these are:

- **Gamma** (γ): “pseudo-regularization” parameter influencing how XGBoost prunes tree branches to minimize model complexity
- **Alpha** (α): L1 (Lasso) regularization parameter of leaf weights, helping the model to focus on a subset of leaf weights
- **Lambda** (λ): L2 (Ridge) regularization parameter of leaf weights acting a smoothing mechanism to prevent large leaf weights

Intercept in XGBoost models

Understanding the base score

```
import numpy as np
import xgboost as xgb

# Set XGBoost parameters
params = dict(
    objective="binary:logistic", max_depth=1, base_score=0.5,
)
params_log_raw = dict(
    objective="binary:logitraw", max_depth=1, base_score=0.0, #logit(0.5)
)
def loglikelihood(preds, train_data):
    eps = 1e-16
    labels, preds = train_data.get_label(), 1.0 / (1.0 + np.exp(-preds))
    grad, hess = preds - labels, preds * (1.0 - preds)
    hess[hess < eps] = eps
    return grad, hess

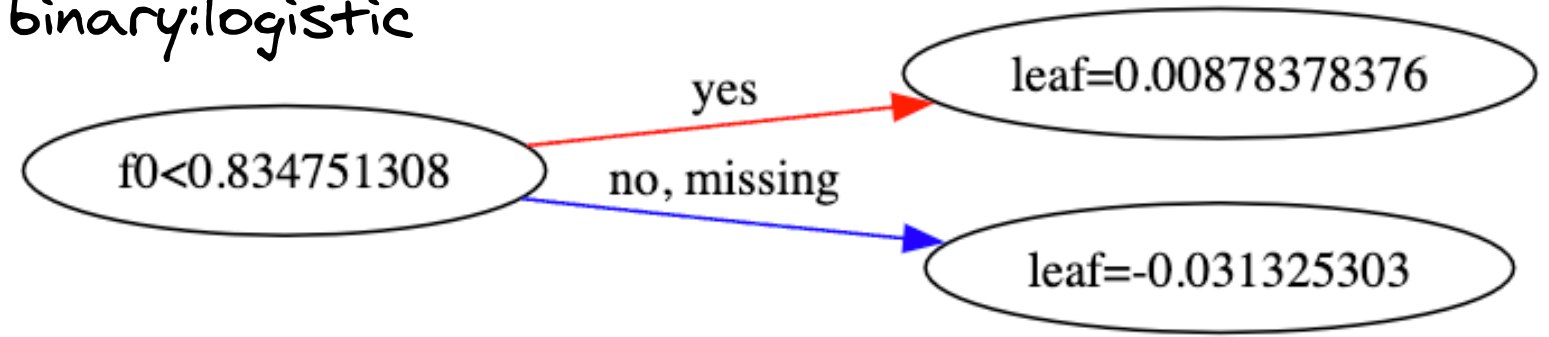
params_log_loss = dict(
    max_depth=1, base_score=0.0, disable_default_eval_metric=1,
)

# Train the XGBoost model
num_rounds, base_score = 3, np.mean(y_train)

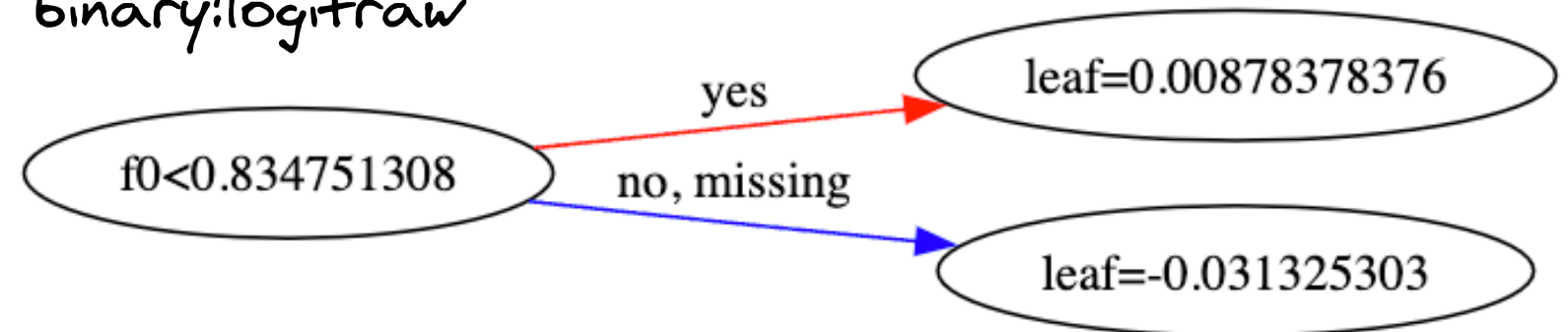
# Train with different parameter sets
xgb_clf = xgb.train(params, dtrain, num_rounds)
xgb_clf_raw = xgb.train(params_log_raw, dtrain, num_rounds)
xgb_clf_ll = xgb.train(
    params_log_loss, dtrain, num_rounds, obj=loglikelihood
)

# Visualize the first tree from each model
display(xgb.to_graphviz(xgb_clf, num_trees=0, rankdir="LR"))
display(xgb.to_graphviz(xgb_clf_raw, num_trees=0, rankdir="LR"))
display(xgb.to_graphviz(xgb_clf_ll, num_trees=0, rankdir="LR"))
```

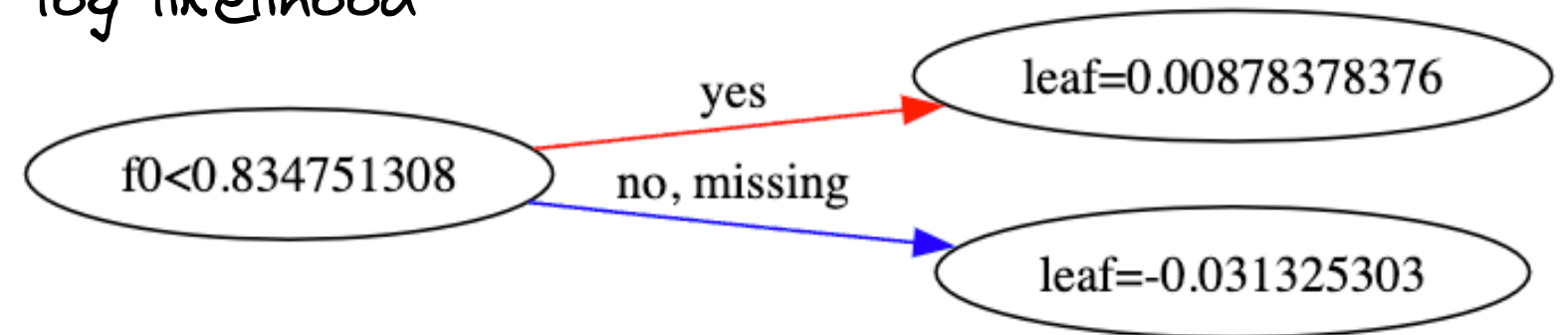
binary:logistic



binary:logitraw



log likelihood



1. When you do a customized loss function, the built-in evaluation error assumes input is after logistic transformation ([link](#)).

Constraining XGBoost models

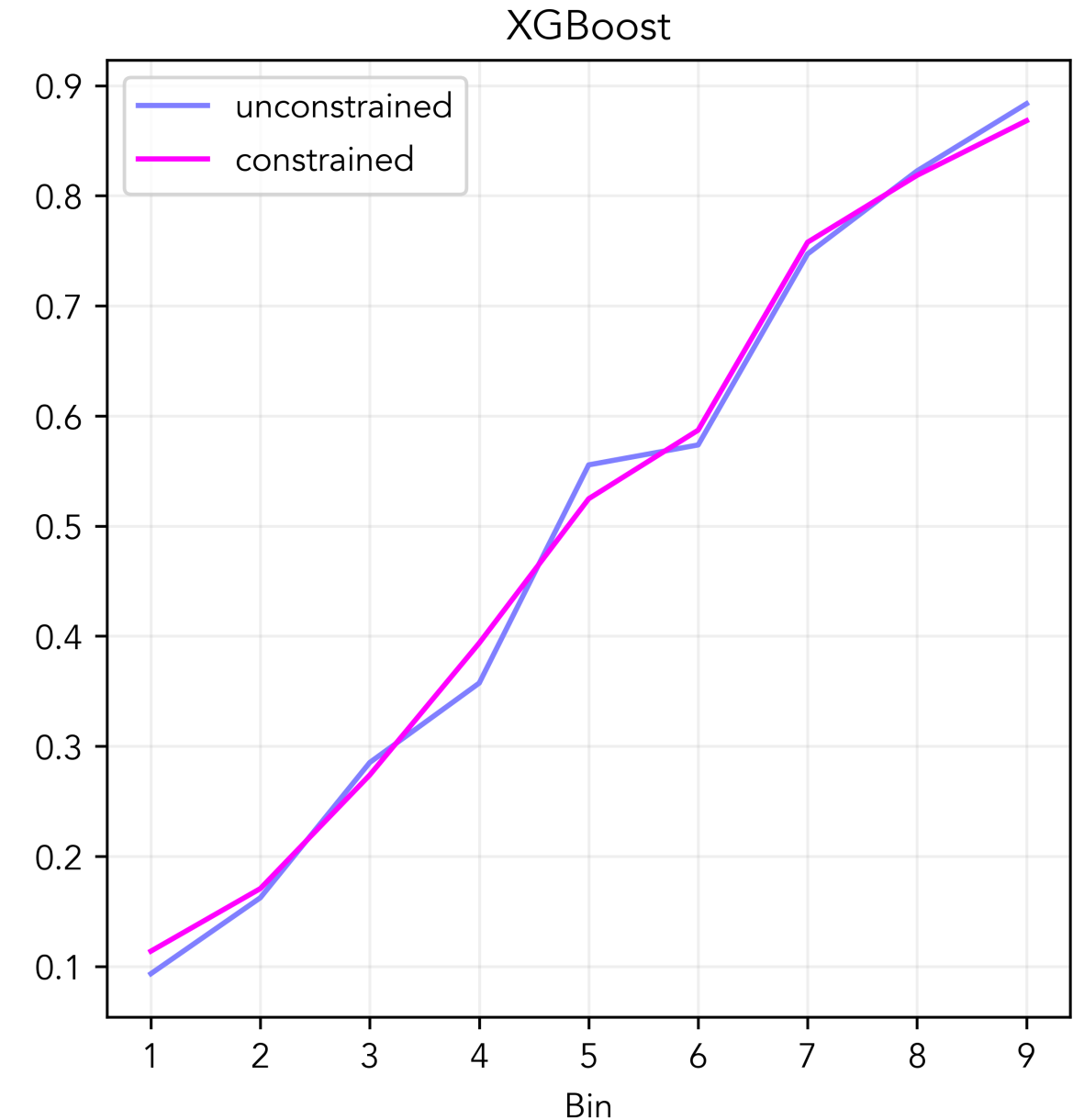
Imposing monotonicity on features

```
import xgboost as xgb

# Monotonic constraints: +1 means prediction
# increases with feature,
# -1 means prediction decreases, 0 means no
# constraint
mono_cst = "(1, -1, 0)" # for 3 features

clf = xgb.XGBClassifier(
    objective="binary:logistic",
    monotone_constraints=mono_cst,
    n_estimators=50,
    max_depth=3,
)

clf.fit(X_train, y_train)
```



Monotonic constraints allow to impose business logic on the direction of splits that GBDT model makes

Constraining XGBoost models

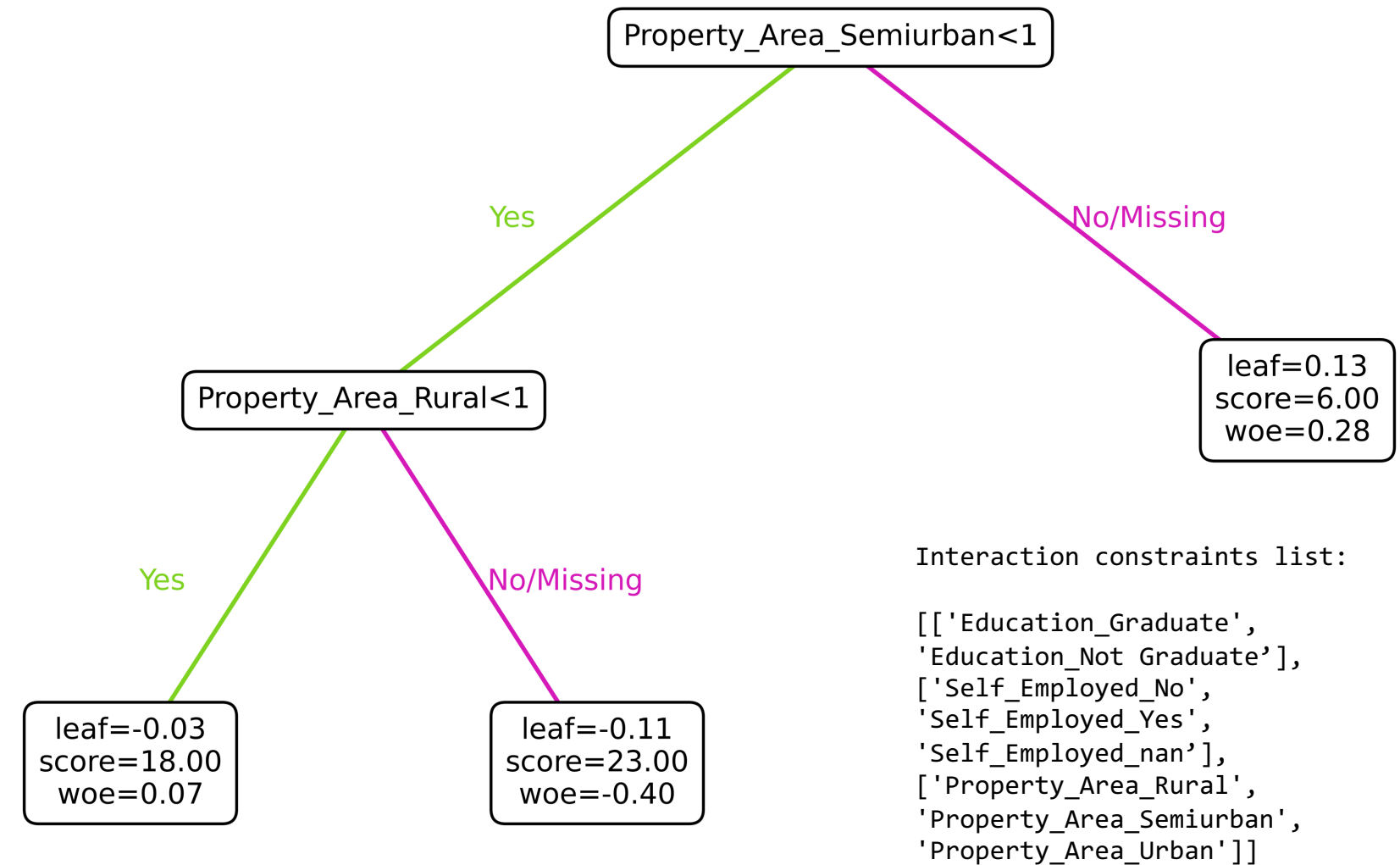
Controlling feature interactions

```
import xgboost as xgb

# Define interaction constraints: each feature
# can only interact with itself
int_cst = [[i] for i in range(x_train.shape[1])]

# Train XGBoost classifier with constraints
clf = xgb.XGBClassifier(
    objective="binary:logistic",
    interaction_constraints=int_cst,
    n_estimators=20,
    max_depth=3,
    # other parameters as needed...
)

clf.fit(X_train, y_train)
```



Interaction constraints allow to impose limitations on which features can interact within a tree

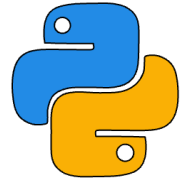
logitboost.

xgboost.

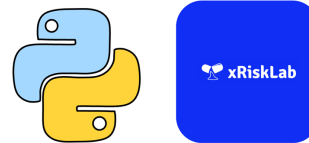
xbooster.

xbooster

Explainable boosted scoring

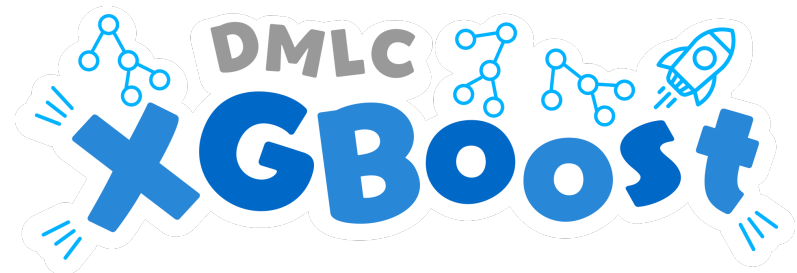


```
!pip install xbooster
```



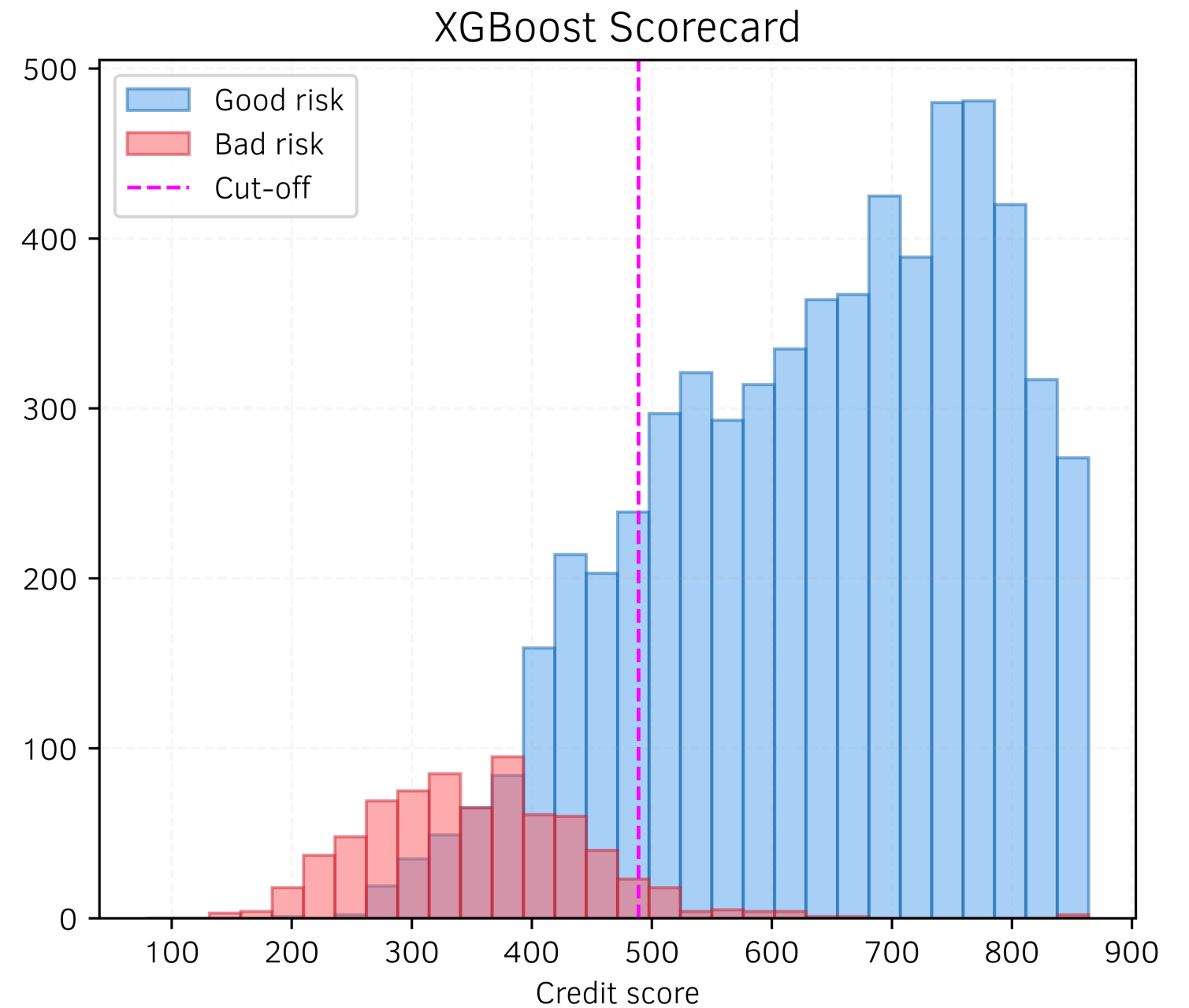
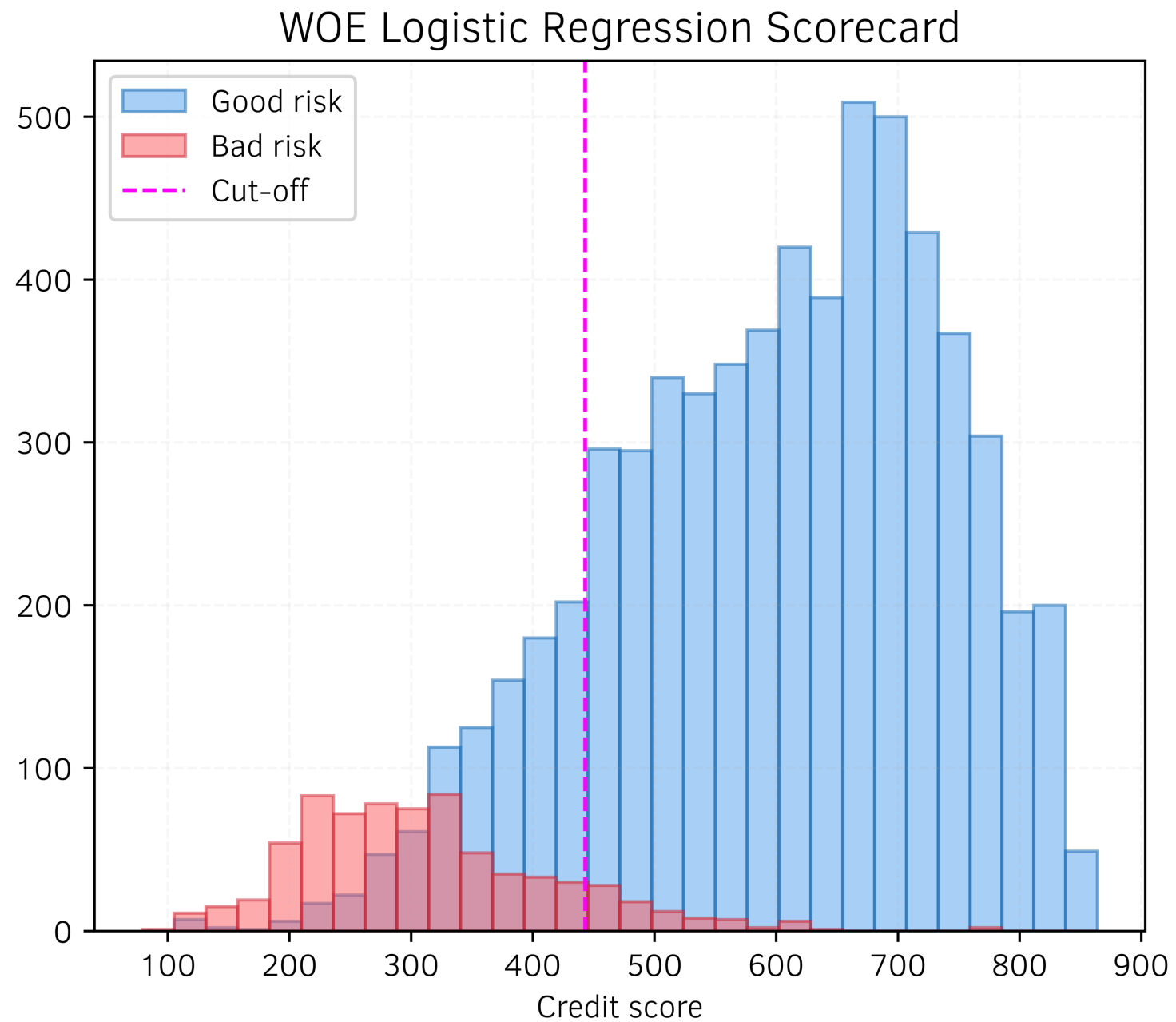
xbooster

Explainable Boosted Scoring



Can Gradient-Boosted Trees Produce Scorecards?

A comparison of a standard scorecards with a boosted one



Converting XGBoost Models to Scorecards

First, we fit an XGBoost model and then build a scorecard

```
import numpy as np
import xgboost as xgb

# Define best parameters
best_params = dict(
    n_estimators=100,
    learning_rate=0.55,
    max_depth=1,
    min_child_weight=10,
    grow_policy="lossguide",
    early_stopping_rounds=5,
)

# Create an XGBoost model
xgb_model = xgb.XGBClassifier(
    **best_params,
    random_state=62
)

# Define evaluation set
evalset = [
    (X.loc[ix_train], y.loc[ix_train]),
    (X.loc[ix_test], y.loc[ix_test]),
]

# Fit the XGBoost model
xgb_model.fit(
    X.loc[ix_train],
    y.loc[ix_train],
    eval_set=evalset,
    verbose=False,
)
```

```
from xgboster.constructor import XGBScorecardConstructor

# Set up the scorecard constructor
scorecard_constructor = XGBScorecardConstructor(
    xgb_model,
    X.loc[ix_train],
    y.loc[ix_train],
)

# Construct the scorecard
xgb_scorecard = scorecard_constructor.construct_scorecard()

# Create a scorecard with points
xgb_scorecard_with_points = scorecard_constructor.create_points(
    pdo=50, # Points to double odds
    target_points=600, # Points at target odds
    target_odds=50, # Target odds (e.g., 50:1)
)

# Make predictions using the scorecard
credit_scores = scorecard_constructor.predict_score(X.loc[ix_test])
```

xbooster is a Python wrapper on top of XGBoost (+ CatBoost) and provides interpretability and scorecard modeling functionality for stumps and GBDT models with tree depth > 1

Beyond Stumps: Interpreting Deeper Trees

Calculating marginal contributions for splits with feature interactions

```
First condition (split component)
| Condition A: DPD < 98
| (single-condition stats on full sample)
| L(θ2 | A) = 2.839 [Condition EventRate=0.2398, WOE=+1.0438]
v
Second condition (split component)
| Condition B: Utilization < 0.6093
| (single-condition stats on full sample)
| L(θ2 | B) = 0.429 [Condition EventRate=0.0455, WOE=-0.8451]
v
LEAF for two conditions, A ∧ B
L(θ1 | A ∧ B) = 1.251 [Split EventRate=0.1220, WOE=+0.2240]
```

- With stumps (depth=1), each tree is just a set of **additive log-odds** (easy to explain).
- With deeper trees, each leaf corresponds to **combinations of conditions** (feature interactions).
- The odds in a node vs. global odds gives a **likelihood ratio** → $\log(\text{LR}) = \text{weight of evidence (WOE)}$.
- Relative importance of conditions in xbooster is measured using **likelihood ratios** (split likelihood ratio **θ1** vs leaf likelihood ratio **θ2**).

Example of tree depth = 2 (2-way interaction)

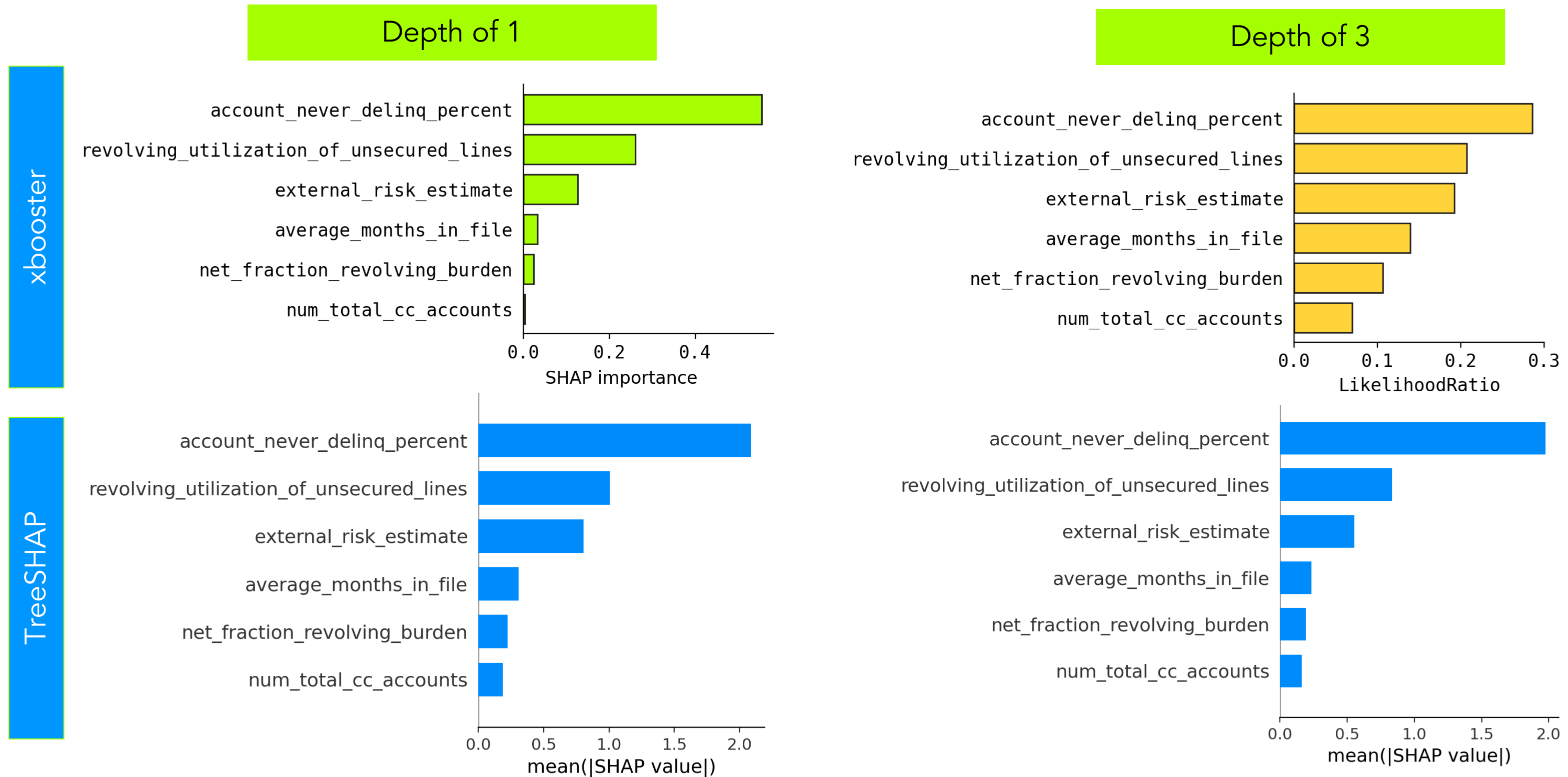
Tree	0	0
Node	3	3
Feature in split	DPD	Utilization
Likelihood $L(\theta_1 x)$	1.251	1.251
Likelihood $L(\theta_2 x)$	2.839	0.429
$L(\theta_1 x) / L(\theta_2 x)$	$1.251/2.839=0.44$	$1.251/0.429=2.91$
WOE 1	0.2240	0.2240
WOE 2	1.04376	-0.845
WOE2 - WOE 1	-0.812	1.069
$e^{(\text{WOE2} - \text{WOE1})}$	$e^{(-0.812)}=0.44$	$e^{(1.069)}=2.91$

Likelihood ratio is e^{WOE}
WOE 1 (in split) = interaction of conditions (same)
WOE 2 (in leaf) = individual condition (by feature)

$$\text{Relative Likelihood} = \frac{L(\theta_1 \mid x)}{L(\theta_2 \mid x)}$$

Global Explainability with xbooster

xbooster explanations match TreeSHAP closely (but faster!)

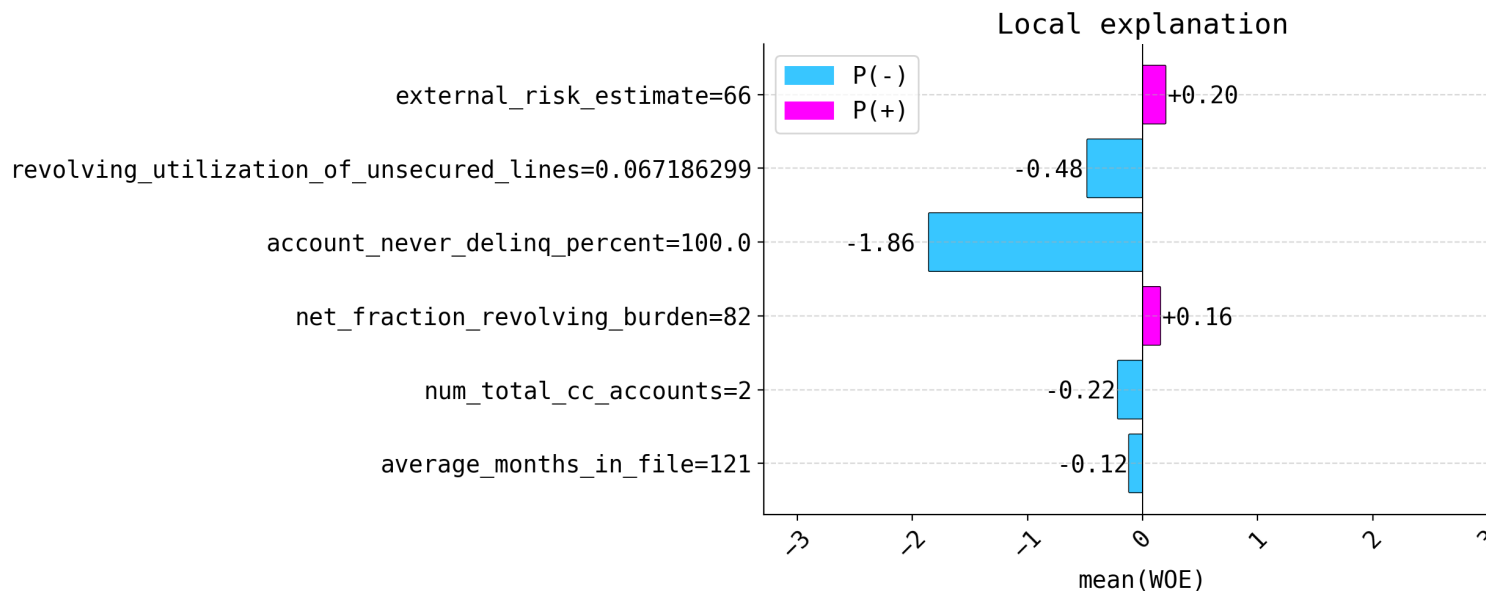


Local Explainability with xbooster

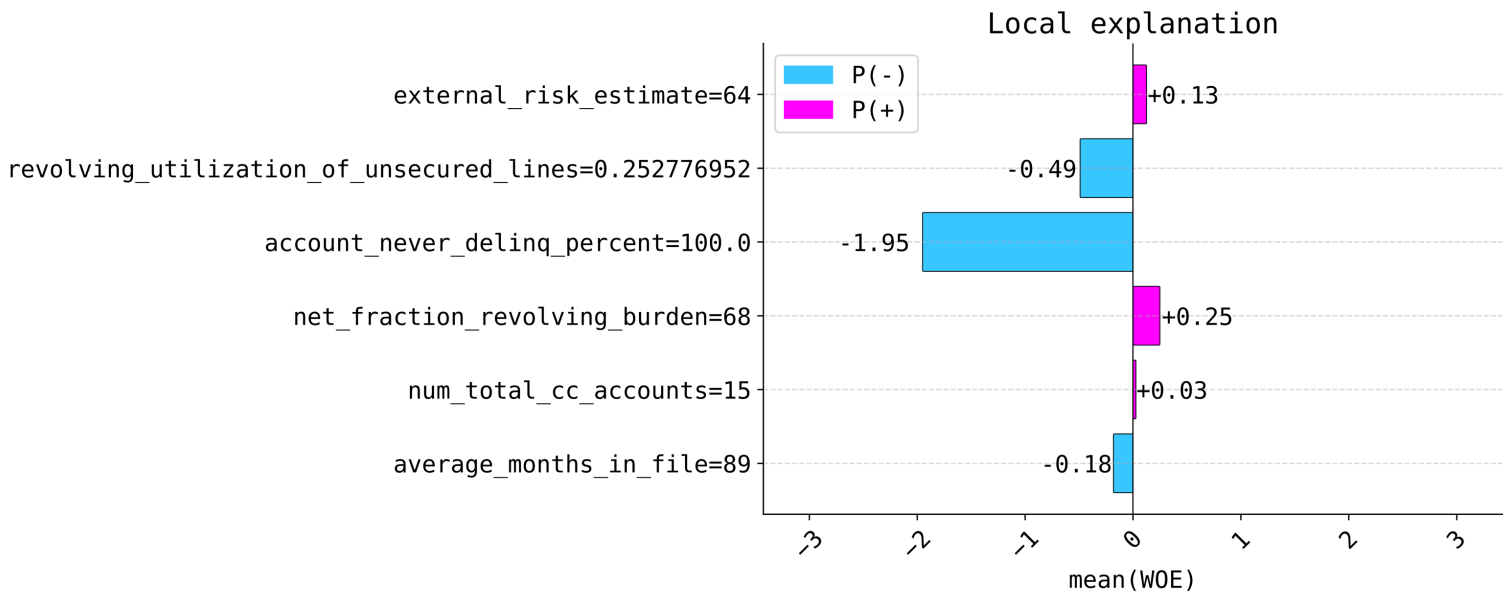
xbooster explanations match TreeSHAP closely (but faster!)

xbooster

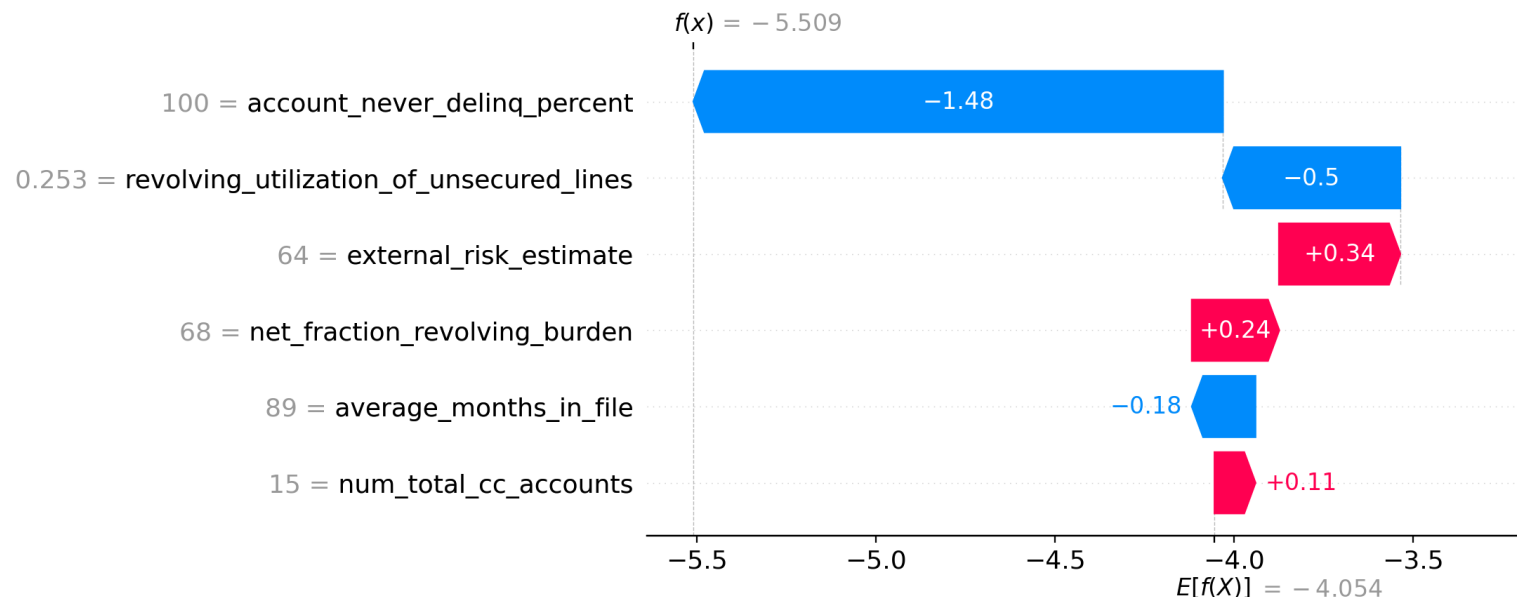
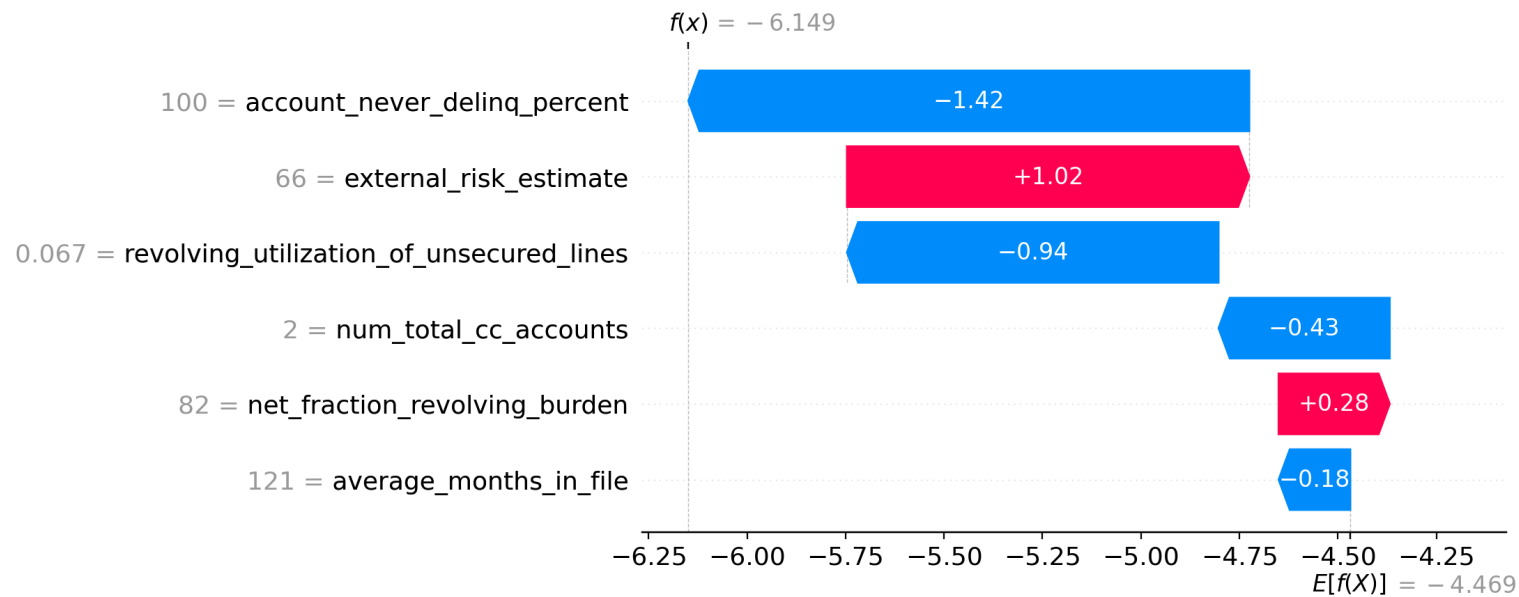
Depth of 1



Depth of 3

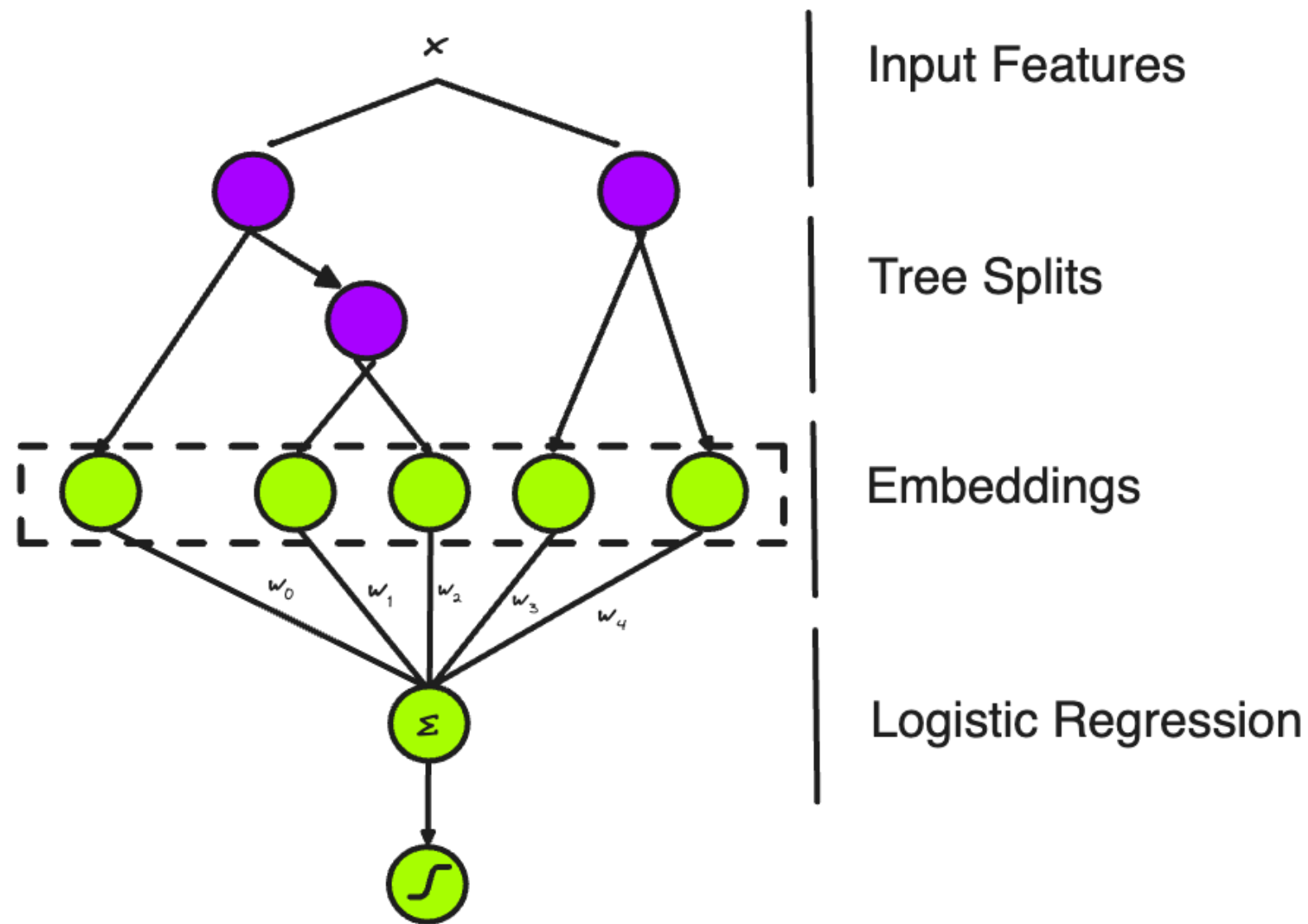


TreeSHAP



From XGBoost trees to logistic regression

Tree leaves as embeddings



One common technique to convert a gradient-boosted decision tree model to a linear model (with non-linearly transformed inputs) is to use one-hot-encoding for the boosted trees' leaf indices.

The trick here for learning non-linear transformations is to bin the feature and treat the bin index as a categorical feature (He et al. 2014).

With this approach, we treat each individual tree as a categorical feature that takes as value the index of the leaf an instance ends up falling in.

This allows to linearize the boosted tree models and even treat them in the credit scoring framework.

From XGBoost trees to logistic regression

Tree leaves as embeddings

Bin ids of each boosted tree's leaves 🌿

	tree_1	tree_2	tree_3	tree_4	tree_5	tree_6	tree_7	tree_8	tree_9	tree_10
0	4.0	3.0	4.0	3.0	3.0	4.0	4.0	3.0	4.0	3.0
1	6.0	6.0	5.0	6.0	5.0	5.0	6.0	6.0	5.0	5.0
2	6.0	6.0	5.0	6.0	5.0	5.0	6.0	6.0	5.0	5.0
3	3.0	4.0	3.0	4.0	4.0	3.0	3.0	4.0	3.0	4.0
4	3.0	4.0	3.0	4.0	4.0	3.0	3.0	4.0	3.0	4.0
...	...	...	...	...	...	...	...	...	...	...
6995	4.0	3.0	4.0	3.0	3.0	4.0	3.0	3.0	4.0	3.0
6996	6.0	6.0	5.0	6.0	5.0	5.0	6.0	6.0	5.0	5.0
6997	6.0	6.0	5.0	6.0	5.0	5.0	6.0	6.0	5.0	5.0
6998	6.0	6.0	5.0	6.0	5.0	5.0	6.0	6.0	5.0	5.0
6999	6.0	6.0	5.0	6.0	5.0	5.0	6.0	6.0	5.0	5.0

Here we show an example of an output DataFrame, which contains leaf indices from the first ten boosting iterations

One-hot encoded
tree 1

	tree_1_4.0	tree_1_6.0
0	1	0
1	0	1
2	0	1
3	0	0
4	0	0
5	0	1

One-hot encoded
tree 2

	tree_2_4.0	tree_2_6.0
0	0	0
1	0	1
2	0	1
3	1	0
4	1	0
5	0	1

Leaf indices are unique per tree but not across trees.
We can convert each feature-index to a one-hot-
encoded representation ("dummies")

From XGBoost trees to logistic regression

Tree leaves as weights of evidence

Alternatively, we can fit a Logistic Regression using the credit approach. Below you can see an example of the leaf weights (classification margins or “XAddEvidence”, terminal nodes of decision trees) 🌿

	tree_1	tree_2	tree_3	tree_4	tree_5	tree_6	tree_7	tree_8	tree_9	tree_10
0	-0.027854	-0.050002	-0.036584	-0.035063	-0.046073	-0.023115	-0.064182	-0.038422	-0.036345	-0.043682
1	-0.115493	-0.113322	-0.112469	-0.109658	-0.108780	-0.107414	-0.105274	-0.103966	-0.104442	-0.103398
2	-0.115493	-0.113322	-0.112469	-0.109658	-0.108780	-0.107414	-0.105274	-0.103966	-0.104442	-0.103398
3	0.186526	0.140952	0.130567	0.129051	0.095748	0.123296	0.065871	0.082802	0.076264	0.064644
4	0.186526	0.140952	0.130567	0.129051	0.095748	0.123296	0.065871	0.082802	0.076264	0.064644
...	...	...	...	...	...	...	...	...	...	...
6995	-0.027854	-0.050002	-0.036584	-0.035063	-0.046073	-0.023115	0.065871	-0.038422	-0.036345	-0.043682
6996	-0.115493	-0.113322	-0.112469	-0.109658	-0.108780	-0.107414	-0.105274	-0.103966	-0.104442	-0.103398
6997	-0.115493	-0.113322	-0.112469	-0.109658	-0.108780	-0.107414	-0.105274	-0.103966	-0.104442	-0.103398
6998	-0.115493	-0.113322	-0.112469	-0.109658	-0.108780	-0.107414	-0.105274	-0.103966	-0.104442	-0.103398
6999	-0.115493	-0.113322	-0.112469	-0.109658	-0.108780	-0.107414	-0.105274	-0.103966	-0.104442	-0.103398

From XGBoost trees to logistic regression

Tree leaves as weights of evidence

The values shown below, if divided by the learning rate, will look like WOE scores commonly used in classical scorecard models of credit risk (with negated signs, bad to good) 🎲

	tree_1	tree_2	tree_3	tree_4	tree_5	tree_6	tree_7	tree_8	tree_9	tree_10
0	-0.278543	-0.500020	-0.365842	-0.350632	-0.460732	-0.231152	-0.641821	-0.384222	-0.363448	-0.436815
1	-1.154928	-1.133223	-1.124693	-1.096582	-1.087804	-1.074142	-1.052737	-1.039665	-1.044419	-1.033982
2	-1.154928	-1.133223	-1.124693	-1.096582	-1.087804	-1.074142	-1.052737	-1.039665	-1.044419	-1.033982
3	1.865259	1.409518	1.305674	1.290506	0.957481	1.232955	0.658707	0.828023	0.762642	0.646442
4	1.865259	1.409518	1.305674	1.290506	0.957481	1.232955	0.658707	0.828023	0.762642	0.646442
...	...	...	...	...	...	...	...	...	...	...
6995	-0.278543	-0.500020	-0.365842	-0.350632	-0.460732	-0.231152	0.658707	-0.384222	-0.363448	-0.436815
6996	-1.154928	-1.133223	-1.124693	-1.096582	-1.087804	-1.074142	-1.052737	-1.039665	-1.044419	-1.033982
6997	-1.154928	-1.133223	-1.124693	-1.096582	-1.087804	-1.074142	-1.052737	-1.039665	-1.044419	-1.033982
6998	-1.154928	-1.133223	-1.124693	-1.096582	-1.087804	-1.074142	-1.052737	-1.039665	-1.044419	-1.033982
6999	-1.154928	-1.133223	-1.124693	-1.096582	-1.087804	-1.074142	-1.052737	-1.039665	-1.044419	-1.033982

From XGBoost trees to logistic regression

Tree leaves as weights of evidence

```
from sklearn.linear_model import LogisticRegression

# Get tree leaf features from the trained XGBoost model
dataframe_with_trees = scorecard_constructor.get_leafs(X.loc[ix_train])
feature_trees = dataframe_with_trees.columns

# Fit a logistic regression on the leaf values directly
xgboost_logreg = LogisticRegression(fit_intercept=True)
xgboost_logreg.fit(dataframe_with_trees, y.loc[ix_train])

print(xgboost_logreg.intercept_, xgboost_logreg.coef_) # Print coefficients
```

Intercept	[-1.62352526]
Tree (feature) weights	[1.2020327 0.97575971 1.15797107 0.94565509 0.95508742 0.90668016 1.28900426 1.20586514 0.81415158 0.74754902 0.75442047 1.15835286 1.00462902 1.07186301 1.32491479 0.86561505 0.86455394 1.61863739 1.00303684 0.94733892 1.0009415 1.07450323 0.8456734 1.26297626 1.4588525 1.05754771 1.03020566 1.29043286 1.06841534 1.37002718 1.25652411 1.16154753 1.10961945 1.1119615 1.32266487 1.3082921 1.14207318 1.50009438 1.2873827 1.02255989 1.20146479 1.19012363 1.01301803 1.12589038 0.71898659 1.07271603 1.20889274 1.10049925 0.99414106 1.24884242 1.1818015 1.26451315 1.13617215 0.79867256 0.79247341 0.97902319 1.36307041 0.88100753 0.99550972 1.00467602 0.69304552 0.69596907 0.82996096 0.94739084 1.37195452 0.82122443 0.89276536 1.01786259 1.20976927 0.60941342 0.68983199 0.82426148 0.81495502 1.01811936 1.07077166 0.69975434 0.61008468 0.77539232 0.58559521 0.96066049 0.91404242 0.73222261 0.79444167 0.59809685 0.83246552 0.93542351 0.80768764 0.58264955 0.53551173 0.69235595 0.94476154 0.44657866 0.53830562 0.72493853 0.74866298 1.11622481 1.03748263 0.55242067 0.50116463 0.43115759]

Notice that all coefficients are positive and the intercept is negative.

Exactly what you would see in a WOE Logistic Regression (bad / good)!

CatBoost Scorecards

Building scorecards with CatBoost in xbooster

```
from catboost import CatBoostClassifier, Pool
from xbooster.constructor import (
    CatBoostScorecardConstructor
)

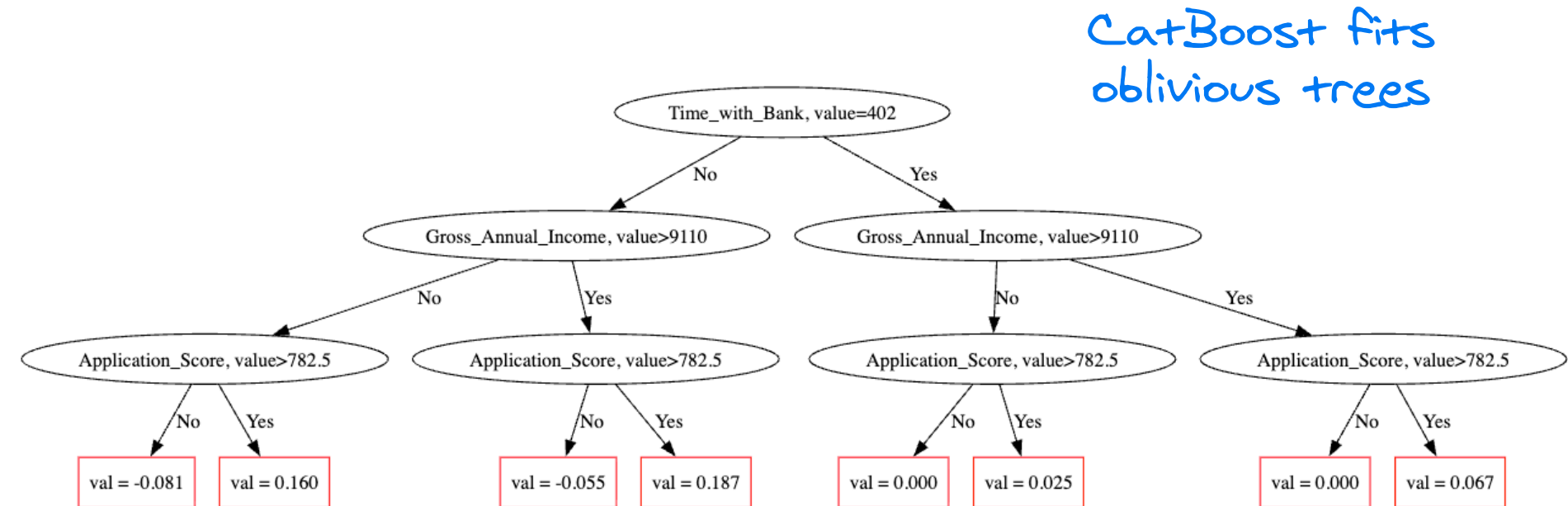
# Create CatBoost Pool
pool = Pool(
    data=X,
    label=y,
    cat_features=categorical_features,
)

# Initialize and train CatBoost model
model = CatBoostClassifier(
    iterations=100,
    allow_writing_files=False,
    depth=3,
    learning_rate=0.1,
    verbose=0,
    one_hot_max_size=9999, # Key for interpretability
)
model.fit(pool)

constructor = CatBoostScorecardConstructor(model, pool)

# Construct the scorecard
scorecard = constructor.construct_scorecard()

raw_scores = constructor.predict_score(X, method="raw")
woe_scores = constructor.predict_score(X, method="woe")
points_scores = constructor.predict_score(X, method="pdo")
```



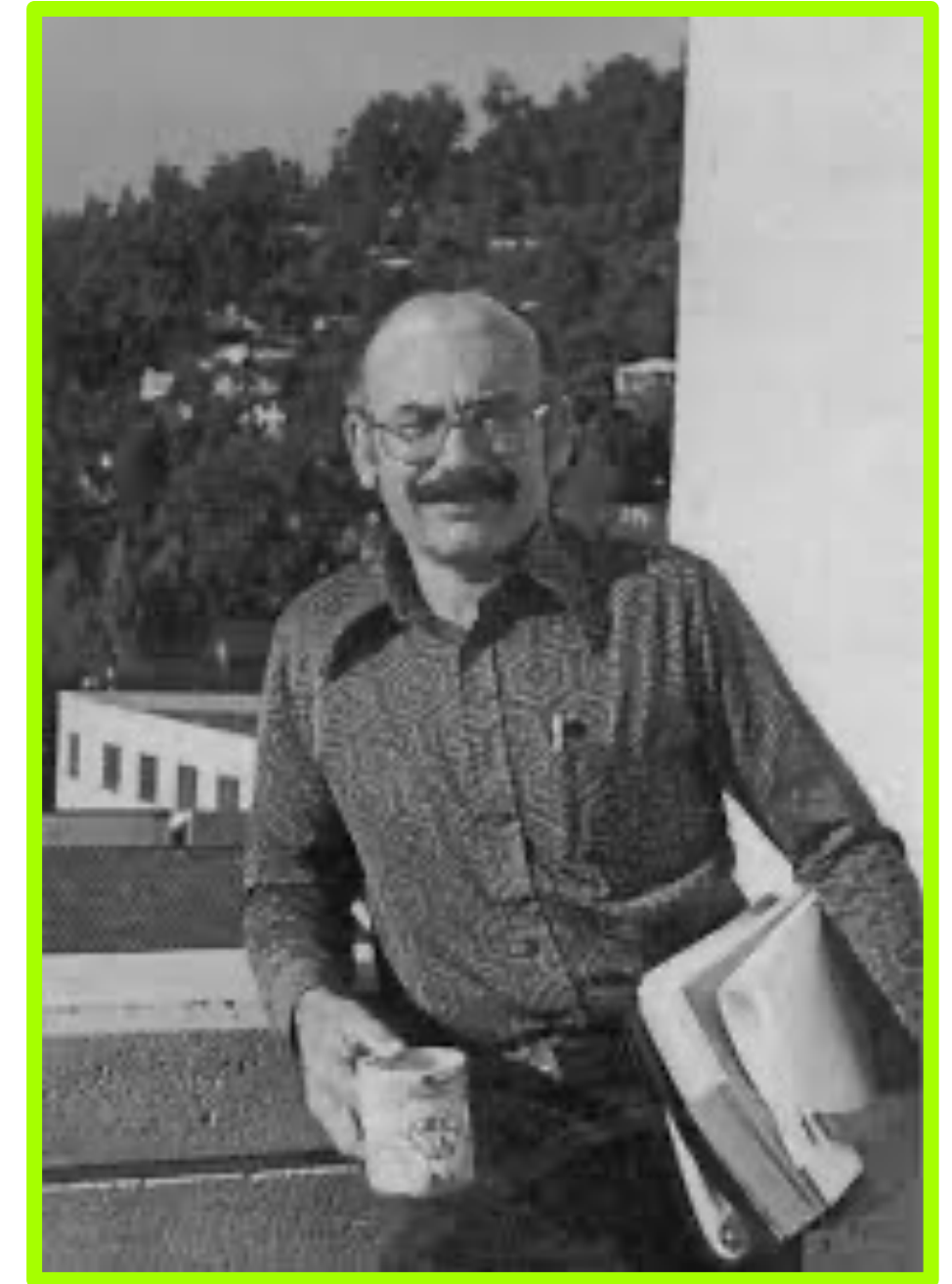
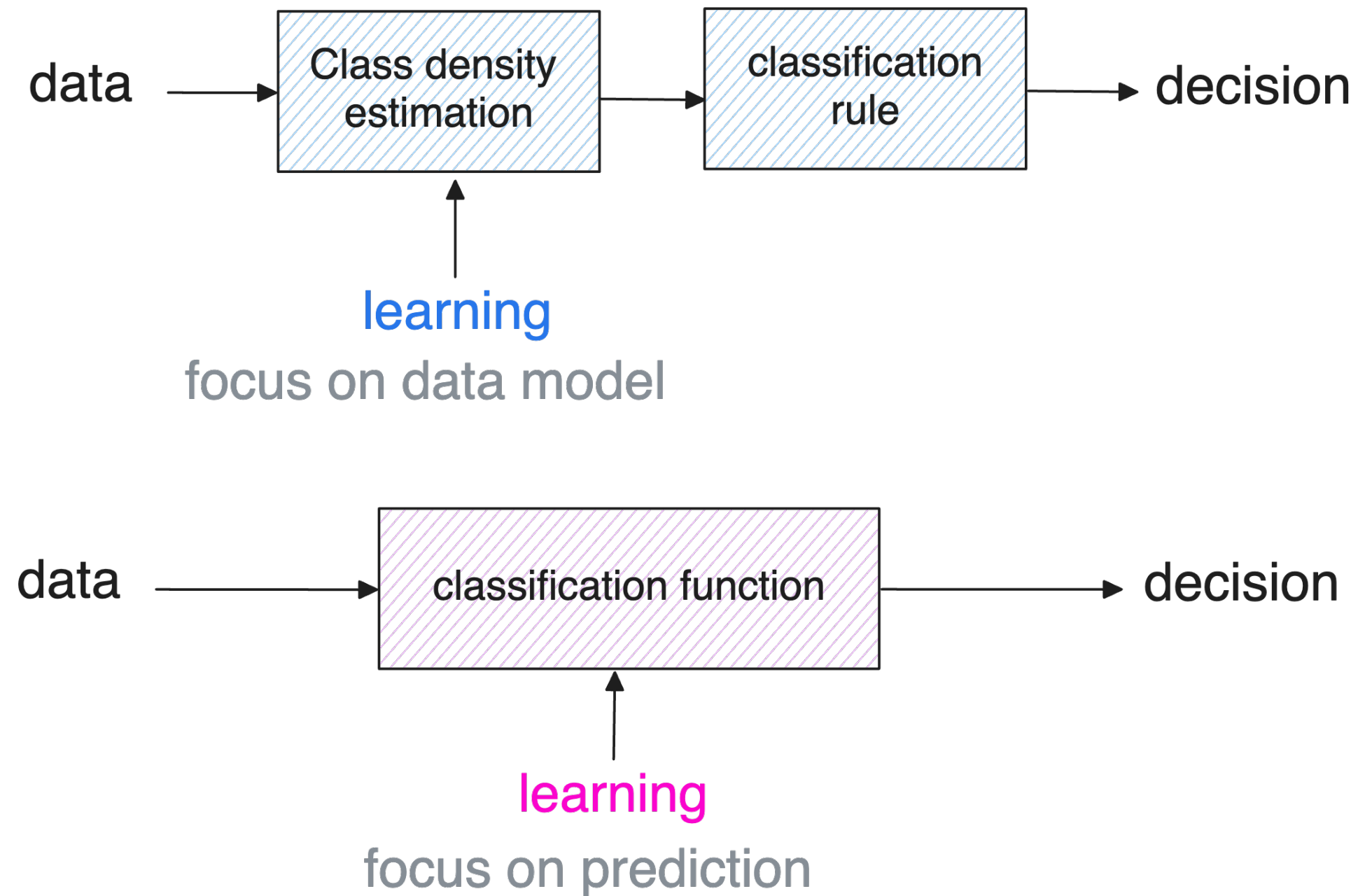
[CatBoost](#), developed by Yandex, has gained recognition for its ability to handle categorical features efficiently and its robustness against overfitting. The use of oblivious (having the same level structure) trees and CTR handling of highly cardinal features (think > 100 unique categories) are unmatched. Also, unlike other gradient boosting algorithms, CatBoost employs ordered boosting, which reduces the risk of overfitting.

Boosted scorecards, which combine the interpretability of traditional scorecards with the accuracy of ML models, benefit significantly from CatBoost's features. The algorithm's ability to handle categorical data natively means that scorecards can be developed with fewer preprocessing steps, effectively preserving the patterns of the original data.

Appendix

The Two Cultures

Leo Breiman's view on statistical modeling



Leo Breiman (1925-2005)

The Changing Role of Statistics in the Age of Computing: Why Statisticians Lost to AI and Machine Learning (according to J. Friedman)

Jerry Friedman makes a powerful point about how **the computer revolution reshaped statistics** (2001). In the past, data collection was costly, so statisticians focused on carefully designed experiments.

- Data collection used to be expensive → careful experiment design.
- The computer revolution made data cheap and abundant.
- Each 10× leap in tech forces us to rethink methods.
- We [statisticians] are no longer the only game in town.

Much earlier, in the 1950s, D.J. Finney wrote to R.A. Fisher about computing. For Fisher, the computer should be a tool to check whether approximations were accurate, not a substitute for scientific thinking. He feared that if scientists simply handed data over to machines, they risked becoming servants of the machine rather than interpreters of their own hypotheses.

The Role of Statistics in Data Revolution? ([link](#)).

The Role of Statistics in the Data Revolution?

Jerome H. Friedman

Department of Statistics, Stanford University, Stanford CA94305, USA

Summary

The nature of data is rapidly changing. Data sets are becoming increasingly large and complex. Modern methodology for analyzing these new types of data are emerging from the fields of Data Base Management, Artificial Intelligence, Machine Learning, Pattern Recognition, and Data Visualization. So far Statistics as a field has played a minor role. This paper explores some of the reasons for this, and why statisticians should have an interest in participating in the development of new methods for large and complex data sets.

Key words: Data analysis; Data mining; Decision support; Computational statistics.

1 Introduction

The theme of The 29th Symposium on the Interface (May 1997, Houston, TX) was Data Mining and the analysis of large data sets. It is perhaps a coincidence that almost exactly 20 years before a “Conference on the Analysis of Large Complex Data Sets” was held in neighboring Dallas, organized by Leo Breiman, and sponsored by the ASA and IMS(!). It seems appropriate now, over 20 years later, to ask, “How far have we come since 1977?” In some respects things have changed a lot, in others not very much.

There has been considerable change in the nature of data. In 1977 large and complex data sets were fairly rare, and little need was seen to attempt to analyze those few that did exist. This of course has changed. The computer revolution over the last 20 years has completely altered the economics of data collection. Twenty years ago most data was still collected manually. The cost of collecting it was proportional to the amount collected. This made the cost of collecting large amounts prohibitively expensive. The goal was to carefully design experiments so that maximal information could be obtained with the fewest possible measurements.

Now much (if not most) data is automatically recorded with computers. There is a very high initial cost associated with purchasing the hardware, and especially developing the software, that is incurred before any data at all is taken. After the system is set up and working, the incremental expense of taking the data is only proportional to the cost of the magnetic medium on which it is recorded. This cost has been exponentially decreasing with time. Thus one tries to record as much data as possible to amortize the high initial set-up cost. The result has been the proliferation of very large databases in terms of both the number of observations, and the number of measurements recorded for each one.

This idea of learning from data has been around for a long time. So it is reasonable to ask why the interest in analyzing these large and complex data sets has recently become so intense. The principal reason is that the field of Data Base Management has become involved. Data, especially large amounts of it, reside in data base management systems (DBMS). Conventional DBMS are focused on on-line transaction processing (OLTP); that is, the storage and fast retrieval of individual records for purposes of data organization. They are used to keep track of inventory, pay-roll records, billing records, invoices, etc.